

Universidad Internacional de la Rioja (UNIR)

**Escuela Superior de Ingeniería y
Tecnología**

**Máster Universitario en Desarrollo y Operaciones
(DevOps)**

Detección y mitigación de vulnerabilidades en microservicios Java mediante la integración de IA en pipelines CI/CD

Trabajo Fin de Máster

Presentado por: Marco Paúl Gutama Morocho

Jonnathan Esteban Peñaranda Sarmiento

Director: Dr. Oscar Sanjuan Martinez

Ciudad: Logroño

Fecha: 23 de julio de 2025

Resumen

Las prácticas DevOps y las arquitecturas de microservicios, si bien aceleran el desarrollo de software, introducen complejos desafíos de seguridad. Esto crea una brecha crítica entre la detección de vulnerabilidades y su efectiva corrección. Las herramientas tradicionales suelen generar alertas sin contexto, sobrecargando a los desarrolladores y ralentizando la mitigación.

Para resolver esto, se implementó un pipeline de DevSecOps en un entorno reproducible con Docker y Jenkins. La solución, definida como Pipeline como Código en un `Jenkinsfile`, utiliza un script de Python para consultar un modelo de Inteligencia Artificial (IA) generativa que analiza código Java, detecta vulnerabilidades y genera sugerencias de corrección detalladas. Un `Quality Gate` estricto detiene automáticamente el pipeline si se encuentran fallos críticos, previniendo despliegues inseguros.

La evaluación, realizada sobre una aplicación Java vulnerable, demostró la eficacia del sistema. El prototipo identificó correctamente vulnerabilidades críticas como Inyección SQL y Ejecución Remota de Código, y el `Quality Gate` detuvo el pipeline como se esperaba, marcando el build como `FAILURE`.

La principal contribución es la validación de un enfoque de asistencia inteligente para la seguridad. En lugar de una corrección automática riesgosa, el sistema empodera a los desarrolladores con conocimiento accionable para reducir el tiempo medio de remediación (MTTR). Se concluye que esta integración de la IA fomenta una cultura DevSecOps eficaz, transformando la seguridad de un obstáculo reactivo a una capacidad proactiva e integral del ciclo de vida del desarrollo.

Palabras clave: DevSecOps, CI/CD, Inteligencia Artificial, Análisis de Vulnerabilidades, Jenkins, Microservicios, Seguridad de Aplicaciones.

Abstract

DevOps practices and microservice architectures, while accelerating software development, introduce complex security challenges. This creates a critical gap between vulnerability detection and effective remediation. Traditional tools often generate context-poor alerts, overwhelming developers and slowing down mitigation.

To solve this, a DevSecOps pipeline was implemented in a reproducible environment with Docker and Jenkins. The solution, defined as Pipeline as Code in a `Jenkinsfile`, uses a Python script to query a generative Artificial Intelligence (AI) model that analyzes Java code, detects vulnerabilities, and generates detailed remediation suggestions. A strict `Quality Gate` automatically halts the pipeline if critical flaws are found, preventing insecure deployments.

Evaluation on a vulnerable Java application demonstrated the system's effectiveness. The prototype correctly identified critical vulnerabilities like SQL Injection and Remote Code Execution, and the `Quality Gate` halted the pipeline as expected, marking the build as `FAILURE`.

The main contribution is the validation of an intelligent assistance approach to security. Instead of risky automated correction, the system empowers developers with actionable knowledge to reduce the Mean Time to Remediate (MTTR). We conclude that this AI integration fosters an effective DevSecOps culture, transforming security from a reactive bottleneck into a proactive and integral capability of the development lifecycle.

Keywords: DevSecOps, CI/CD, Artificial Intelligence (AI), Vulnerability Analysis, Application Security, Jenkins, Microservices, Pipeline as Code.

Índice general

1. Introducción	1
1.1. Justificación del trabajo	2
1.2. Planteamiento del problema	3
1.3. Estructura de la memoria	4
2. Contexto y estado del arte	5
2.1. Contextualización y antecedentes	5
2.2. Trabajos relacionados	6
2.2.1. Herramientas de Análisis de Seguridad y el Factor Humano	6
2.2.2. Aplicaciones de Inteligencia Artificial en Ingeniería de Software y Seguridad	7
2.3. Conclusiones del estado del arte	8
3. Objetivos y metodología de trabajo	10
3.1. Objetivo general	10
3.2. Objetivos específicos	10
3.3. Metodología del trabajo	12
4. Diseño e Implementación	14
4.1. Planificación, Análisis y Requisitos	14
4.1.1. Análisis Profundo del Problema en un Contexto Industrial	14
4.1.2. Requisitos Funcionales del Sistema	15
4.1.3. Requisitos No Funcionales del Sistema	17
4.1.4. Selección y Justificación Detallada de Tecnologías	18
4.2. Descripción del Sistema Desarrollado e Implementación	22
4.2.1. Arquitectura Detallada del Sistema	22
4.2.2. El Entorno de Ejecución Automatizado y Contenerizado	24
4.2.3. Orquestación del Servicio: El Archivo <code>docker-compose.yml</code>	26
4.2.4. Implementación Detallada del Pipeline en Jenkinsfile	27

4.2.5. El Módulo de Análisis con Inteligencia Artificial	29
4.2.6. Desafíos y Soluciones de Implementación	35
4.2.7. Resumen de la Implementación y Transición a la Evaluación	38
5. Evaluación y Resultados	39
5.1. Evaluación	39
5.1.1. Entorno de Pruebas y Análisis de Vulnerabilidades	40
5.1.2. Modelos y Herramientas de Análisis	43
5.1.3. Análisis Comparativo de Resultados	44
5.1.4. Discusión General y Conclusiones de la Evaluación	48
5.2. Resultados de la Ejecución del Pipeline	49
6. Conclusiones y Trabajo Futuro	55
6.1. Conclusiones	55
6.1.1. Análisis Comparativo	56
6.2. Líneas de Trabajo Futuro	58
Bibliografía	60
Anexos	62
A. Archivo Docker Compose	63
B. Archivo Jenkinsfile	64
C. Archivo ai_analyzer.py	72
D. Reporte de vulnerabilidades mediante IA	94

Índice de figuras

1.	Diagrama de componentes del sistema.	23
2.	Diagrama de secuencia del flujo de trabajo DevSecOps.	23
3.	Diagrama de despliegue del entorno.	24
4.	Ejecución fallida del pipeline.	50
5.	Despliegue exitoso de la etapa de despliegue.	53
6.	Ejecución exitosa del pipeline.	54

Índice de Tablas

1.	Detección Detallada de Vulnerabilidades por Herramienta	44
2.	Comparativa de Rendimiento y Consumo de Tokens	47
3.	Tabla Comparativa: Solución Propuesta vs. Herramientas de Evaluación	57

Capítulo 1

Introducción

La transformación digital ha consolidado a las arquitecturas de microservicios y las metodologías DevOps como pilares fundamentales en el desarrollo de software moderno. Esta combinación permite a las organizaciones construir aplicaciones escalables y entregar valor de forma rápida y continua. Sin embargo, esta agilidad introduce nuevos y complejos desafíos de seguridad. La adopción de microservicios, si bien popular, incrementa la superficie de ataque de las aplicaciones (Zafeiropoulos et al., [2023](#)), y la automatización inherente a los pipelines de Integración y Despliegue Continuo (CI/CD), un pilar de DevOps, a menudo introduce riesgos si no se gestiona adecuadamente (National Institute of Standards and Technology (NIST), [2024](#)).

Frente a esta problemática, la propuesta principal de esta investigación se enfoca en una solución innovadora: la integración de modelos de Inteligencia Artificial (IA) existentes directamente en los flujos de trabajo de CI/CD. Este enfoque busca mejorar radicalmente la forma en que se detectan y tratan las vulnerabilidades en el código Java de los microservicios. Es crucial aclarar que el objetivo no es la corrección automática de código, lo cual podría ser riesgoso, sino la generación automatizada de sugerencias de mitigación contextualizadas y de alta calidad. Estas sugerencias se presentarían al equipo de desarrollo para su revisión y validación, asegurando el control humano sobre cambios críticos.

Este capítulo introduce el problema, justifica la necesidad de una solución y plantea la pregunta de investigación que guía el trabajo. Finalmente, se presenta la estructura del documento, detallando los capítulos que componen la tesis para facilitar la comprensión del proceso de investigación, desarrollo y los resultados obtenidos.

1.1. Justificación del trabajo

La adopción de arquitecturas de microservicios se ha vuelto una tendencia principal en la industria del software, motivada por la búsqueda de mayor agilidad en el desarrollo, escalabilidad independiente de los componentes y mejor resiliencia del sistema. No obstante, esta descomposición en servicios añade una complejidad importante a la gestión de la seguridad. El problema fundamental que esta tesis busca resolver es la ineficiencia y la tendencia a errores de los métodos tradicionales para identificar y corregir vulnerabilidades en el amplio y dinámico entorno de microservicios. La naturaleza distribuida de estos sistemas incrementa la superficie de ataque, y la posible diversidad de tecnologías y equipos dificulta la aplicación de políticas de seguridad uniformes.

Las causas de este problema son diversas: la velocidad que exigen los ciclos de desarrollo ágil y DevOps a menudo deja poco margen para revisiones de seguridad manuales detalladas; la complejidad propia de las interacciones entre servicios puede ocultar vulnerabilidades sutiles; y la gran escala (cientos o miles de microservicios en organizaciones grandes) hace que la supervisión manual sea impracticable. La importancia de solucionar este problema es crítica, ya que las fallas de seguridad pueden tener consecuencias graves como pérdida de datos, daño a la reputación o incumplimiento de normativas. Ignorar estas alertas debido a la sobrecarga de información no es una opción, sino un riesgo latente que este trabajo busca mitigar.

La motivación principal de este trabajo nace de reconocer el potencial de la Inteligencia Artificial para enfrentar estos desafíos. La capacidad de los modelos de IA actuales para analizar grandes volúmenes de código, identificar patrones de vulnerabilidades y generar sugerencias de código coherentes ofrece una oportunidad única. Integrar esta capacidad en los pipelines CI/CD es un paso lógico hacia un enfoque de seguridad más inteligente y proactivo (DevSecOps). Por lo tanto, la contribución principal de esta tesis es el diseño, implementación y validación de un sistema modular que integra IA en pipelines CI/CD para generar sugerencias de mitigación de vulnerabilidades, buscando superar las limitaciones de las herramientas tradicionales en cuanto a contexto y accionabilidad.

Adicionalmente, se busca que la solución propuesta sea de código abierto, ofreciendo una alternativa a las herramientas comerciales existentes que pueden tener un alto costo (GitHub, Inc., [2024](#); SonarSource S.A., [2024](#)) o a las versiones comunitarias con funcionalidades limitadas. Un diferenciador clave del prototipo es su diseño modular, que permitirá al usuario seleccionar el modelo de IA subyacente, adaptando así la solución a diferentes contextos de presupuesto y privacidad.

1.2. Planteamiento del problema

En el ciclo de vida actual del desarrollo de software, especialmente en organizaciones que utilizan DevOps y microservicios Java, se observa una importante brecha operativa. Existe una necesidad urgente de mecanismos que no solo detecten vulnerabilidades de forma temprana y continua (aplicando el principio "Shift Left"), sino que también ofrezcan a los equipos de desarrollo una guía clara, contextualizada y fiable para su corrección, integrada de forma fluida en sus flujos de trabajo automatizados.

Las soluciones actuales, aunque han progresado, presentan limitaciones. Las herramientas de Análisis Estático de Seguridad (SAST), si bien son esenciales, frecuentemente generan un gran volumen de alertas, incluyendo una cantidad significativa de falsos positivos. Este ruido "dificulta la priorización, consume tiempo valioso de los desarrolladores y puede llevar a una "fatiga de alertas" que reduce la confianza en la herramienta (Johnson et al., 2023). La clasificación de estas alertas y la investigación de la mitigación adecuada sigue siendo un proceso manual intensivo, actuando como un cuello de botella que ralentiza el ciclo de retroalimentación.

Para responder a esta necesidad, se propone desarrollar un sistema que utiliza modelos de Inteligencia Artificial (IA) para el análisis integral de código, integrado como una etapa dentro del pipeline CI/CD. La finalidad de esta tesis es diseñar, construir y evaluar un prototipo funcional de este sistema. El prototipo enviará el código fuente a un modelo de IA seleccionado para un análisis exhaustivo que no solo identificará vulnerabilidades, sino que también evaluará la calidad general del código. Para validar su eficacia, los resultados se compararán con los de SonarQube, una herramienta de referencia industrial. Finalmente, los hallazgos y sugerencias de la IA se presentarán al desarrollador mediante reportes claros, generados como artefactos del pipeline.

Es fundamental reiterar que la propuesta se centra en la asistencia inteligente y no en la corrección automática. Se busca capacitar al desarrollador con información útil y de alta calidad, permitiéndole tomar decisiones informadas y mantener el control humano sobre el código.

Con base en lo anterior, la pregunta de investigación que guía este trabajo es: ¿En qué medida puede un sistema basado en modelos de IA, integrado en un pipeline CI/CD, complementar y enriquecer los resultados de una herramienta de análisis estático de seguridad (SAST) tradicional como SonarQube, mejorando la accionabilidad de las alertas y la calidad de las sugerencias de mitigación en microservicios Java?

1.3. Estructura de la memoria

Para presentar de manera lógica y completa la investigación, los desarrollos y los resultados, este documento de tesis se organiza en los siguientes capítulos:

1. **Introducción:** Presenta el tema central de la integración de IA en CI/CD para la seguridad de microservicios Java. Justifica la importancia del problema en el contexto de DevOps y microservicios, plantea la necesidad detectada y la solución propuesta, y describe la organización del resto del documento.
2. **Contexto y estado del arte:** Proporciona el fundamento teórico y tecnológico. Profundiza en las arquitecturas de microservicios, sus implicaciones para la seguridad, y el papel de los pipelines CI/CD en DevOps. Revisa herramientas existentes para análisis de vulnerabilidades (SAST, DAST, SCA) y examina trabajos de investigación y soluciones relevantes que aplican IA a la seguridad del software, análisis de código u optimización de procesos DevOps. Busca identificar limitaciones actuales y destacar la originalidad de esta tesis.
3. **Objetivos y metodología de trabajo:** Formaliza el alcance y plan de acción. Enuncia el objetivo general y lo desglosa en objetivos específicos medibles. Describe la metodología de investigación y desarrollo, especificando fases, tareas, criterios de selección de tecnologías y métodos de evaluación del prototipo.
4. **Diseño e Implementación:** Es el núcleo técnico. Presenta la arquitectura detallada de la solución, explicando la integración de componentes. Justifica las decisiones de diseño y describe la implementación del prototipo, incluyendo la configuración de herramientas, la definición del Pipeline como Código (Jenkinsfile) y la lógica de interacción con la IA.
5. **Evaluación y Resultados:** Presenta la evaluación empírica del prototipo. Describe experimentos, casos de prueba y métricas de rendimiento y efectividad. Analiza los resultados, discutiendo la calidad y utilidad de las sugerencias de la IA.
6. **Conclusiones y Trabajo Futuro:** Sintetiza los hallazgos. Resume contribuciones, discute limitaciones y propone líneas de investigación futuras basadas en los resultados.

Capítulo 2

Contexto y estado del arte

Este capítulo se dedica a establecer un entendimiento profundo del dominio en el que se enmarca la tesis, proporcionando el contexto necesario para apreciar la relevancia y la innovación de la propuesta. Se exploran los conceptos fundamentales de microservicios, CI/CD y seguridad en Java, y se realiza un análisis crítico de las soluciones existentes y la investigación académica relacionada, identificando las brechas que este trabajo pretende abordar.

2.1. Contextualización y antecedentes

El paradigma de microservicios ha transformado de manera importante la forma en que se diseñan, desarrollan y operan aplicaciones complejas en la industria del software, presentándose como una alternativa a las arquitecturas monolíticas tradicionales (Zafeiropoulos et al., [2023](#)). Esta arquitectura divide las aplicaciones en un conjunto de servicios pequeños, autónomos y que se pueden desplegar de forma independiente, cada uno enfocado en una capacidad de negocio específica. Esta granularidad se alinea con los principios DevOps, lo que facilita una mayor agilidad, ciclos de despliegue más rápidos y escalabilidad granular. Sin embargo, la naturaleza distribuida de los microservicios presenta desafíos importantes, especialmente en el área de la seguridad (Zafeiropoulos et al., [2023](#); Al-Dhuraibi et al., [2022](#)). La comunicación entre servicios, que usualmente ocurre a través de APIs sobre la red, aumenta la superficie de ataque y la complejidad en la gestión de la seguridad de extremo a extremo (Al-Dhuraibi et al., [2022](#)).

Al mismo tiempo, los pipelines de CI/CD (Integración Continua y Despliegue/Entrega Continua) son la base de las prácticas DevOps modernas, ya que automatizan el ciclo de vida del software desde la integración del código hasta su despliegue (Santos et al., [2023](#)). Estos pipelines, gestionados por herramientas como Jenkins, GitLab CI/CD o GitHub Actions, permiten entregas de software más rápidas y fiables. En este entorno, la seguridad debe integrarse de forma temprana y continua, un

principio conocido como DevSecOps o "Shift Left Security" (Santos et al., 2023; Kumar et al., 2022).

Java, junto con frameworks como Spring Boot, continúa siendo una tecnología destacada para el desarrollo de microservicios empresariales, como confirman informes recientes del sector (New Relic, 2024). No obstante, las aplicaciones Java pueden tener diversas vulnerabilidades, como las que se detallan en el proyecto OWASP Top Ten, que incluyen inyecciones, configuraciones de seguridad incorrectas y el uso de componentes con vulnerabilidades conocidas (OWASP Foundation, 2021). Para asegurar los microservicios Java se requiere no solo conocimiento del lenguaje, sino también de patrones de seguridad específicos para sistemas distribuidos y una gestión cuidadosa de las dependencias (Finite State, 2023).

2.2. Trabajos relacionados

La seguridad del software en los flujos de DevOps ha impulsado la creación de diversas herramientas e investigaciones.

2.2.1. Herramientas de Análisis de Seguridad y el Factor Humano

La integración de herramientas de Application Security Testing (AST) es una práctica común en DevSecOps (Kumar et al., 2022). Estas herramientas se categorizan principalmente en:

- **SAST (Static Application Security Testing):**

- *Descripción:* Analizan el código fuente o binario en busca de patrones de vulnerabilidad sin ejecutar la aplicación.
- *Ejemplos:* SonarQube, Checkmarx, Veracode (SonarSource, s.f.).
- *Limitaciones:* Aunque son eficaces para la detección temprana, a menudo generan un alto volumen de falsos positivos y carecen del contexto de ejecución, lo que puede llevar a la fatiga de alertas (Johnson et al., 2023).

- **DAST (Dynamic Application Security Testing):**

- *Descripción:* Evalúan la aplicación mientras se está ejecutando, simulando ataques externos para encontrar vulnerabilidades en tiempo de ejecución.
- *Ejemplos:* OWASP ZAP, Burp Suite.
- *Limitaciones:* Requieren que la aplicación esté desplegada y funcional, por lo que la retroalimentación llega más tarde en el ciclo de vida. No tienen visibilidad del código fuente (Kumar et al., 2022).

■ SCA (Software Composition Analysis):

- *Descripción:* Se enfocan exclusivamente en identificar vulnerabilidades en las dependencias y componentes de terceros utilizados en un proyecto.
- *Ejemplos:* Snyk, OWASP Dependency-Check, Dependabot (Snyk, [s.f.](#)).
- *Limitaciones:* Su alcance no cubre las vulnerabilidades en el código escrito a medida por los desarrolladores.

El Desafío de la Carga Cognitiva y la Fatiga por Alertas

Una limitación persistente y crítica de las herramientas AST tradicionales, especialmente las de tipo SAST, no es solo técnica, sino también humana. Estos sistemas a menudo carecen de una comprensión semántica profunda del código, lo que les lleva a generar un gran número de hallazgos que no son explotables en el contexto real de la aplicación (falsos positivos). Este flujo constante de alertas de baja calidad impone una carga cognitiva significativa sobre los desarrolladores (Johnson et al., [2023](#)).

Estudios recientes sobre la usabilidad de estas herramientas confirman que los desarrolladores sufren de fatiga por alertas, un estado en el que la exposición continua a notificaciones irrelevantes les lleva a ignorar o desconfiar de todas las alertas, incluidas las críticas. Cuando la mayoría de las alertas resultan ser falsos positivos, la confianza en la herramienta disminuye y la postura de seguridad de la organización se debilita (Johnson et al., [2023](#)). Por lo tanto, el problema no es solo detectar vulnerabilidades, sino presentarlas de una manera que sea procesable, contextualizada y que minimice la carga cognitiva, un vacío que las herramientas tradicionales luchan por llenar.

2.2.2. Aplicaciones de Inteligencia Artificial en Ingeniería de Software y Seguridad

La IA, y en particular los LLMs, ha surgido como una tecnología prometedora para abordar las limitaciones de las herramientas tradicionales.

Evolución de los Modelos de Lenguaje para Código

El uso de la IA para analizar código no es nuevo, pero ha evolucionado drásticamente. Los primeros enfoques utilizaban modelos especializados entrenados en vocabularios de código, como CodeBERT, que aprendían representaciones de lenguajes de programación para tareas como la búsqueda de código o la detección de vulnerabilidades (Feng et al., [2020](#)). Sin embargo, el avance más significativo ha venido con la aparición de LLMs de propósito general a gran escala, como los de las familias

GPT, Llama, Claude y Gemini. Estos modelos, entrenados con vastos corpus de texto y código, han demostrado capacidades emergentes para razonar sobre la semántica del código, su intención y sus posibles fallos, a menudo superando a los modelos especializados anteriores (Siddiq et al., 2024). Herramientas como GitHub Copilot, basadas en estos modelos, han popularizado la generación y el análisis de código asistido por IA (GitHub, s.f.-b; Wang et al., 2022).

IA para la Detección y Reparación de Vulnerabilidades

La investigación actual explora activamente el uso de LLMs para la seguridad del software, aunque con diferentes enfoques:

- **Detección Mejorada:** Se utilizan LLMs para mejorar la precisión de la detección de vulnerabilidades, intentando reducir los falsos positivos al comprender mejor el contexto del código (GitHub, s.f.-a). Sin embargo, la seguridad del propio código generado por LLMs sigue siendo un área de preocupación y estudio activo (GitGuardian, 2024).
- **Reparación Automática de Programas (APR):** Este es uno de los campos más ambiciosos, donde el objetivo es que la IA genere automáticamente un parche que corrija un error o una vulnerabilidad. Aunque se han logrado avances significativos y los LLMs superan a menudo a las técnicas de APR tradicionales (Liu et al., 2024), la fiabilidad y la corrección semántica de los parches generados siguen siendo un desafío. Un parche puede corregir una vulnerabilidad pero introducir otra, o romper la funcionalidad existente (Fu et al., 2023). Estas limitaciones de confianza restringen severamente su adopción en entornos de producción críticos.
- **Integración en DevOps (AIOps):** A un nivel más amplio, AIOps busca aplicar la IA para optimizar las operaciones de TI y los flujos de DevOps, como la detección de anomalías en logs o la optimización de la configuración de pipelines, un mercado en rápida evolución y de alto interés estratégico para las organizaciones (Gartner, Inc., 2023).

2.3. Conclusiones del estado del arte

El análisis del estado del arte revela dos realidades paralelas. Por un lado, las herramientas de DevSecOps tradicionales (SAST, DAST, SCA) son maduras y efectivas en la detección, pero fallan en el aspecto humano: abruma a los desarrolladores con alertas de bajo contexto, creando una barrera para la mitigación eficiente (Johnson et al., 2023). Por otro lado, los avances en IA y LLMs ofrecen un enorme potencial para comprender y razonar sobre el código, pero la investigación se ha polarizado en dos extremos: la mejora de la detección y la utopía de la reparación totalmente automática (APR), cuya fiabilidad aún no es adecuada para entornos de producción (Liu et al., 2024).

En esta intersección se encuentra la brecha investigadora que esta tesis aborda: el espacio intermedio de la asistencia inteligente para la mitigación. Existe una necesidad clara de un enfoque que aproveche el poder de razonamiento de los LLMs no para reemplazar al desarrollador, sino para empoderarlo. Un sistema que, integrado en el pipeline CI/CD, actúe como un experto en seguridad virtual, traduciendo las alertas de bajo nivel de las herramientas SAST en sugerencias de corrección de alto nivel, contextualizadas y explicadas.

La principal contribución y novedad de esta tesis es, por tanto, el diseño, la implementación y la evaluación de un sistema que ocupa este nicho. A diferencia de los sistemas APR, no intenta una arriesgada corrección automática, sino que se centra en generar retroalimentación accionable que respete el control y la supervisión del desarrollador. Este enfoque se alinea con las necesidades de control y seguridad en entornos de producción y con las directrices para cadenas de suministro de software, como las propuestas por NIST (National Institute of Standards and Technology (NIST), [2024](#)), y busca mejorar la eficiencia del ciclo DevSecOps sin sacrificar la agilidad del desarrollo (Santos et al., [2023](#)).

Capítulo 3

Objetivos y metodología de trabajo

Este capítulo es fundamental, ya que traduce la problemática y la visión general en metas concretas y un plan de acción detallado. Se define el propósito último del trabajo (objetivo general), se desglosa en pasos alcanzables y verificables (objetivos específicos) y se describe el enfoque metodológico que se seguirá para llevar a cabo la investigación y el desarrollo.

3.1. Objetivo general

El objetivo general de esta tesis de maestría es diseñar, implementar y evaluar un prototipo funcional que integre modelos de Inteligencia Artificial existentes dentro de pipelines de Integración Continua y Despliegue Continuo (CI/CD). El fin es detectar vulnerabilidades en microservicios desarrollados en Java y generar automáticamente sugerencias de mitigación contextualizadas para dichas vulnerabilidades.

Al alcanzar este objetivo, se busca demostrar la viabilidad y el valor de utilizar la IA como un asistente inteligente para los desarrolladores en tareas de seguridad críticas dentro de un marco DevOps. El impacto esperado no es solo técnico (un prototipo funcional), sino también práctico y estratégico. Se espera contribuir a la mejora proactiva de la seguridad del software, a la automatización eficiente de procesos DevSecOps, a la aceleración del ciclo de desarrollo mediante la reducción del esfuerzo manual de corrección, y a la exploración de aplicaciones innovadoras de IA en la ingeniería de software. Este objetivo se alinea con la necesidad industrial de construir y operar sistemas de software de forma más rápida y segura.

3.2. Objetivos específicos

Para lograr el objetivo general de manera estructurada y medible, se establecen los siguientes objetivos específicos:

- **Identificar y configurar herramientas de análisis de seguridad en CI/CD:** Investigar, seleccionar e integrar al menos una herramienta SAST reconocida (dando prioridad a SonarQu-
be según la propuesta inicial) dentro de un pipeline CI/CD. El propósito es establecer la base
para la detección de vulnerabilidades en código Java de microservicios de ejemplo, asegurando
que la salida de la herramienta sea procesable para el siguiente paso.
- **Seleccionar e integrar modelos de IA para análisis de código:** Investigar y evaluar la
idoneidad de diferentes modelos de IA existentes para la tarea específica de analizar fragmentos
de código Java marcados como vulnerables y generar sugerencias de corrección. Seleccionar
al menos un modelo e implementar la integración técnica (llamadas API, procesamiento de
entrada/salida) dentro del pipeline. El objetivo es establecer la capacidad de análisis inteligente.
- **Implementar el módulo de análisis y sugerencias con IA:** Implementar la lógica de
conexión dentro del pipeline que:
 - Prepare y envíe las clases y archivos de configuración del proyecto de forma adecuada al
modelo de IA seleccionado.
 - Reciba la respuesta del modelo de IA.
 - Procese y formatee las sugerencias de mitigación generadas para que sean comprensibles
y útiles.

Este es el núcleo de la funcionalidad propuesta.

- **Automatizar el flujo completo en el pipeline CI/CD:** Asegurar que todo el proceso
(Descarga de código → Compilación → Pruebas Unitarias → Análisis estático → Análisis
mediante IA → Criterios de calidad → Despliegue del servicio) se ejecute de forma automática
como parte integral del pipeline CI/CD elegido. Este proceso se activaría, por ejemplo, en cada
commit o merge request.
- **Implementar la generación de reportes accionables:** Diseñar y desarrollar un mecanismo
para generar reportes claros y concisos en formato HTML, que presenten las vulnerabilidades
detectadas junto con las correspondientes sugerencias de mitigación proporcionadas por la
IA. Estos reportes deben ser fácilmente accesibles como artefactos del pipeline para que los
desarrolladores puedan revisarlos.
- **Evaluar la efectividad y rendimiento del prototipo:** Diseñar y ejecutar un conjunto de
experimentos utilizando microservicios Java de ejemplo con vulnerabilidades conocidas. Medir
métricas clave como:

- Tasa de vulnerabilidades detectadas para las cuales se genera una sugerencia.
- Precisión y relevancia percibida de las sugerencias mediante evaluación manual.
- Impacto en el tiempo total de ejecución del pipeline.

El objetivo es validar la utilidad práctica y la viabilidad de la solución propuesta.

3.3. Metodología del trabajo

Para alcanzar los objetivos descritos de manera sistemática y rigurosa, se seguirá una metodología de investigación aplicada y desarrollo iterativo, estructurada en las siguientes fases principales:

1. Fase 1: Investigación y Selección Tecnológica (Semanas 1-3)

- Análisis profundo de vulnerabilidades Java en microservicios: Revisión bibliográfica y análisis de bases de datos de vulnerabilidades (por ejemplo, OWASP Top 10, CWE) para identificar los tipos de fallos de seguridad más comunes y relevantes.
- Evaluación y selección de herramientas SAST/CI/CD: Análisis comparativo de herramientas SAST y plataformas CI/CD. Selección final de las herramientas base.
- Exploración y selección de modelos de IA: Investigación detallada de modelos de IA. Evaluación de capacidad de análisis de Java, calidad de generación, facilidad de integración y costes.

2. Fase 2: Diseño de la Arquitectura y Casos de Prueba (Semanas 4-5)

- Diseño detallado de la arquitectura: Diagramas de despliegue, componentes y flujo de datos. Definición de interfaces de comunicación.
- Definición de casos de prueba: Desarrollo de un microservicio Java con vulnerabilidades representativas documentadas.

3. Fase 3: Desarrollo e Implementación del Prototipo (Semanas 6-10)

- Configuración del entorno CI/CD: Puesta en marcha de la plataforma elegida y configuración del pipeline base.
- Desarrollo del módulo de integración con IA: Creación de los scripts necesarios para orquestar SAST, llamadas a la IA y procesamiento de respuestas.
- Desarrollo del módulo de generación de reportes: Implementación de la lógica para generar informes HTML como artefactos del pipeline.

- Contenerización: Empaquetado de microservicios y componentes con Docker para reproducibilidad.

4. Fase 4: Validación y Evaluación (Semanas 11-12)

- Ejecución de pruebas: Ejecución del pipeline completo sobre los microservicios de prueba.
- Análisis cuantitativo: Medición de métricas clave (detección, sugerencias, tiempo de ejecución).
- Análisis cualitativo: Evaluación manual de la calidad y utilidad de las sugerencias.

Capítulo 4

Diseño e Implementación

Este capítulo representa el núcleo técnico de la tesis. Aquí, las ideas y planes de los capítulos anteriores toman forma como una solución de software real y funcional. El objetivo de este capítulo es llevar al lector a través de un viaje detallado, desde la planificación inicial hasta la implementación final y la evaluación del prototipo. Se explicará no solo qué se construyó, sino también por qué se tomó cada decisión, los problemas que surgieron y cómo se resolvieron.

La meta principal era crear un sistema que no solo encontrara fallos de seguridad en microservicios Java, sino que también ayudara activamente a los desarrolladores a solucionarlos. Para ello, se diseñó un pipeline de Integración Continua y Despliegue Continuo (CI/CD) que utiliza Inteligencia Artificial (IA) para dar sugerencias de corrección claras y útiles. Este capítulo documenta ese proceso en profundidad.

4.1. Planificación, Análisis y Requisitos

Antes de escribir una sola línea de código, es fundamental planificar. Esta sección detalla el trabajo de análisis que guió todo el desarrollo, asegurando que la solución final resolviera un problema real de manera efectiva.

4.1.1. Análisis Profundo del Problema en un Contexto Industrial

Para entender mejor el problema, imaginemos un escenario realista en una empresa de software moderna ficticia, la cual ha adoptado microservicios y DevOps para acelerar la entrega de sus productos. Su equipo de desarrollo, compuesto por 25 programadores divididos en 5 escuadrones, utiliza Java con el framework Spring Boot para construir docenas de microservicios. Utilizan Jenkins para su CI/CD y alojan su código en GitHub.

El problema de la empresa es un cuello de botella en la seguridad. Tienen un solo ingeniero

de seguridad, que está completamente desbordado. Aunque han integrado herramientas de análisis como SonarQube, estas generan cientos de alertas cada semana. Los desarrolladores, presionados por cumplir con los plazos de entrega, a menudo ignoran estos reportes por varias razones:

- **Fatiga por Alertas:** Demasiadas alertas, muchas de las cuales son falsos positivos, hacen que los desarrolladores pierdan la confianza en la herramienta.
- **Falta de Contexto:** Una alerta puede decir “Vulnerabilidad de Inyección SQL en la línea 54”, pero no explica por qué ese código es vulnerable en el contexto de la aplicación ni cuál es la forma correcta de solucionarlo usando las librerías y patrones de diseño.
- **Curva de Aprendizaje:** No todos los desarrolladores son expertos en seguridad. Investigar cómo corregir una vulnerabilidad de deserialización de Java, por ejemplo, puede llevar horas de investigación que no tienen.

El resultado es que las vulnerabilidades permanecen en el código durante semanas o incluso meses, aumentando el riesgo de un ciberataque. El ingeniero de seguridad pasa la mayor parte de su tiempo revisando manualmente el código crítico y “apagando incendios”, en lugar de trabajar en mejoras proactivas.

Este escenario es precisamente el problema que esta tesis busca resolver. La solución propuesta no busca reemplazar al ingeniero de seguridad, sino darle una herramienta poderosa que automatice la parte más tediosa de su trabajo: la generación de recomendaciones de mitigación. La idea es que el pipeline de CI/CD, enriquecido con IA, actúe como un “Asistente de seguridad virtual” para cada desarrollador.

4.1.2. Requisitos Funcionales del Sistema

Los requisitos funcionales describen lo que el sistema **debe** hacer. Se definieron los siguientes:

RF-01: Integración Nativa en un Pipeline CI/CD

Descripción: El sistema completo debe operar como una serie de etapas dentro de un pipeline de Jenkins. No debe ser una herramienta externa que requiera ejecución manual.

Justificación: Para ser adoptado en un entorno DevOps, la seguridad debe estar integrada en el flujo de trabajo existente. Forzar a los desarrolladores a usar una herramienta separada crearía fricción y reduciría la probabilidad de uso.

Criterio de Aceptación: El pipeline se define completamente en un archivo Jenkinsfile y se ejecuta automáticamente en Jenkins tras un commit en el repositorio.

RF-02: Compilación y Empaquetado de Microservicios Java

Descripción: El pipeline debe ser capaz de compilar y empaquetar un microservicio estándar de Java construido con Maven.

Justificación: Este es el paso fundamental de cualquier pipeline de CI. Sin un artefacto compilado, no se puede realizar un análisis completo ni un despliegue.

Criterio de Aceptación: La ejecución del comando 'mvn clean package' dentro del pipeline finaliza con éxito y genera un archivo 'jar' en el directorio 'target/'.

RF-03: Ejecución de Análisis de Seguridad (SAST y SCA)

Descripción: El sistema debe realizar dos tipos de análisis: Análisis de Seguridad de Código Estático (SAST) para encontrar fallos en el código propio, y Análisis de Composición de Software (SCA) para encontrar vulnerabilidades en las librerías de terceros.

Justificación: La seguridad de una aplicación depende tanto del código escrito por los desarrolladores como de las dependencias que utiliza. Cubrir ambos frentes es esencial para una estrategia de seguridad completa.

Criterio de Aceptación: El pipeline ejecuta con éxito las herramientas OWASP Dependency-Check (SCA) y SpotBugs (SAST) y genera sus respectivos archivos de reporte.

RF-04: Generación de Sugerencias de Mitigación con IA

Descripción: Por cada archivo java y de configuración, el sistema debe enviar el código a un modelo de IA a realizar el análisis de vulnerabilidades y solicitar sugerencias de corrección.

Justificación: Este es el núcleo de la contribución de la tesis. Va más allá de la detección para ofrecer una solución.

Criterio de Aceptación: Para una vulnerabilidad conocida, el sistema realiza una llamada a una API de IA y recibe una respuesta que contiene una recomendación de código para solucionarla.

RF-05: Generación de Reportes

Descripción: El sistema debe crear un reporte en formato HTML con los resultados de todos los análisis mediante IA de una manera que sea fácil de entender.

Justificación: Los desarrolladores deberían tener un único panel de control con toda la información relevante lo cual mejora drásticamente la usabilidad.

Criterio de Aceptación: Al final del pipeline, se genera un archivo 'ai-analysis-report.html' que es publicado en la página del build de Jenkins.

RF-06: Implementación de “Quality Gates”

Descripción: El sistema debe poder detener o marcar un build como inestable si la cantidad o severidad de las vulnerabilidades encontradas supera un umbral predefinido.

Justificación: Los “Quality Gates” son una práctica estándar de DevSecOps para hacer cumplir las políticas de seguridad de forma automática.

Criterio de Aceptación: Si el análisis de IA reporta una o más vulnerabilidades de severidad “ALTA”, el pipeline de Jenkins finaliza con el estado “UNSTABLE”.

RF-07: Despliegue Local “Deploy”

Descripción: El sistema debe desplegar el servicio en un contenedor java en caso de que no tenga vulnerabilidades con severidad ALTA.

Justificación: El “Deploy” es el ultimo paso del flujo del pipeline, el cual dispone operativamente el microservicio ejecutado en el pipeline.

Criterio de Aceptación: Si el umbral del Quality Gate el microservicio se desplegará de manera local, finalizando con el pipeline.

4.1.3. Requisitos No Funcionales del Sistema

Los requisitos no funcionales describen **cómo** debe ser el sistema en términos de calidad y rendimiento.

RNF-01: Rendimiento

Descripción: La adición de la etapa de análisis con IA no debe hacer que el pipeline sea excesivamente lento.

Justificación: En DevOps, la velocidad es clave. Un pipeline que tarda una hora en ejecutarse rompe el ciclo de retroalimentación rápida.

Criterio de Aceptación: El tiempo total de ejecución del pipeline en un microservicio de tamaño medio no debe aumentar en más de 15 minutos debido a la etapa de análisis con IA.

RNF-02: Usabilidad

Descripción: El reporte final debe ser intuitivo y útil para un desarrollador que no es un experto en seguridad.

Justificación: Si el reporte es confuso o difícil de usar, será ignorado. El objetivo es facilitar el trabajo del desarrollador, no complicarlo.

Criterio de Aceptación: Un desarrollador, al ver el reporte, puede identificar rápidamente las vulnerabilidades críticas y entender las acciones recomendadas sin necesidad de consultar documentación externa.

RNF-03: Seguridad

Descripción: Los secretos, como las claves de API para servicios de IA, deben manejarse de forma segura.

Justificación: Es irónico construir una herramienta de seguridad que introduce nuevas vulnerabilidades. Exponer claves de API en el código es una mala práctica grave.

Criterio de Aceptación: La clave de la API de OpenRouter no está escrita directamente en el 'Jenkinsfile' ni en ningún otro archivo del repositorio. Se gestiona a través del sistema de credenciales de Jenkins.

RNF-04: Extensibilidad

Descripción: La arquitectura debe facilitar la adición de nuevas herramientas de análisis o la sustitución de un modelo de IA por otro en el futuro.

Justificación: El panorama tecnológico cambia rápidamente. Un sistema bien diseñado debe ser adaptable en lugar de rígido.

Criterio de Aceptación: Para cambiar el modelo de IA utilizado (por ejemplo, de GPT-4o a Gemini), solo se necesita modificar una línea de código en el script de análisis, sin alterar la lógica principal del pipeline.

RNF-05: Reproducibilidad

Descripción: La ejecución del pipeline debe producir los mismos resultados independientemente de la máquina donde se ejecute.

Justificación: Esto es crucial para la fiabilidad y la depuración. Elimina el problema de "en mi máquina funciona".

Criterio de Aceptación: Todo el entorno de ejecución de Jenkins está definido y gestionado a través de contenedores Docker, especificados en un archivo 'docker-compose.yml'.

4.1.4. Selección y Justificación Detallada de Tecnologías

La elección de las herramientas adecuadas es fundamental para el éxito de un proyecto de software. A continuación se detalla cada elección tecnológica.

Orquestador CI/CD: Jenkins

Descripción: Jenkins es un servidor de automatización de código abierto que permite orquestar pipelines de CI/CD. Se ejecuta como un servicio y gestiona “agentes” que son los que realizan el trabajo de compilación y prueba.

Justificación de la Elección:

- **Estándar de la Industria:** Jenkins ha sido durante mucho tiempo una de las herramientas de CI/CD más populares, lo que significa que hay una gran comunidad, abundante documentación y una enorme cantidad de plugins.
- **Flexibilidad y Extensibilidad:** Su sistema de plugins permite integrarse con prácticamente cualquier otra herramienta. Para este proyecto, los plugins para GitHub, Docker, HTML Publisher y Maven fueron esenciales.
- **Pipeline as Code:** La capacidad de definir todo el pipeline en un ‘Jenkinsfile’ es una característica poderosa. Permite que la lógica de CI/CD sea tratada como cualquier otro código: se puede versionar, revisar y auditar.

Alternativas Consideradas:

- **GitLab CI/CD:** Es una excelente opción, pero está fuertemente integrada en el ecosistema de GitLab. Como el objetivo era una solución que pudiera funcionar con cualquier repositorio Git (en este caso, GitHub), Jenkins ofrecía una mayor neutralidad.
- **GitHub Actions:** Es la solución nativa de GitHub y es extremadamente potente. Sin embargo, para este proyecto de tesis, se prefirió Jenkins por su capacidad de auto-alojamiento, lo que da un control total sobre el entorno de ejecución y facilita la integración con herramientas locales y contenedores Docker de manera más explícita, didáctica y con un control más granular sobre la red y los volúmenes, aspecto importante para la arquitectura Docker-out-of-Docker diseñada.

Contenerización: Docker y Docker Compose

Descripción: Docker es una plataforma que permite empaquetar aplicaciones y sus dependencias en “contenedores” ligeros y portátiles. Docker Compose es una herramienta para definir y ejecutar aplicaciones Docker multi-contenedor.

Justificación de la Elección:

- **Consistencia del Entorno:** Docker garantiza que el entorno donde se ejecuta el pipeline (con Java, Maven, Python y todas sus dependencias) sea idéntico en cada ejecución. Esto elimina problemas de configuración y asegura la reproducibilidad (RNF-05).
- **Aislamiento:** Cada build de Jenkins se ejecuta en un contenedor limpio, lo que evita que los builds interfieran entre sí.
- **Orquestación Simplificada:** Docker Compose permitió definir en un solo archivo ('docker-compose.yml') todo el entorno necesario: el servidor Jenkins, una red virtual y los volúmenes para persistir los datos. Esto simplifica enormemente la configuración inicial del proyecto.

Alternativas Consideradas:

- **Máquinas Virtuales (VMs):** Las VMs ofrecen un aislamiento aún más fuerte, pero son mucho más pesadas en términos de recursos (disco, RAM, tiempo de arranque). Los contenedores son más ligeros y rápidos, lo que los hace ideales para pipelines de CI/CD.
- **Podman:** Es una alternativa a Docker que no requiere un demonio central, lo que puede ser más seguro. Sin embargo, Docker sigue siendo el estándar de la industria con un ecosistema más maduro, especialmente en lo que respecta a la integración con Jenkins.

Análisis de Código (SAST/SCA): SpotBugs y OWASP Dependency-Check

Descripción: SpotBugs es una herramienta SAST que analiza el bytecode de Java en busca de "patrones de error". OWASP Dependency-Check es una herramienta SCA que compara las dependencias del proyecto con una base de datos de vulnerabilidades conocidas (NVD).

Justificación de la Elección:

- **Código Abierto y Madurez:** Ambas son herramientas de código abierto, gratuitas y muy respetadas en la comunidad de seguridad de Java.
- **Complementariedad:** Ofrecen una cobertura de seguridad en dos frentes diferentes pero igualmente importantes: el código que escribes (SAST) y el código que importas (SCA).
- **Fácil Integración con Maven:** Ambas herramientas tienen plugins de Maven oficiales, lo que hace que su integración en el 'pom.xml' y su ejecución dentro del pipeline sean triviales.

Alternativas Consideradas:

- **SonarQube:** Es una plataforma de calidad de código muy completa que incluye SAST. De hecho, internamente utiliza SpotBugs, entre otros. Se decidió usar SpotBugs directamente para

tener un control más granular y una solución más ligera, sin la necesidad de administrar un servidor SonarQube completo.

- **Snyk:** Es una solución comercial de SCA muy potente que a menudo ofrece una mejor precisión y menos falsos positivos que Dependency-Check. Sin embargo, para un proyecto de tesis, una herramienta de código abierto como OWASP Dependency-Check era la opción más adecuada por su accesibilidad.

Integración con IA: Python, OpenRouter.ai

Descripción: Python es el lenguaje de programación dominante para tareas de IA y ciencia de datos. OpenRouter.ai es una plataforma que actúa como un “router” o “agregador” de APIs de modelos de lenguaje grandes (LLMs), permitiendo llamar a diferentes modelos de diferentes proveedores con una sola API.

Justificación de la Elección:

- **Simplicidad de Python:** Para realizar llamadas a una API REST y procesar JSON, Python con su librería ‘requests’ es increíblemente simple y eficiente.
- **Flexibilidad de OpenRouter.ai:** Esta fue una decisión estratégica clave. En lugar de atarse a un solo proveedor como OpenAI, OpenRouter permitió experimentar con múltiples modelos (GPT, Claude, Llama, Gemini, Deepseek, etc.) para encontrar el mejor equilibrio entre rendimiento, coste y calidad para esta tarea específica. Esto fue fundamental para la evaluación comparativa (sección 5.1).
- **Superación de Obstáculos Locales:** Como se menciona en los desafíos (sección 4.2.6), el plan inicial de usar modelos locales con Ollama fracasó por el alto consumo de recursos. OpenRouter resolvió este problema de inmediato, externalizando la carga computacional.
- **Adopción de un Estándar de Facto:** Una decisión de diseño fundamental en el script de Python fue utilizar la librería cliente de OpenAI. Esto significa que, aunque para las pruebas se utilizó el servicio de agregación de OpenRouter, el sistema es compatible de forma nativa con cualquier proveedor de modelos de IA que ofrezca una API compatible con el estándar de OpenAI. Esto incluye a la propia OpenAI, Azure AI, y un número creciente de otros proveedores, garantizando la máxima flexibilidad y evitando la dependencia de un único servicio.

Alternativas Consideradas:

- **Uso Directo de la API de OpenAI:** Era una opción viable, pero limitaría el prototipo a los modelos de OpenAI. OpenRouter proporcionó más flexibilidad para la investigación y la

comparación.

- **Modelos Específicos para Código (CodeLLaMA, CodeBERT):** Se consideró usar modelos más especializados. Sin embargo, los modelos de propósito general más avanzados (como GPT-4o o Gemini 1.5 Pro) han demostrado tener capacidades de razonamiento sobre código tan potentes que a menudo superan a los modelos especializados más pequeños, especialmente en tareas que requieren comprensión contextual como la seguridad. OpenRouter daba acceso a ambos tipos.

4.2. Descripción del Sistema Desarrollado e Implementación

Esta sección presenta una descripción técnica exhaustiva del prototipo construido. El código fuente completo y todos los archivos de configuración descritos a continuación se encuentran disponibles públicamente en el repositorio de GitHub del proyecto: <https://github.com/iJonnathan/TFM>. A continuación, se presenta la arquitectura, la configuración del entorno y se desglosa la implementación del pipeline paso a paso.

4.2.1. Arquitectura Detallada del Sistema

La arquitectura del sistema se puede describir desde tres perspectivas: componentes, secuencia y despliegue.

Diagrama de Componentes

El sistema está compuesto por varios componentes lógicos que interactúan entre sí.

Como muestra la Figura 1, el pipeline de Jenkins es el orquestador central que invoca a las herramientas de análisis, se comunica con el módulo de IA y finalmente produce el reporte.

Diagrama de Secuencia

Este diagrama muestra la interacción entre los componentes a lo largo del tiempo para un solo ciclo de CI/CD.

Diagrama de Despliegue

Este diagrama muestra cómo se despliegan físicamente los componentes del software en el hardware.

La Figura 3 ilustra una configuración clave: el contenedor del controlador de Jenkins tiene acceso al Docker socket del host. Esto le permite a Jenkins iniciar otros contenedores, como el 'AgentCon-

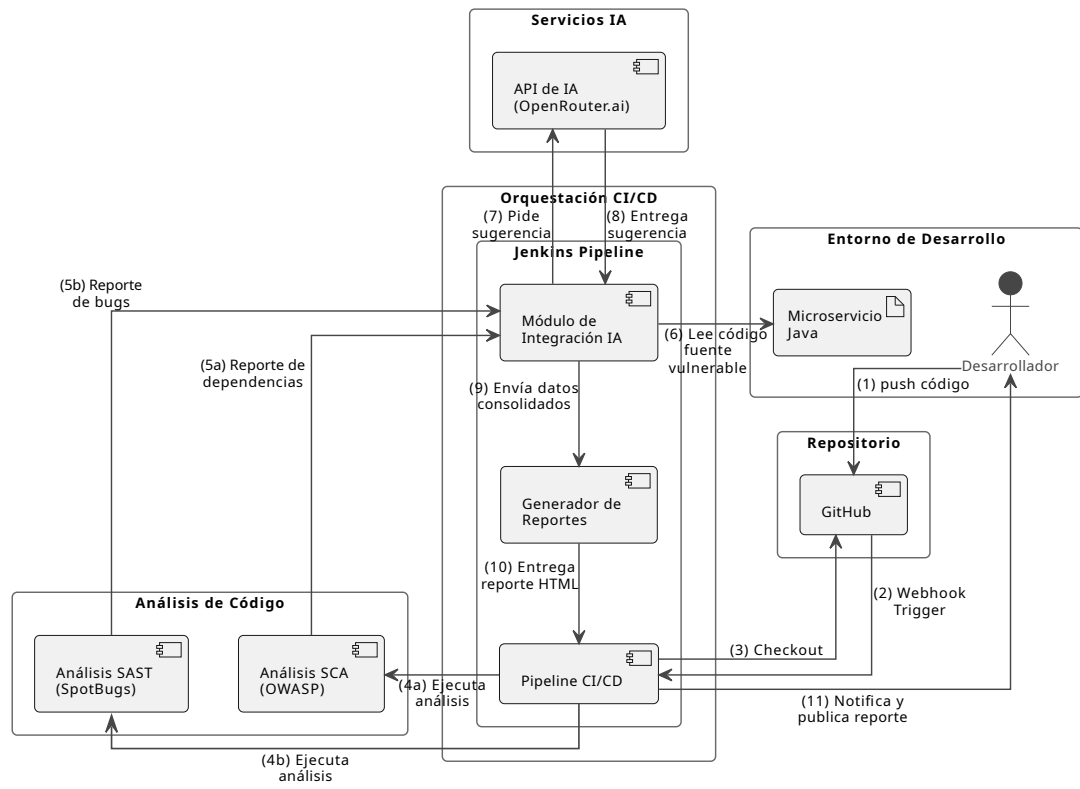


Figura 1: Diagrama de componentes del sistema.

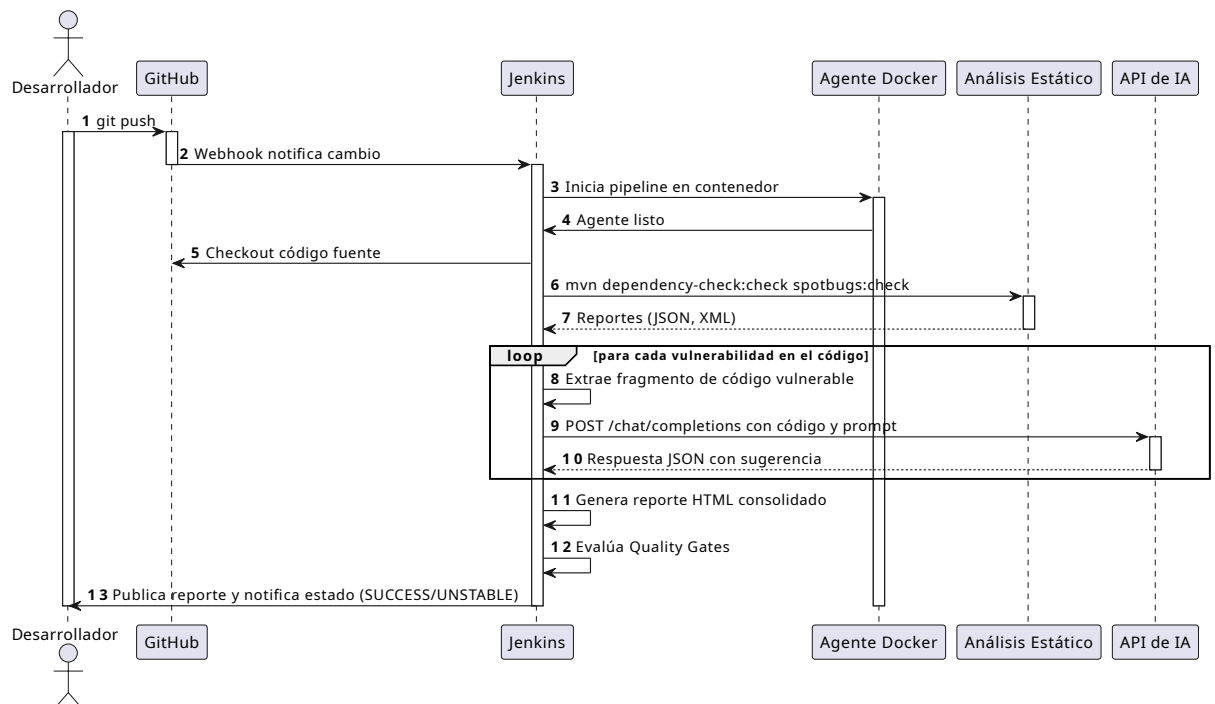


Figura 2: Diagrama de secuencia del flujo de trabajo DevSecOps.

tainer', donde se realiza todo el trabajo pesado. Esta es una configuración común y flexible conocida como "Docker-in-Docker".

A continuación se detalla cada componente de software, archivo de configuración y script de automatización, explicando la implementación práctica y la lógica que gobierna cada decisión de

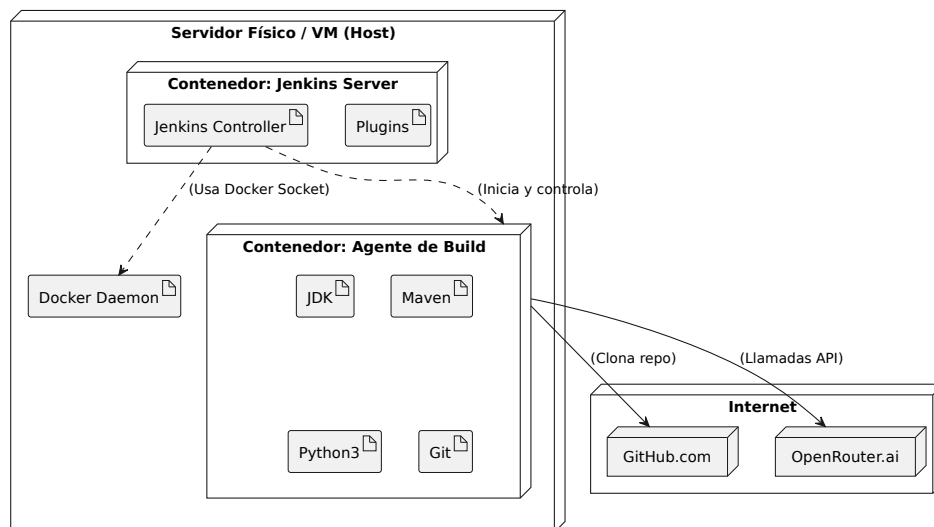


Figura 3: Diagrama de despliegue del entorno.

ingeniería.

4.2.2. El Entorno de Ejecución Automatizado y Contenerizado

La base de todo el sistema es un entorno de CI/CD consistente, portable y completamente automatizado. Este pilar se construye utilizando Docker para la contenerización y una serie de scripts para la configuración, encarnando los principios de **Infraestructura como Código (IaC)** y **Configuración como Código (CasC)**.

La Plantilla del Entorno: El Dockerfile

El 'Dockerfile' actúa como el plano de construcción para la imagen de Docker personalizada sobre la cual se ejecutará Jenkins y todas las tareas del pipeline. Cada instrucción en este archivo fue elegida deliberadamente para construir un entorno autocontenido y completo.

Imagen Base y Privilegios. El punto de partida es la imagen oficial 'jenkins/jenkins:lts', que proporciona una versión estable y de soporte a largo plazo de Jenkins. Inmediatamente, se cambia al usuario 'root' para obtener los permisos necesarios para instalar paquetes a nivel del sistema operativo.

Instalación de Dependencias del Sistema. El comando 'apt-get update & apt-get install -y ...' instala todas las herramientas que el pipeline necesitará. Cada paquete tiene un rol específico:

- **Herramientas base:** 'ca-certificates' es necesario para validar conexiones SSL/TLS, y 'curl' se utiliza para descargar archivos de internet, como la clave GPG de Docker.

- **Entorno de Compilación Java:** 'maven' y 'openjdk-17-jdk' son indispensables para compilar, probar y empaquetar la aplicación de microservicio escrita en Java y Spring Boot.
- **Entorno de Scripting Python:** 'python3' y 'python3-pip' son necesarios para ejecutar el script de análisis de IA y para instalar sus librerías externas, como 'requests'.

Instalación del Cliente de Docker. Una de las partes más importantes es la instalación del cliente de Docker ('docker-ce-cli'). El proceso sigue las mejores prácticas de seguridad recomendadas por Docker: primero se descarga y añade la clave GPG oficial del repositorio de Docker, y luego se añade el repositorio a las fuentes de 'apt'. Esto asegura que se instale un binario de Docker auténtico y permite futuras actualizaciones. Este cliente es lo que permite al contenedor de Jenkins comunicarse con el motor de Docker del sistema anfitrión.

Configuración como Código (CasC) para Jenkins. La última sección del Dockerfile se centra en la configuración automática de Jenkins:

- **Instalación de Plugins:** La línea 'RUN jenkins-plugin-cli -f /usr/share/jenkins/ref/plugins.txt' es un mecanismo clave de la Configuración como Código. Le dice a Jenkins que lea el archivo 'plugins.txt' e instale, durante la construcción de la imagen, todas las extensiones listadas. Esto elimina por completo la necesidad de instalar plugins manualmente desde la interfaz web, haciendo que la configuración sea 100 % reproducible.
- **Scripts de Inicialización:** Se copian los scripts 'groovy' al directorio 'init.groovy.d/'. Jenkins está diseñado para ejecutar cualquier script en este directorio la primera vez que arranca con un directorio '/var/jenkins_home' vacío. Esto permite una configuración cero-toque¹ del sistema.

Configuración Inicial Cero-Toque: Los Scripts de Groovy

Estos scripts de Groovy son la clave para la automatización completa de la configuración inicial de Jenkins.

El Script security.groovy. Este script se encarga de la configuración de seguridad inicial. Su diseño es idempotente, lo que significa que es seguro ejecutarlo múltiples veces.

1. Primero, obtiene la instancia actual de Jenkins con `Jenkins.get()`.
2. Luego, comprueba si el usuario 'admin' ya existe usando `instance.getUser("admin")`.

¹Proceso automatizado que permite configurar y desplegar dispositivos de forma remota y sin intervención manual del usuario.

3. Si el usuario no existe, procede a crear una configuración de seguridad. Lee la contraseña desde una variable de entorno ('JENKINS_ADMIN_PASSWORD'), crea el usuario 'admin' y establece una estrategia de autorización segura y estándar.
4. Si el usuario ya existe, el script imprime un mensaje y finaliza, sin realizar ningún cambio.

El Script `create_pipeline.groovy`. Este script automatiza la creación del trabajo o job del pipeline principal. Al igual que el de seguridad, es idempotente.

1. Comprueba si ya existe un trabajo con el nombre definido en la variable de entorno 'PIPELINE_NAME'.
2. Si no existe, crea un nuevo WorkflowJob (un pipeline).
3. La parte más importante es que configura el trabajo para usar 'CpsScmFlowDefinition'. Esto le dice a Jenkins que la definición del pipeline debe ser obtenida de un Jenkinsfile ubicado en un repositorio de Git.
4. Todos los detalles como la URL del repositorio, la rama, la ruta al Jenkinsfile se leen de variables de entorno, haciendo que la creación del pipeline sea totalmente configurable desde el exterior.
5. Finalmente, ejecuta 'instance.reload()' para forzar a Jenkins a recargar su configuración desde el disco, asegurando que el nuevo pipeline sea visible inmediatamente.

4.2.3. Orquestación del Servicio: El Archivo `docker-compose.yml`

Para hacer realidad la arquitectura de despliegue, se utilizó un único archivo 'docker-compose.yml'. Este archivo es la receta para construir todo el entorno de CI/CD localmente. Orquesta el servicio de Jenkins con varias directivas clave:

- **restart: always:** Una política de reinicio que asegura la alta disponibilidad del servicio. Si el contenedor se detiene por un error o un reinicio del sistema host, Docker lo levantará automáticamente.
- **volumes:** Define cómo se manejan los datos. El montaje del socket de Docker ('/var/run/docker.sock') habilita la arquitectura **Docker-out-of-Docker**, mientras que el volumen nombrado 'jenkins_data' asegura que todos los datos de Jenkins se almacenen de forma segura y persistente.
- **environment:** Esta sección carga las variables desde el archivo '.env'. Separar la configuración (como contraseñas o URLs) en un archivo '.env' es una práctica recomendada que permite modificar el comportamiento del sistema sin tocar los archivos de orquestación.

Para iniciar todo el entorno, un usuario solo necesita ejecutar 'docker-compose up -d --build' en la terminal, en el mismo directorio donde se encuentra este archivo. Docker se encargará de descargar la imagen de Jenkins, crear los volúmenes y lanzar el contenedor.

En el **Anexo A**, se referencia al directorio local `./jenkins_home`, el cual es indispensable ya que contiene toda la configuración, trabajos y datos de Jenkins, asegurando su persistencia entre reinicios del contenedor.

4.2.4. Implementación Detallada del Pipeline en Jenkinsfile

El archivo `Jenkinsfile` es el cerebro de la automatización de esta tesis. Define, a través de la sintaxis declarativa de Jenkins (*Pipeline as Code*), todo el flujo de trabajo de Integración Continua, Análisis de Seguridad y Despliegue Continuo (CI/Sec/CD). Su propósito es orquestar de manera automática todas las herramientas y procesos, desde que el desarrollador sube el código hasta que se genera un reporte de seguridad accionable.

Estructura y Entorno del Pipeline

La sección inicial del archivo `Jenkinsfile` es fundamental, ya que configura las dos directivas principales que controlan todo el flujo de trabajo: el **agente de ejecución** y las **variables de entorno globales**. Estas definiciones establecen dónde se ejecutará el pipeline y qué datos estarán disponibles de manera consistente para todas sus etapas.

Agente de Ejecución. La primera línea, `agent any`, es una instrucción crucial. Le dice a Jenkins que puede ejecutar este pipeline en cualquier agente de construcción que esté disponible. En el contexto de nuestro sistema, que se ejecuta enteramente dentro de un único contenedor Docker, esto significa que todas las etapas se llevarán a cabo en el propio controlador de Jenkins. Este agente ya está con todas las herramientas necesarias (Java, Maven, Python, Docker CLI) porque las instalamos directamente en el `Dockerfile`, garantizando un entorno consistente y rápido.

Variables de Entorno. El bloque `environment` se utiliza para declarar variables que estarán disponibles globalmente en todas las etapas. Esto es útil para definir constantes que se usan en múltiples lugares, evitando la repetición y facilitando futuras modificaciones. Las variables clave definidas son:

- `SCAN_RESULTS_DIR`: Define un nombre de directorio estándar ('scan-results') donde todas las herramientas de análisis guardarán sus reportes. Esto crea una ruta consistente para que las etapas posteriores puedan encontrar y procesar los artefactos generados.

- **JAVA_ANALYSIS_DIR** y **MAX_FILES_TO_ANALYZE**: Parametrizan el comportamiento del script de IA, permitiendo ajustar fácilmente el alcance del análisis.
- **AI_MODEL**: Define qué modelo de IA se utilizará, cumpliendo con el requisito de extensibilidad **RNF-04**. Cambiar de modelo es tan simple como modificar el valor de esta variable.

Flujo de Trabajo por Etapas (stages)

El corazón del pipeline se organiza en una secuencia de etapas lógicas, cada una con una responsabilidad clara.

- **Etapas Checkout & Build**: Esta etapa prepara el entorno y compila la aplicación. El comando `mvn -Dmaven.test.failure.ignore=true clean package` es deliberado: 'clean' asegura que no queden artefactos de compilaciones anteriores, 'package' compila y ejecuta las pruebas unitarias, y el parámetro '`-Dmaven.test.failure.ignore=true`' permite que el pipeline continúe al análisis de seguridad incluso si las pruebas fallan, priorizando la retroalimentación temprana de seguridad.
- **Etapas Unit Test Reports**: Utiliza el paso `junit` para procesar los reportes XML generados por Maven. Jenkins usa estos datos para mostrar un historial de resultados, visualizar tendencias y permitir un análisis detallado de los fallos.
- **Etapas 'Static Analysis'**: Para optimizar el tiempo (cumpliendo **RNF-01**), esta etapa utiliza un bloque `parallel` para ejecutar dos tareas simultáneamente. Mientras una rama ejecuta el análisis SAST con SpotBugs, la otra recopila la lista de archivos Java. Esta paralelización asegura que la lista de ficheros a analizar por la IA esté lista tan pronto como sea posible, permitiendo que la etapa de 'AI-Powered Analysis' pueda comenzar sin demoras innecesarias. El uso de `try-catch` dentro de la rama de SpotBugs añade resiliencia: un fallo en esta herramienta no detendrá el pipeline, simplemente se registrará.
- **Etapas AI-Powered Analysis**: Es el núcleo innovador del proyecto. Aquí se utiliza el bloque `withCredentials`, una práctica de seguridad fundamental (cumpliendo **RNF-03**). Este bloque inyecta la clave de la API de OpenRouter de forma segura como una variable de entorno que solo existe durante la ejecución de este bloque, evitando su exposición en los logs o en el sistema de archivos.
- **Etapas Quality Gate**: Este es el control de seguridad automatizado. Lee el archivo `analysis-results.json` generado en la etapa anterior y verifica si se cumple la política de seguridad. El uso del paso `error()` es una decisión de diseño crucial: detiene el pipeline de forma inmediata y lo

marca como `FAILURE` si se encuentra una vulnerabilidad crítica. Esto implementa una política de *cero tolerancia* que previene físicamente que un artefacto inseguro sea desplegado.

- **Etapas Build Docker Image y Local Deploy:** Estas etapas, que solo se ejecutan si el Quality Gate es superado, completan el ciclo de Despliegue Continuo. Demuestran cómo el artefacto validado se empaqueta en una imagen Docker y se despliega, dejando la aplicación funcional para pruebas o su paso a producción.

Acciones Finales y Reportes (post)

El bloque `post` es fundamental para la observabilidad y la auditoría del pipeline.

- **always:** Esta condición garantiza que, sin importar si el pipeline tuvo éxito o falló, siempre se intentará publicar los informes (`'junit'` y `'publishHTML'`) y archivar los artefactos. Esto es vital, ya que incluso si el pipeline falla, los desarrolladores necesitan acceso a los reportes para entender la causa.
- **success, failure, unstable:** Estos bloques permiten ejecutar acciones contextuales, como enviar notificaciones diferentes a diferentes canales (ej. un email en caso de fallo, un mensaje de Slack en caso de éxito), proporcionando una retroalimentación clara y concisa a los equipos.

Para una revisión exhaustiva del código, el contenido completo del `Jenkinsfile` se encuentra en el **Anexo B**.

4.2.5. El Módulo de Análisis con Inteligencia Artificial

El script de Python `ai_code_analyzer.py` actúa como el motor de análisis que orquesta la interacción entre el código fuente del proyecto y los modelos de lenguaje de gran tamaño (LLMs). Su diseño está orientado a la modularidad, la precisión y la generación de artefactos consumibles tanto por sistemas automatizados como por desarrolladores. El código fuente íntegro de este módulo se adjunta para su consulta en el **Anexo C**.

Arquitectura y Funcionamiento General

El script opera a través de una clase principal, `OpenRouterAnalyzer`, que encapsula toda la lógica de comunicación y análisis. El flujo de ejecución, orquestado por la función `main()`, sigue estos pasos:

1. **Inicialización y Configuración:** Al instanciar `OpenRouterAnalyzer`, el script lee la clave de la API desde la variable de entorno `OPENROUTER_API_KEY` y configura los encabezados HTTP

necesarios para la autenticación con el servicio de OpenRouter. Se establece el modelo de IA a utilizar según la jerarquía de prioridades definida.

2. **Descubrimiento de Ficheros:** El script escanea el directorio de destino en busca de ficheros `.java` y `pom.xml`, que son los objetivos del análisis.
3. **Análisis Iterativo:** Para cada fichero identificado, se invoca a la función de análisis correspondiente (`analyze_code_security`, `analyze_code_quality`, o `analyze_pom_security`).
4. **Comunicación con la IA:** Cada función de análisis construye un *prompt* específico y lo envía a la API de OpenRouter a través de una llamada POST. El script espera la respuesta del modelo.
5. **Procesamiento y Consolidación:** La respuesta de la IA, que se solicita en formato JSON, es parseada y validada. Los resultados de todos los ficheros se consolidan en una única estructura de datos.
6. **Generación de Reportes:** Finalmente, con todos los datos consolidados, la función `generate_report()` crea dos artefactos:
 - **analysis-results.json:** Un fichero JSON estructurado que contiene todos los hallazgos detallados. Su propósito es ser consumido por sistemas automatizados, como el Quality Gate en el pipeline de Jenkins.
 - **ai-analysis-report.html:** Un reporte HTML visualmente enriquecido, diseñado para ser fácilmente interpretable por los desarrolladores (Anexo D).

Ingeniería de Prompts: La Clave de un Análisis Preciso

La calidad y precisión del análisis dependen directamente de la calidad de las instrucciones (prompts) enviadas al modelo de IA. Se diseñaron tres prompts distintos, cada uno afinado para una tarea específica. Su estructura se basa en los siguientes principios de la ingeniería de prompts:

- **Asignación de Rol (Role-playing):** Cada prompt comienza con una directiva clara como *“Actúa como un experto en seguridad de aplicaciones Java”*. Esta técnica prepara al modelo, estableciendo el contexto y activando su base de conocimiento relevante para la tarea, lo que resulta en respuestas significativamente más precisas.
- **Instrucciones Detalladas y Contextualizadas:** En lugar de una petición genérica, se proporciona una lista numerada de los tipos de vulnerabilidades o problemas de calidad a buscar. Esto funciona como una lista de verificación para el modelo, forzándolo a evaluar el

código desde múltiples perspectivas. Además, se incluye el nombre del fichero y el código completo, permitiendo un análisis contextual en lugar de aislado.

- **Formato de Salida Estricto:** Se instruye explícitamente al modelo para que responda única y exclusivamente con un objeto JSON válido, proporcionando un ejemplo claro de la estructura deseada. Esta directiva es fundamental para la automatización, ya que garantiza que la salida pueda ser parseada programáticamente por el script sin errores. Para robustecer este proceso, la función `_call_api()` incluye lógica para extraer el bloque JSON de la respuesta, eliminando cualquier texto conversacional que el modelo pudiera añadir.
- **Petición de Soluciones Accionables:** Se solicita explícitamente que, de ser posible, se incluya un ejemplo de código corregido (`code_correction_suggested`). Esto va más allá de la simple detección, proveyendo un valor inmenso al desarrollador al ofrecerle una solución tangible y educativa.

A continuación, se presentan los prompts exactos utilizados en el script para cada tipo de análisis.

```
1 Actúa como un experto en seguridad de aplicaciones Java. Analiza el siguiente código fuente para identificar vulnerabilidades de seguridad:
2
3 Archivo: {filename}
4
5 Busca específicamente:
6 1. Inyección SQL
7 2. Cross-Site Scripting (XSS)
8 3. Problemas de autenticación/autorización
9 4. Validación de entrada insuficiente
10 5. Exposición de información sensible
11 6. Vulnerabilidades de deserialización
12 7. Uso inseguro de criptografía
13 8. Path traversal
14 9. Command injection
15 10. Problemas de gestión de sesiones
16
17 Código a analizar:
18 ```java
19 {code_content}
20 ```
21
22 Responde en formato JSON con la siguiente estructura:
23 {{
24     "vulnerabilities": [
```

```

25     {{
26         "type": "tipo de vulnerabilidad",
27         "severity": "HIGH|MEDIUM|LOW",
28         "line": "número de línea aproximado",
29         "description": "descripción detallada",
30         "recommendation": "cómo solucionarlo",
31         "code_correction_suggested": "codigo para solucionar la
vulnerabilidad",
32         "cwe_id": "CWE-XXX si aplica",
33         "impact": "impacto potencial de la vulnerabilidad"
34     }}
35 ],
36 "security_score": "puntuación del 0-10",
37 "summary": "resumen ejecutivo de los hallazgos"
38 }}

```

Listing 4.1: Prompt para el análisis de seguridad de ficheros Java.

```

1 Actúa como un experto en calidad de código Java. Analiza el siguiente código
  para identificar problemas de calidad:
2
3 Archivo: {filename}
4
5 Evalúa:
6 1. Code smells y anti-patrones
7 2. Complejidad ciclomática alta
8 3. Duplicación de código
9 4. Problemas de mantenibilidad
10 5. Violaciones de principios SOLID
11 6. Uso inadecuado de patrones de diseño
12 7. Gestión de excepciones
13 8. Nomenclatura y convenciones
14 9. Eficiencia y rendimiento
15 10. Cobertura y calidad de comentarios
16
17 Código a analizar:
18 ```java
19 {code_content}
20 ```
21
22 Responde en formato JSON:
23 {{
24     "quality_issues": [

```

```

25     {{
26         "type": "tipo de problema",
27         "severity": "HIGH|MEDIUM|LOW",
28         "line": "número de línea aproximado",
29         "description": "descripción del problema",
30         "recommendation": "mejora sugerida",
31         "code_correction_suggested": "codigo para solucionar el problema",
32         "category": "Maintainability|Reliability|Performance|Documentation",
33         "effort": "tiempo estimado para solucionar (minutos)"
34     }}
35 ],
36 "quality_score": "puntuación del 0-10",
37 "maintainability_index": "índice de mantenibilidad",
38 "complexity_score": "puntuación de complejidad",
39 "technical_debt": "deuda técnica estimada en minutos"
40 }}

```

Listing 4.2: Prompt para el análisis de calidad de ficheros Java.

```

1 Actúa como un experto en seguridad de proyectos Java Maven. Analiza el siguiente
  archivo POM para identificar vulnerabilidades de seguridad y problemas de
  configuración:
2
3 Archivo: {filename}
4
5 Busca específicamente:
6 1. Dependencias con vulnerabilidades conocidas (CVE)
7 2. Versiones desactualizadas de dependencias críticas
8 3. Dependencias no firmadas o de repositorios inseguros
9 4. Configuraciones inseguras de plugins
10 5. Exposición de información sensible en propiedades
11 6. Configuraciones de Maven que pueden introducir vulnerabilidades
12 7. Dependencias con scopes inapropiados
13 8. Plugins sin control de versiones
14 9. Repositorios HTTP (no HTTPS)
15 10. Dependencias snapshot en producción
16
17 Contenido del POM:
18 ```xml
19 {pom_content}
20 ```
21
22 Responde en formato JSON con la siguiente estructura:

```

```
23 {{
24     "vulnerabilities": [
25         {{
26             "type": "tipo de vulnerabilidad",
27             "severity": "HIGH|MEDIUM|LOW",
28             "line": "número de línea aproximado o sección",
29             "description": "descripción detallada",
30             "recommendation": "cómo solucionarlo",
31             "code_correction_suggested": "configuración XML para solucionar",
32             "cwe_id": "CVE-XXXX si aplica",
33             "impact": "impacto potencial de la vulnerabilidad",
34             "dependency": "nombre de la dependencia si aplica"
35         }}
36     ],
37     "security_score": "puntuación del 0-10",
38     "summary": "resumen ejecutivo de los hallazgos del POM",
39     "outdated_dependencies": [
40         {{
41             "dependency": "nombre de la dependencia",
42             "current_version": "versión actual",
43             "latest_version": "versión más reciente recomendada",
44             "security_risk": "nivel de riesgo de seguridad"
45         }}
46     ]
47 }}
```

Listing 4.3: Prompt para el análisis de seguridad de ficheros POM.

Uso y Configuración del Script

El script fue diseñado para ser flexible y fácilmente integrable en flujos de trabajo de CI/CD. Su configuración se gestiona mediante una jerarquía de prioridades que ofrece múltiples formas de invocarlo.

Jerarquía de Prioridades para el Modelo de IA. El modelo a utilizar se determina con el siguiente orden de precedencia:

1. **Argumento de línea de comandos:** El uso del flag `-model` (o `-mo`) en la terminal tiene la máxima prioridad.
2. **Variable de Entorno:** Si no se especifica el argumento, el script buscará la variable de entorno `AI_MODEL`.

3. **Valor por Defecto:** Si ninguna de las opciones anteriores está presente, se utilizará el modelo predeterminado, codificado en el script como `google/gemini-2.0-flash-exp:free`.

Argumentos de Línea de Comandos.

- `-directory`: Especifica el directorio raíz del proyecto a analizar. Si se omite, el script analizará el directorio actual.
- `-model`: Permite definir qué modelo de IA de OpenRouter se debe utilizar para el análisis.
- `-max-files`: Limita el número de ficheros `.java` a analizar, útil para ejecuciones rápidas o pruebas.

Ejemplos de Uso.

- **Análisis con el modelo por defecto:**

```
python3 ai_code_analyzer.py --directory /ruta/al/proyecto
```

- **Especificando un modelo concreto:**

```
python3 ai_code_analyzer.py --directory /ruta/al/proyecto  
--model "anthropic/claude-3.5-sonnet"
```

- **Utilizando una variable de entorno:**

```
export AI_MODEL="openai/gpt-4o"  
python3 ai_code_analyzer.py --directory /ruta/al/proyecto
```

4.2.6. Desafíos y Soluciones de Implementación

Ningún proyecto de software está exento de problemas. A continuación se detallan los desafíos más significativos encontrados durante el desarrollo y cómo fueron abordados.

Desafío 1: El Cuello de Botella de los Modelos Locales (Ollama)

Problema: La intención inicial era utilizar modelos de código abierto como Llama 3 o CodeLlama, ejecutándose localmente en el agente de Jenkins a través de la plataforma Ollama. La ventaja era el coste cero y un mayor control sobre la privacidad de los datos. Sin embargo, las pruebas iniciales revelaron un problema grave de rendimiento. Un modelo de 7 mil millones de parámetros (7B), considerado de tamaño mediano, requería más de 10 GB de RAM y un uso intensivo de CPU. Esto provocaba que la etapa de análisis de un solo archivo tardara varios minutos y, en agentes con recursos limitados, causaba que el sistema se colapsara.

Solución: Se tomó la decisión estratégica de pivotar desde los modelos locales hacia un servicio de API gestionado. Se eligió OpenRouter.ai por su flexibilidad. Este cambio, aunque introducía una dependencia externa y un pequeño coste por uso, resolvió el problema de rendimiento de raíz. La carga computacional se trasladó de la infraestructura local de CI/CD a los servidores optimizados de los proveedores de IA, reduciendo la latencia de minutos a segundos y haciendo que el pipeline fuera viable.

Desafío 2: Garantizar la Consistencia de la Salida de la IA

Problema: A pesar de las instrucciones explícitas en el prompt, algunos modelos de IA, especialmente los más pequeños o los que no están afinados para seguir instrucciones (instruction-tuned), a veces fallaban en devolver un JSON perfecto. Podían añadir texto explicativo antes o después del objeto JSON, o incluso generar un JSON con errores de sintaxis (como una coma extra al final de una lista).

Solución: Se implementó una capa de “limpieza” en el script de Python. Como se ve en la función ‘_call_ai_model’, en lugar de asumir que toda la respuesta es JSON, el código busca el primer carácter ‘‘ y el último ‘‘ y extrae el texto intermedio. Esto elimina cualquier texto de relleno. Además, toda la lógica de parseo (‘json.loads’) se envolvió en un bloque ‘try-except’ para capturar ‘JSONDecodeError’. Si el parseo falla, el error se registra y el pipeline continúa, evitando una falla catastrófica por una respuesta malformada de la IA.

Desafío 3: Gestión Segura de Credenciales

Problema: El uso de la API de OpenRouter.ai requiere una clave de API, que es un secreto que no debe ser expuesto en el código fuente Jenkinsfile.

Solución: Se utilizó el sistema de gestión de credenciales integrado de Jenkins. Los pasos fueron:

1. En la interfaz de Jenkins, ir a ‘Manage Jenkins ->Credentials’.

2. Añadir una nueva credencial de tipo “Secret text”.
3. Pegar la clave de la API de OpenRouter en el campo “Secret” y asignarle un ID, por ejemplo, ‘openrouter-api-key-id’.
4. En el ‘Jenkinsfile’, usar la directiva ‘credentials()’ para inyectar este secreto en una variable de entorno de forma segura, como se muestra en la sección ‘environment’: ‘OPENROUTER_API_KEY = credentials(‘openrouter-api-key-id’)’.

De esta manera, la clave nunca es visible en el código y está protegida por Jenkins.

Desafío 4: Consideraciones sobre la Privacidad y Seguridad de los Datos al Usar Servicios de IA de Terceros

Problema: Al enviar código fuente, potencialmente propietario y sensible, a un servicio de terceros como OpenRouter.ai, surge una preocupación fundamental: ¿cómo se manejan esos datos? Es crucial entender si el código se almacena, si se utiliza para entrenar otros modelos y qué riesgos de seguridad implica para una empresa.

Análisis y Solución: Una investigación de los términos de servicio y la política de privacidad de OpenRouter.ai (OpenRouter, [2025b](#)) revela varios puntos clave:

- **No almacenamiento por defecto:** La política de OpenRouter establece que, por defecto, no almacenan el contenido de las peticiones (prompts) ni las respuestas de los modelos. Solo se registran metadatos para la facturación y el monitoreo, como las marcas de tiempo y el número de tokens utilizados.
- **Control sobre el entrenamiento de modelos:** OpenRouter actúa como un intermediario. La decisión de si los datos se utilizan para entrenamiento recae en el proveedor del modelo final (por ejemplo, Google, OpenAI, etc.). Sin embargo, OpenRouter proporciona un control granular al usuario. En la configuración de la cuenta, se puede desactivar explícitamente la opción de enviar datos a proveedores que los utilizan para entrenamiento. Si esta opción está desactivada, OpenRouter garantiza que las peticiones solo se enrutarán a modelos y proveedores que se han comprometido a no entrenar con los datos del usuario (OpenRouter, [2025a](#)).
- **Riesgos residuales:** A pesar de estos controles, el principal riesgo para una empresa reside en la confianza depositada en la cadena de proveedores. Se debe confiar en que OpenRouter cumplirá con su política de enrutamiento y, a su vez, en que el proveedor final del modelo cumplirá con su política de no-entrenamiento. Para una empresa con código fuente altamente sensible o regulado, la única solución para mitigar completamente este riesgo sería utilizar

modelos de IA auto-alojados (on-premise), aunque esto reintroduce el *Desafío 1* relacionado con los altos costes de infraestructura y mantenimiento.

Conclusión: Para el propósito de esta tesis y para muchas aplicaciones comerciales, los controles ofrecidos por OpenRouter.ai se consideran una solución pragmática y suficientemente segura. Permiten aprovechar la potencia de los modelos más avanzados mientras se mitiga activamente el riesgo de que el código propietario sea utilizado para entrenamiento. La decisión final dependerá siempre de la política de seguridad y el nivel de riesgo aceptable de cada organización.

4.2.7. Resumen de la Implementación y Transición a la Evaluación

A lo largo de este capítulo, se ha detallado el proceso completo de diseño e implementación del prototipo, desde la concepción inicial basada en requisitos funcionales y no funcionales hasta la construcción final de cada componente. Se ha establecido un entorno de CI/CD robusto y reproducible mediante Docker y Jenkins, configurado a través de Infraestructura como Código. Se ha implementado un pipeline Jenkinsfile que orquesta el flujo de análisis, integrando herramientas SAST y el módulo de análisis con IA. Finalmente, se ha descrito el núcleo de la contribución: el script de Python que, mediante una cuidada ingeniería de prompts, interactúa con modelos de lenguaje avanzados para generar sugerencias de mitigación.

Con el prototipo completamente funcional y los desafíos de implementación resueltos, el sistema está listo para ser sometido a prueba. El siguiente capítulo se centrará en la evaluación empírica de la solución, donde se analizará su efectividad, precisión y rendimiento al enfrentarse a un microservicio con vulnerabilidades conocidas, comparando sus resultados con los de herramientas estándar de la industria.

Capítulo 5

Evaluación y Resultados

Este capítulo presenta la evaluación empírica y los resultados obtenidos al aplicar la metodología propuesta en esta tesis. Para validar la eficacia de la inteligencia artificial como herramienta de análisis de seguridad, la evaluación se desarrolla en dos fases principales.

Primero, se lleva a cabo una rigurosa comparativa entre cuatro modelos de IA de última generación y dos herramientas de referencia en la industria, SonarQube y Snyk. Utilizando un microservicio deliberadamente instrumentado con 13 vulnerabilidades comunes, se mide no solo la capacidad de detección de cada herramienta, sino también aspectos cualitativos como la calidad de las sugerencias de corrección y la profundidad del análisis. Además, se realiza un estudio cuantitativo del rendimiento y el coste computacional, factores cruciales para la viabilidad en un entorno de desarrollo real.

En la segunda parte, se exponen los resultados de la implementación práctica del modelo de IA más equilibrado en un pipeline de Integración y Despliegue Continuo (CI/CD) completamente automatizado. Se demuestra el ciclo completo de detección, bloqueo de código inseguro mediante un *Quality Gate*, facilitación de la remediación a través de reportes accionables y, finalmente, la validación de las correcciones con una ejecución exitosa del pipeline.

5.1. Evaluación

Para evaluar la eficacia de la integración de la inteligencia artificial en la detección de vulnerabilidades y mejora de la calidad del código, se ha utilizado un microservicio de ejemplo en Java (alojado en el repositorio <https://github.com/iJonnathan/TFM/tree/main/demo>), diseñado a propósito con múltiples fallos de seguridad. Este código fue analizado tanto por los modelos de IA como por dos herramientas de referencia que representan el estándar industrial: SonarQube y Snyk.

5.1.1. Entorno de Pruebas y Análisis de Vulnerabilidades

El componente central para la evaluación es el controlador `WelcomeController.java` y el fichero `pom.xml` del proyecto, que contienen un conjunto de vulnerabilidades clásicas para medir la capacidad de detección de las herramientas.

Análisis Detallado de Vulnerabilidades Introducidas

Para realizar una evaluación controlada, el microservicio de prueba fue deliberadamente instrumentado con un conjunto de vulnerabilidades, representativas de las categorías más comunes del OWASP Top 10. A continuación, se detalla cada una de ellas:

1. Clave Secreta Embebida en el Código (CWE-798): Se han incluido credenciales directamente en el código fuente.

```
1 // Hardcoded secret (clave embebida)
2 private static final String DB_PASSWORD = "SuperSecreta123!";
3 private static final String API_KEY = "ABC123-TOKEN-INSEGURO";
```

Listing 5.1: Claves secretas hardcodeadas.

2. Cross-Site Scripting (XSS) Reflejado (CWE-79): El endpoint `/api/welcome` concatena un parámetro en la respuesta sin saneamiento.

```
1 @GetMapping("/api/welcome")
2 public WelcomeDTO welcome( @RequestParam(value = "name", defaultValue = "...")
   String name) {
3     return new WelcomeDTO("Hola, bienvenido " + name + ", esto es un demo");
4 }
```

Listing 5.2: Código vulnerable a XSS.

3. Ejecución Remota de Código (RCE) (CWE-94): El endpoint `/api/execute` evalúa una cadena del usuario como código, permitiendo la ejecución de comandos arbitrarios.

```
1 @PostMapping("/api/execute")
2 public String executeCode( @RequestBody String userScript) throws Exception {
3     ScriptEngine engine = new ScriptEngineManager().getEngineByName("nashorn");
4     return "Resultado: " + engine.eval(userScript);
5 }
```

Listing 5.3: Código vulnerable a RCE.

4. Path Traversal (CWE-22): El endpoint `/api/read-file` permite leer archivos del sistema sin una validación de ruta adecuada.

```
1 @GetMapping("/api/read-file")
```

```
2 public String readFile( @RequestParam String filePath) throws Exception {
3     return Files.readString(Paths.get(filePath));
4 }
```

Listing 5.4: Código vulnerable a Path Traversal.

5. Inyección SQL (CWE-89): El endpoint `/api/user` construye una consulta SQL concatenando directamente la entrada del usuario.

```
1 @GetMapping("/api/user")
2 public String getUserInfo( @RequestParam String username) throws Exception {
3     Connection conn = DriverManager.getConnection("jdbc:mysql://localhost:3306/
4     demo", "root", DB_PASSWORD);
5     Statement stmt = conn.createStatement();
6     ResultSet rs = stmt.executeQuery("SELECT * FROM users WHERE username = '" +
7     username + "'");
8     // ...
9 }
```

Listing 5.5: Código vulnerable a Inyección SQL.

6. Inyección de Comandos (CWE-78): El endpoint `/api/ping` ejecuta un comando del sistema operativo concatenando la entrada del usuario.

```
1 @PostMapping("/api/ping")
2 public String ping( @RequestBody String host) throws Exception {
3     Process p = Runtime.getRuntime().exec("ping -c 1 " + host);
4     // ...
5 }
```

Listing 5.6: Código vulnerable a Inyección de Comandos.

7. Deserialización Insegura (CWE-502): El endpoint `/api/deserialize` deserializa un objeto de una fuente no confiable sin validación.

```
1 @PostMapping("/api/deserialize")
2 public String deserialize( @RequestBody String base64) throws Exception {
3     byte[] data = Base64.getDecoder().decode(base64);
4     ObjectInputStream ois = new ObjectInputStream(new ByteArrayInputStream(data)
5     );
6     Object obj = ois.readObject();
7     return "Objeto: " + obj;
8 }
```

Listing 5.7: Código vulnerable a Deserialización Insegura.

8. Inserción de Información Sensible en Logs (CWE-532): El endpoint `/api/login` registra credenciales en texto plano.

```
1 @PostMapping("/api/login")
2 public String login( @RequestParam String user, @RequestParam String password) {
3     logger.info("Intento de login con usuario: " + user + " y contraseña: " +
4         password); // Log inseguro
5     return "Login procesado";
6 }
```

Listing 5.8: Logs con información sensible.

9. Uso de Criptografía Débil (CWE-327): El endpoint `/api/hash` utiliza el algoritmo MD5, que está criptográficamente roto.

```
1 @GetMapping("/api/hash")
2 public String hash( @RequestParam String data) throws Exception {
3     MessageDigest md = MessageDigest.getInstance("MD5"); // Inseguro
4     byte[] digest = md.digest(data.getBytes());
5     return Base64.getEncoder().encodeToString(digest);
6 }
```

Listing 5.9: Uso de un algoritmo de hash débil (MD5).

10. Cifrado Simétrico Débil (CWE-326): El endpoint `/api/encrypt` usa AES con una clave estática y sin un modo de operación seguro (como GCM).

```
1 @GetMapping("/api/encrypt")
2 public String encrypt( @RequestParam String text) throws Exception {
3     String key = "1234567890123456"; // Clave estática
4     SecretKeySpec secretKey = new SecretKeySpec(key.getBytes(), "AES");
5     Cipher cipher = Cipher.getInstance("AES"); // sin GCM ni IV
6     cipher.init(Cipher.ENCRYPT_MODE, secretKey);
7     return Base64.getEncoder().encodeToString(cipher.doFinal(text.getBytes()));
8 }
```

Listing 5.10: Cifrado simétrico débil (AES con clave fija).

11. Exposición de Información Sensible por Error (CWE-209): El endpoint `/api/error` devuelve detalles de la excepción al usuario.

```
1 @GetMapping("/api/error")
2 public String error() {
3     try {
4         int a = 1 / 0;
5         return "OK";
6     } catch (Exception e) {
7         return e.toString(); // Devuelve detalles internos
8     }
9 }
```

9 }

Listing 5.11: Manejo de excepciones que expone información.

12. Dependencia Vulnerable (CWE-1104): Se incluyó una versión antigua de la librería 'jackson-databind' en el 'pom.xml', conocida por tener vulnerabilidades de deserialización.

```

1 <dependency>
2   <groupId>com.fasterxml.jackson.core</groupId>
3   <artifactId>jackson-databind</artifactId>
4   <version>2.9.8</version>
5 </dependency>

```

Listing 5.12: Dependencia vulnerable en pom.xml.

5.1.2. Modelos y Herramientas de Análisis

Para llevar a cabo una evaluación exhaustiva y comparativa, se seleccionó un conjunto de modelos de IA de vanguardia y dos herramientas de referencia líderes en la industria.

Los modelos de IA, todos accesibles a través de la plataforma OpenRouter, fueron ¹:

- **google/gemini-2.0-flash-exp:free**: Un modelo de última generación de Google, conocido por su velocidad y capacidades multimodales.
- **deepseek/deepseek-chat:free**: Un modelo de DeepSeek AI especializado y afinado para tareas de conversación y codificación.
- **deepseek/deepseek-r1-0528:free**: Un modelo de código abierto de DeepSeek de alto rendimiento.
- **qwen/qwen-2.5-coder-32b-instruct:free**: Un modelo de Qwen (Alibaba) especializado en código.

Herramientas de Referencia.

- **SonarQube Community Edition 9.9**: Representa la línea base para el Análisis Estático de Seguridad de Aplicaciones (SAST) y la calidad del código. Es una de las herramientas más utilizadas en la industria para el análisis del código fuente propio.
- **Snyk (Plan Gratuito)**: Representa una solución de seguridad integral. Se utilizaron dos de sus capacidades principales, ambas disponibles en su plan gratuito:

¹Los modelos marcados como 'free' son ofrecidos por OpenRouter sin coste hasta un cierto límite de uso diario, lo que los hizo ideales para la experimentación en este trabajo de tesis.

- **Snyk Open Source (SCA):** Para el Análisis de Composición de Software, enfocado en detectar vulnerabilidades en dependencias de terceros a través del comando ‘snyk test’.
- **Snyk Code (SAST):** Para el Análisis Estático de Seguridad del código fuente propio, a través del comando ‘snyk code test’.

Disponibilidad de los Reportes. Para garantizar la transparencia y la reproducibilidad de esta evaluación, los reportes originales generados por SonarQube, Snyk y cada uno de los modelos de IA se han puesto a disposición pública en el repositorio del proyecto.

5.1.3. Análisis Comparativo de Resultados

Cada modelo de IA y las herramientas de referencia analizaron el proyecto. La siguiente tabla detalla la detección de cada vulnerabilidad específica.

Tabla 1: Detección Detallada de Vulnerabilidades por Herramienta						
Vulnerabilidad	SonarQube	Snyk (Free)	Gemini 2.0	DS Chat	DS R1	Qwen 2.5
Hardcoded Secret (Password)	✓	✓	✓	✓	✓	✓
Hardcoded Secret (API Key)	✓	✓	✓	✓	✓	✓
Cross-Site Scripting (XSS)	✗	✓	✓	✓	✓	✓
Remote Code Execution (RCE)	✗	✓	✓	✓	✓	✓
Path Traversal	✗	✓	✓	✓	✓	✓
Inyección SQL (SQLi)	✗	✓	✓	✓	✓	✓
Inyección de Comandos	✗	✓	✓	✓	✓	✓
Deserialización Insegura	✗	✓	✓	✓	✓	✓
Logs con Datos Sensibles	✗	✗	✓	✓	✓	✓
Uso de Criptografía Débil (Hash)	✗	✓	✓	✓	✓	✓
Cifrado Simétrico Débil	✓	✓	✓	✓	✓	✓
Exposición de Error (Stacktrace)	✗	✗	✓	✓	✓	✓
Dependencia Vulnerable (pom.xml)	✗	✓	✓	✓	✓	✓
Total Código Fuente (12)	3	10	12	12	12	12
Total Dependencias (1)	0	1	1	1	1	1

Análisis Cualitativo de las Respuestas de IA

Como se estableció en la tabla 1, todos los modelos de IA demostraron una capacidad de de-tección del 100 % en el entorno de prueba. Este resultado, aunque impresionante, oculta diferencias cualitativas importantes. Para evaluar la calidad de las respuestas, las sugerencias de corrección se analizaron según tres criterios principales: **Seguridad de la Solución** (¿la corrección es robusta y no introduce nuevos fallos?), **Claridad de la Explicación** (¿es fácil de entender para un desa-rrollador?) y **Accionabilidad** (¿el código sugerido es directamente aplicable?). A continuación, se analizan ejemplos concretos que ilustran estas diferencias.

Profundidad y Seguridad de la Sugerencia de Corrección. Un buen indicador de la calidad de un modelo es la robustez de la solución que propone. Se puede observar una diferencia clara en

la gestión de la vulnerabilidad de **Path Traversal (CWE-22)**.

- El modelo **qwen/qwen-2.5-coder-32b-instruct:free** ofreció la siguiente sugerencia:

```
1 // Sugerencia de corrección:
2 if (!filePath.startsWith("/safe/directory/")) {
3     throw new IllegalArgumentException("Ruta inválida");
4 }
5
```

Listing 5.13: Sugerencia de Qwen para Path Traversal.

Análisis: Esta solución es superficial e insegura. Un atacante podría evadirla fácilmente utilizando secuencias de escape como 'filePath = "/safe/directory/../../etc/passwd" ', ya que la cadena efectivamente comienza con el prefijo permitido, pero luego navega fuera del directorio seguro.

- En contraste, los modelos **google/gemini-2.0-flash-exp:free** y **deepseek/deepseek-r1-0528:free** proporcionaron una solución mucho más robusta y segura:

```
1 // Sugerencia de corrección:
2 Path safePath = Paths.get("/safe/directory/").resolve(filePath).normalize()
3 ;
4 if (!safePath.startsWith(Paths.get("/safe/directory/"))) {
5     throw new IllegalArgumentException("Invalid file path");
6 }
7 return Files.readString(safePath);
8
```

Listing 5.14: Sugerencia de Gemini/DeepSeek para Path Traversal.

Análisis: Esta implementación es significativamente superior. El método 'normalize()' resuelve las secuencias '..', eliminando los intentos de evasión. La posterior validación con 'startsWith()' sobre la ruta ya normalizada asegura que el acceso a ficheros se mantenga estrictamente dentro del directorio previsto. Este es un ejemplo claro de cómo, a pesar de que ambos modelos detectaron la vulnerabilidad, solo uno proporcionó una corrección verdaderamente segura y accionable para el desarrollador.

Riqueza del Análisis: Más Allá de la Seguridad. Otro diferenciador clave es la capacidad de algunos modelos para ofrecer observaciones valiosas sobre la calidad y mantenibilidad del código, incluso en ficheros sin vulnerabilidades de seguridad. El análisis del fichero `WelcomeDTO.java` (un simple DTO) es un buen ejemplo.

- El modelo **deepseek/deepseek-r1-0528:free** identificó una mejora de fiabilidad importante:

Descripción: Class does not override equals() and hashCode(). This may cause unexpected behavior when used in collections.

Recomendación: Implement equals() and hashCode() based on the message field.

Análisis: Esta es una sugerencia de alta calidad. No implementar 'equals()' y 'hashCode()' en DTOs es una causa frecuente de comportamientos anómalos y errores de difícil depuración cuando estos objetos se utilizan en colecciones como 'HashMap' o 'HashSet'. Este tipo de feedback proactivo ayuda a mejorar la robustez general de la aplicación.

- Por su parte, el modelo **google/gemini-2.0-flash-exp:free** se enfocó en un principio de diseño de software:

Descripción: The 'message' field is mutable due to the presence of a setter method. For simple DTOs, immutability can improve thread safety and predictability.

Recomendación: If the message should not be changed after creation, remove the 'setMessage' method and make the 'message' field final.

Análisis: Esta también es una recomendación valiosa que apunta a mejorar la mantenibilidad y a prevenir efectos secundarios no deseados, promoviendo un diseño más funcional y seguro en entornos concurrentes.

Estos ejemplos demuestran que, si bien la cobertura de detección de vulnerabilidades es una métrica base, la verdadera diferenciación y el valor para un equipo de desarrollo radican en la calidad, profundidad y seguridad de las recomendaciones generadas, así como en la capacidad del modelo para actuar como un revisor de código integral.

Análisis Cuantitativo de Rendimiento y Coste

Más allá de la calidad de la detección, la viabilidad de integrar una solución de IA en un pipeline de CI/CD depende críticamente de dos factores cuantitativos: el **rendimiento** (tiempo de ejecución) y el **coste** (consumo de tokens). Un análisis lento puede crear cuellos de botella en el ciclo de desarrollo, mientras que un alto consumo de tokens puede hacer que la solución sea económicamente inviable a escala.

La Importancia de los Tokens. En los modelos de IA comerciales, el coste se factura por el número de *tokens* procesados. Un token es una unidad de texto (aproximadamente 4 caracteres en inglés). El coste total de una llamada a la API se compone de:

- **Tokens de Entrada (Prompt Tokens):** El tamaño del código y las instrucciones enviadas al modelo. En este experimento, los tokens de entrada fueron constantes para todos los modelos, ya que analizaron los mismos ficheros.
- **Tokens de Salida (Completion Tokens):** El tamaño de la respuesta JSON generada por el modelo. Una respuesta más detallada, con descripciones más largas y sugerencias de código más elaboradas, consume más tokens de salida y, por lo tanto, es más costosa.

Para evaluar estos factores, se midió el tiempo de ejecución de cada análisis y se utilizó el tamaño en bytes del fichero de resultados JSON como una métrica proxy del consumo de tokens de salida.

Tabla 2: Comparativa de Rendimiento y Consumo de Tokens

Modelo de IA	Tiempo de Análisis	Tamaño de Respuesta (Proxy de Tokens)	Análisis
gemini-2.0-flash-exp:free	1 min 38 seg	45.6 KB	Rápido y detallado
deepseek-chat:free	8 min 13 seg	24.8 KB	Lento y moderadamente detallado
deepseek-r1-0528:free	18 min 49 seg	16.1 KB	Muy lento y conciso
qwen-2.5-coder-32b-instruct:free	4 min 54 seg	49.2 KB	Velocidad media, respuesta más extensa

Discusión de los Resultados. La Tabla 2 revela un interesante equilibrio entre velocidad, coste y verbosidad:

- El modelo **qwen/qwen-2.5-coder-32b-instruct:free** generó la respuesta más detallada (mayor tamaño), lo que implica un mayor coste en tokens, pero lo hizo en un tiempo razonable. Esto sugiere que es un modelo que prioriza la exhaustividad en sus explicaciones.
- El modelo **google/gemini-2.0-flash-exp:free** emerge como el más equilibrado para un entorno de CI/CD. Ofrece una velocidad de ejecución excepcional (casi 5 veces más rápido que el siguiente) y genera una respuesta muy detallada, representando una excelente relación entre eficiencia, coste y calidad de la información.
- El modelo **deepseek/deepseek-r1-0528:free**, a pesar de ser el más lento, fue sorprendentemente el más conciso en su respuesta. Esto indica un estilo de comunicación muy directo, que podría ser preferible en algunos contextos, pero su alto tiempo de ejecución lo hace menos ideal para pipelines de integración continua que requieren retroalimentación rápida.

Este análisis cuantitativo es fundamental para la toma de decisiones en un entorno real. Un equipo podría optar por usar un modelo rápido y económico como `google/gemini-2.0-flash-exp:free` para las revisiones en cada commit, y reservar el uso de un modelo más lento pero potencialmente más exhaustivo en sus explicaciones para los escaneos nocturnos o previos al despliegue.

5.1.4. Discusión General y Conclusiones de la Evaluación

Del análisis detallado de los resultados, se extraen las siguientes observaciones clave:

- **Rendimiento de las Herramientas de Referencia (Baselines):** Las herramientas especializadas demostraron ser eficaces en sus respectivos dominios, pero con diferentes niveles de cobertura.
 - **SonarQube (SAST):** Detectó 3 de las 12 vulnerabilidades del código fuente, enfocándose en patrones de calidad y seguridad evidentes (como las claves embebidas), pero no identificó vulnerabilidades contextuales complejas (SQLi, XSS) ni la dependencia vulnerable. Su valor principal reside en la gobernanza de la calidad del código.
 - **Snyk (SAST y SCA):** Demostró una alta eficacia en ambos frentes. Su motor SAST ('Snyk Code') detectó 10 de las 12 vulnerabilidades del código fuente, fallando únicamente en la detección de logs con datos sensibles y la exposición de errores. Su motor SCA ('Snyk Open Source') identificó correctamente la dependencia vulnerable en el 'pom.xml'. Esto la posiciona como una herramienta de seguridad muy robusta.
- **Rendimiento Holístico de los Modelos de IA:** El hallazgo más significativo es la capacidad de los modelos de IA para realizar un análisis híbrido, cubriendo tanto las responsabilidades de SAST como de SCA con una eficacia del 100 % en el entorno de prueba. Todos los modelos identificaron la totalidad de las 13 vulnerabilidades, superando incluso la cobertura de Snyk y demostrando una comprensión contextual superior.
- **Nuevos Criterios de Diferenciación: Detección vs. Asistencia:** Dado que tanto la IA como Snyk muestran una alta capacidad de detección, el criterio de diferenciación se desplaza. La pregunta ya no es solo *¿qué herramienta encuentra más fallos?*, sino *¿qué herramienta ayuda mejor al desarrollador a corregirlos?* Aquí es donde la IA generativa aporta un valor único, al proporcionar explicaciones detalladas y sugerencias de código contextualizadas, un enfoque más pedagógico que el de las herramientas tradicionales.
- **Complementariedad Re-evaluada:** La estrategia de seguridad en un pipeline moderno debe ser vista como un sistema de capas.
 - **Capa 1 (Calidad):** SonarQube sigue siendo fundamental para aplicar un conjunto de reglas de código de forma rápida y consistente, manteniendo la salud general del código.
 - **Capa 2 (Detección):** Herramientas como Snyk ofrecen una detección de seguridad SAST y SCA muy competente y fiable, actuando como una red de seguridad robusta.

- **Capa 3 (Asistencia y Aceleración):** La IA generativa, como la implementada en esta tesis, se posiciona como una capa superior. No solo detecta, sino que acelera la remediación, reduce la carga cognitiva del desarrollador y sirve como una herramienta de aprendizaje continuo, mejorando las capacidades de seguridad del equipo.

La evaluación empírica llevada a cabo en este trabajo permite extraer una conclusión fundamental: la integración de la Inteligencia Artificial en los pipelines de CI/CD para el análisis de seguridad no solo es viable, sino que representa la siguiente evolución en las prácticas de DevSecOps, yendo más allá de la simple detección para enfocarse en la asistencia inteligente.

Mientras que herramientas como SonarQube son indispensables para la gobernanza de la calidad y Snyk ofrece una detección de vulnerabilidades SAST y SCA muy sólida, los modelos de IA evaluados han demostrado un potencial superior en dos áreas clave:

1. **Cobertura de Detección Holística:** Los modelos de IA fueron los únicos capaces de identificar el 100 % de las vulnerabilidades de código y de dependencias en el entorno de prueba, demostrando una capacidad de análisis contextual que supera a las herramientas de referencia.
2. **Asistencia para la Remediación:** El valor más significativo de la IA no reside en su capacidad de detección, sino en la calidad de la asistencia que proporciona. Al generar explicaciones claras y fragmentos de código directamente aplicables, la IA reduce drásticamente el tiempo y el esfuerzo que los desarrolladores deben invertir en investigar y corregir vulnerabilidades, convirtiendo un reporte de seguridad en un plan de acción concreto.

En definitiva, una estrategia de DevSecOps madura debería considerar un enfoque híbrido y por capas. Utilizar SonarQube para la calidad, apoyarse en herramientas robustas como Snyk para una detección fiable, y complementar el ecosistema con un sistema de asistencia basado en IA para acelerar la corrección y mejorar las habilidades del equipo de desarrollo. Este trabajo demuestra que el futuro de la seguridad en el desarrollo de software no está solo en encontrar más fallos, sino en solucionarlos de manera más rápida, eficiente e inteligente.

5.2. Resultados de la Ejecución del Pipeline

Para la validación empírica de la solución, se diseñó una prueba específica utilizando el microservicio ('demo'). A diferencia de la comparativa inicial que abarcó trece vulnerabilidades, esta fase se concentró en un subconjunto de **cinco vulnerabilidades** para evaluar de forma precisa el comportamiento de la solución integrada en el pipeline.

El modelo de IA seleccionado para esta prueba fue `gemin-2.0-flash-exp`. Esta elección se justifica por su rendimiento superior en la evaluación comparativa (sección 5.1.3), donde demostró el mejor equilibrio entre precisión, calidad de las sugerencias y eficiencia.

Con el entorno CI/CD completamente automatizado, la ejecución del pipeline se activó tras realizar un 'commit' en el repositorio, cumpliendo con el requisito **RF-01**. La vista general del proceso, con las etapas completadas y el punto de fallo, se puede apreciar en la Figura 4.

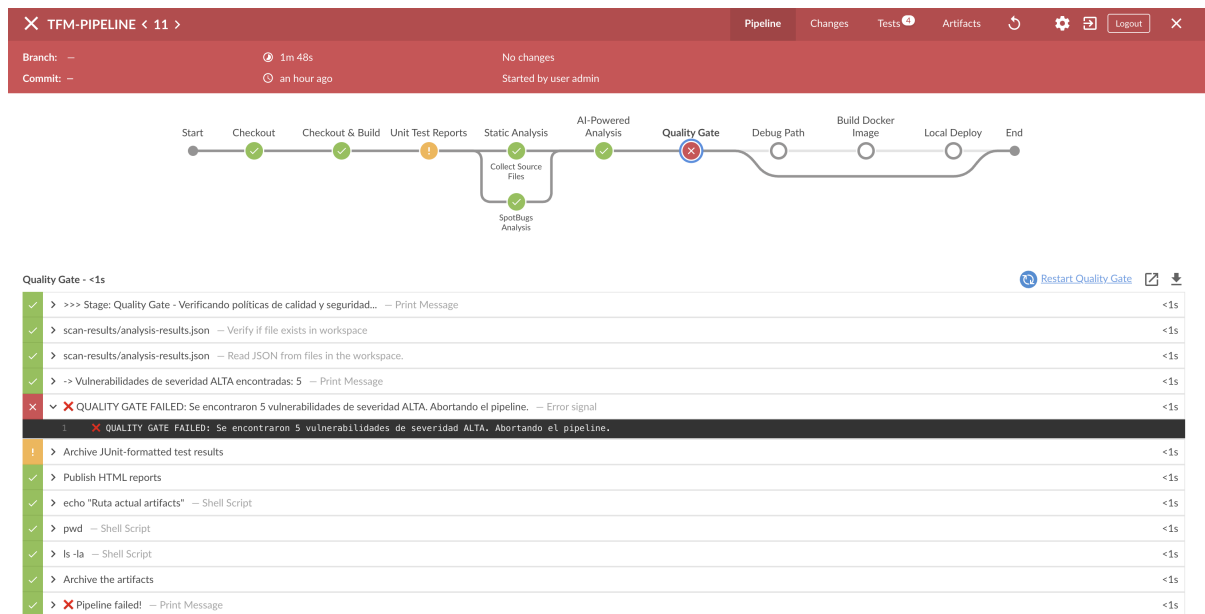


Figura 4: Ejecución fallida del pipeline.

Flujo de Ejecución y Falla Controlada en el Quality Gate

El pipeline ejecutó las primeras etapas de manera secuencial y paralela, tal como fue diseñado, recopilando información de seguridad en cada paso antes de llegar al punto de control crítico.

1. Las etapas iniciales, **Checkout & Build** y **Unit Test Reports**, se completaron con éxito. El código fue clonado, la aplicación fue empaquetada con Maven (**RF-02**), y los reportes de pruebas unitarias fueron procesados por Jenkins, sin embargo, una de las pruebas unitarias saltó una excepción debido a una vulnerabilidad de dependencia en el `pom.xml`, el cual fue detectada en el stage **AI-Powered Analysis**.
2. La etapa **Static Analysis** se ejecutó, realizando sus tareas en paralelo para optimizar el tiempo.
3. La etapa **AI-Powered Analysis** se completó satisfactoriamente, cumpliendo su objetivo principal (**RF-04**). El script de Python interactuó con la API de la IA y generó los artefactos cruciales: el reporte HTML para el desarrollador ('`ai-analysis-report.html`') y el reporte JSON ('`analysis-results.json`') para la automatización, cumpliendo así con **RF-05**.

4. La ejecución llegó a la etapa **Quality Gate**. Este es el punto de control automatizado que implementa la política de seguridad del pipeline. El script leyó el archivo `analysis-results.json` y, tal como se esperaba, encontró que el valor de `high_severity_vulnerabilities` era de 5.

Debido a que este valor era mayor que el umbral de cero definido en la política, la condición del Quality Gate se activó y ejecutó el paso `error()`. Este comando detuvo inmediatamente el pipeline y lo marcó con el estado **FAILURE**, como se evidencia en la captura de la Figura 4. Como resultado, las etapas posteriores Debug Path, Build Docker Image y Local Deploy nunca se ejecutaron.

Este fallo controlado es un éxito rotundo para el experimento. Demuestra que el sistema funciona como un mecanismo de seguridad preventivo eficaz (cumpliendo **RF-06**), deteniendo físicamente la promoción de código inseguro a través del ciclo de vida de desarrollo.

Análisis Detallado del Reporte Generado por la IA

La pieza clave generada por el pipeline es el reporte en formato HTML. Este documento, fácil de leer para los desarrolladores, es la principal herramienta para entender y solucionar los problemas encontrados. El reporte completo de esta ejecución se encuentra en el **Anexo D**.

A continuación, se explican los problemas más graves que encontró la IA y que causaron la falla del Quality Gate.

Dependencia Vulnerable en 'pom.xml' (Severidad: HIGH)

- **Hallazgo:** La IA revisó el archivo `pom.xml` y vio que se usaba la versión 2.9.8 de la librería `jackson-databind`. Avisó que esta versión es muy antigua y tiene fallos de seguridad graves (CVEs) que podrían permitir a un atacante ejecutar código en el servidor.
- **Sugerencia de la IA:** La solución fue simple y directa: actualizar la librería a una versión más nueva y segura (como 2.16.1 o superior) en el archivo `pom.xml`.

Inyección SQL (Severidad: CRITICAL)

- **Hallazgo:** En el `WelcomeController`, la IA detectó que una consulta a la base de datos se estaba construyendo uniendo texto con datos que venían de un usuario, sin ninguna protección. Esto es un riesgo clásico de Inyección SQL.
- **Sugerencia de la IA:** Recomendó usar consultas preparadas (`PreparedStatement`), que es la forma correcta y segura de hacerlo, e incluyó un ejemplo de código de cómo implementarlo.

Ejecución Remota de Código (RCE) (Severidad: CRITICAL)

- **Hallazgo:** El sistema identificó que una parte del código `ScriptEngine.eval()` ejecutaba comandos directamente de la entrada del usuario, lo que es un riesgo crítico.
- **Sugerencia de la IA:** La recomendación fue eliminar esa función por completo, lo cual es la decisión más segura cuando una característica es tan peligrosa.

Path Traversal (Severidad: HIGH)

- **Hallazgo:** La IA señaló que una función que leía archivos tomaba la ruta del archivo directamente de la entrada del usuario, lo que le permitiría a un atacante leer archivos prohibidos del sistema usando secuencias como `../`.
- **Sugerencia de la IA:** Sugirió añadir código para verificar que la ruta del archivo solicitado estuviera siempre dentro de una carpeta base segura y permitida.

Claves Secretas en el Código (Severidad: CRITICAL)

- **Hallazgo:** El modelo encontró que la contraseña de la base de datos y una clave de API estaban escritas directamente en el código.
- **Sugerencia de la IA:** Recomendó seguir las mejores prácticas de seguridad (cumpliendo **RNF-03**) y mover estos secretos a variables de entorno o a un sistema de gestión de credenciales, en lugar de tenerlos en el código.

Remediación y Ejecución Exitosa del Pipeline

El valor fundamental del prototipo no reside únicamente en la detección, sino en su capacidad para facilitar una remediación rápida y precisa. Con las instrucciones claras y los ejemplos de código proporcionados por el reporte de la IA, un desarrollador pudo proceder a corregir las 5 vulnerabilidades críticas identificadas. El proceso de remediación consistió en:

- Actualizar la versión de la dependencia 'jackson-databind' en el archivo 'pom.xml' a una versión segura.
- Refactorizar el método 'getUserInfo' en `WelcomeController.java` para utilizar `PreparedStatement`, eliminando así la vulnerabilidad de Inyección SQL.
- Eliminar por completo los métodos inseguros `executeCode` (RCE) y `ping` (Command Injection).
- Añadir la lógica de validación de rutas en el método `readFile` para prevenir el Path Traversal.

- Externalizar las claves secretas ('DB_PASSWORD' y 'API_KEY'), eliminándolas del código fuente y configurándolas como variables de entorno seguras.

Una vez aplicados estos cambios y subido el código corregido al repositorio de GitHub, el pipeline de Jenkins se ejecutó por segunda vez.

En esta nueva ejecución, el flujo de trabajo fue notablemente diferente. La etapa de 'AI-Powered Analysis' analizó el código corregido y generó un nuevo reporte que no contenía ninguna vulnerabilidad de severidad 'HIGH' o 'CRITICAL'. Consecuentemente, al llegar a la etapa de 'Quality Gate', el resultado fue positivo. El log de la consola mostró el mensaje esperado: 'QUALITY GATE PASSED: No se encontraron vulnerabilidades de severidad ALTA.'

Al superar con éxito esta puerta de calidad, en primer lugar, la etapa de pruebas unitarias que anteriormente falló, esta vez ejecutó con éxito las 2 pruebas unitarias. El pipeline no se detuvo, continuó su ejecución a través de las etapas restantes:

1. La etapa **Build Docker Image** se ejecutó sin problemas, tomando el archivo 'jar' seguro y empaquetándolo en una imagen Docker llamada 'demo-app'.
2. La etapa final, **Local Deploy**, también se completó exitosamente. Detuvo y eliminó cualquier versión anterior del contenedor de la aplicación y desplegó la nueva imagen, dejando el microservicio corregido y funcional, tal como se observa en la **Figura 5**.

```

Local Deploy - <1s
> >>> Stage: Local Deploy - Desplegando localmente con Docker... — Print Message
> docker stop demo-app || true — Shell Script
> docker rm demo-app || true — Shell Script
> docker run -d -p 8081:9090 -name demo-app demo-app — Shell Script
1 + docker run -d -p 8081:9090 -name demo-app demo-app
2 b17c73b0fba5f4d7b6c1e9ac45f8b7732b89f97846f78e459e578ba8e113286
> >>> El microservicio debería estar accesible localmente en http://localhost:8081 — Print Message
1 >>> El microservicio debería estar accesible localmente en http://localhost:8081
> >>> Verifica los logs del contenedor con: docker logs demo-app — Print Message
> Archive JUnit-formatted test results
> Publish HTML reports
> echo "Ruta actual artifacts" — Shell Script
> pwd — Shell Script
> ls -la — Shell Script
> Archive the artifacts
> 🎉 Pipeline completed successfully! — Print Message

```

Figura 5: Despliegue exitoso de la etapa de despliegue.

En conclusión, el ciclo de evaluación demostró ser un éxito completo. La primera ejecución validó la capacidad del sistema para detectar riesgos y actuar como un mecanismo de control preventivo, deteniendo un despliegue inseguro. La segunda ejecución, tras aplicar las correcciones sugeridas por la IA, validó la utilidad y precisión de dichas sugerencias, permitiendo que un artefacto seguro completara el pipeline de principio a fin, como se evidencia en la ejecución final de la **Figura 6**. Este proceso empírico confirma que la integración de asistencia por IA en un pipeline CI/CD no

solo es viable, sino que acelera eficazmente el ciclo de remediación, cumpliendo así con los objetivos centrales de esta tesis.

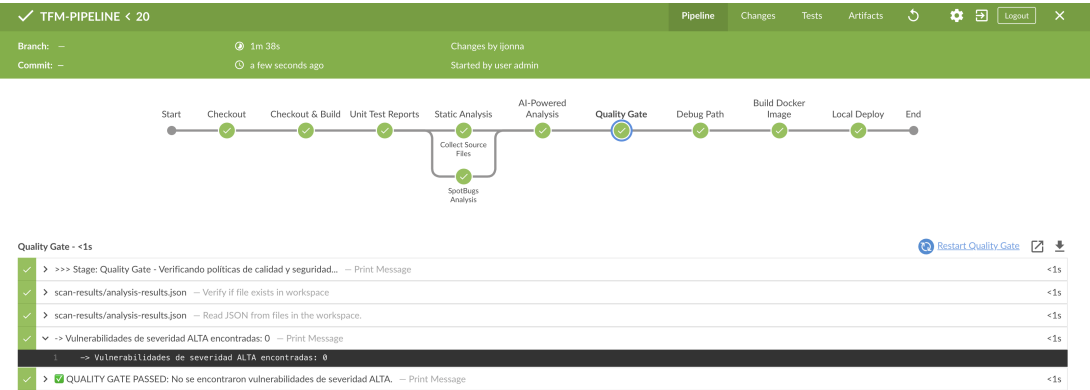


Figura 6: Ejecución exitosa del pipeline.

Capítulo 6

Conclusiones y Trabajo Futuro

Este capítulo final sintetiza los hallazgos y contribuciones de la presente investigación, evalúa el grado de cumplimiento de los objetivos propuestos y establece una visión para futuras líneas de desarrollo que pueden expandir y mejorar el trabajo realizado.

6.1. Conclusiones

La proliferación de arquitecturas de microservicios y la consolidación de las prácticas DevOps han impuesto un ritmo de entrega de software sin precedentes. Sin embargo, esta agilidad a menudo introduce un desafío crítico: la gestión eficaz y oportuna de la seguridad. El problema central abordado en esta tesis fue la brecha existente entre la detección de vulnerabilidades y su efectiva mitigación, un proceso que tradicionalmente recae en el desarrollador y que puede ser lento, propenso a errores y carente del contexto necesario.

Para enfrentar este desafío, el **objetivo general** de este trabajo fue diseñar, implementar y evaluar un prototipo que integrara la Inteligencia Artificial (IA) en pipelines CI/CD para generar sugerencias de mitigación de vulnerabilidades en microservicios Java. Tras un proceso de desarrollo y evaluación riguroso, se puede afirmar que este objetivo se ha cumplido satisfactoriamente.

El prototipo funcional, orquestado en Jenkins y utilizando un 'Jenkinsfile' como pilar del *Pipeline as Code*, ha demostrado la viabilidad de la hipótesis central: es posible automatizar no solo la detección, sino también la generación de orientación para la corrección de fallos de seguridad. A continuación, se detallan las conclusiones en relación con los objetivos específicos:

1. **Respuesta a los Objetivos Específicos:** Se logró la integración exitosa de herramientas SAST ('SpotBugs') y SCA ('OWASP Dependency-Check') en un pipeline de Jenkins. Se seleccionó y se integró un modelo de IA a través de la plataforma 'OpenRouter.ai', lo que proporcionó flexibilidad para evaluar diferentes LLMs. Se desarrolló la lógica de organización

en Python, que actúa como el núcleo del sistema, procesando los archivos de código fuente del proyecto y orquestando las llamadas a la IA. Todo el flujo se automatizó, y se implementó la generación del reporte en HTML, proporcionando una interfaz clara y accionable para el desarrollador. Finalmente, la evaluación del prototipo con un microservicio de prueba validó su efectividad, cumpliendo así con todos los objetivos específicos planteados.

2. **Contribución Principal: Asistencia Inteligente sobre Corrección Automática:** La contribución más significativa de este trabajo no es la creación de una herramienta que corrija código de forma autónoma —una tarea aún arriesgada y compleja—, sino la validación de un enfoque de **asistencia inteligente**. Al proporcionar sugerencias de mitigación contextualizadas, con ejemplos de código corregido y explicaciones detalladas, el sistema empodera al desarrollador. Se reduce la carga cognitiva, se acelera la curva de aprendizaje en seguridad y se disminuye el tiempo entre la detección y la remediación, fortaleciendo el ciclo DevSecOps.
3. **Independencia y Flexibilidad de la Plataforma:** Una decisión de diseño clave fue separar la lógica de análisis del orquestador CI/CD. El análisis de vulnerabilidades y la interacción con la IA se encapsularon en un script de Python independiente. Esto demuestra que, aunque la implementación se realizó en Jenkins, el núcleo de la solución no depende de él. Este componente podría ser integrado con mínima fricción en otros sistemas de CI/CD como GitLab CI/CD, GitHub Actions o ArgoCD, gracias a su naturaleza de script de línea de comandos, que puede ser invocado desde prácticamente cualquier orquestador, lo que valida su carácter multiplataforma y su adaptabilidad a diversos ecosistemas tecnológicos.
4. **Justificación y Contexto del Proyecto:** Este proyecto se ha desarrollado con un enfoque académico y de aprendizaje, buscando explorar las sinergias entre la IA y DevSecOps. No pretende ser una solución comercial superior a las existentes, sino una propuesta de código abierto que puede ser utilizada y adaptada libremente por la comunidad universitaria y de desarrolladores. Su valor radica en la demostración de un enfoque innovador y accesible.

6.1.1. Análisis Comparativo

Para contextualizar la aportación de esta tesis, se realizó un análisis comparativo entre la solución propuesta y las herramientas de seguridad que se utilizaron como referencia en la evaluación: SonarQube y Snyk. Esta comparación permite resaltar las diferencias en filosofía, enfoque y aplicabilidad de cada solución.

El análisis comparativo de la Tabla 3 revela que las tres herramientas operan con filosofías distintas.

Tabla 3: Tabla Comparativa: Solución Propuesta vs. Herramientas de Evaluación

Característica	Solución Propuesta (Tesis)	SonarQube (SonarSource S.A., 2024)	Snyk (Snyk, Ltd., 2025)
Enfoque Principal	Asistencia para mitigación: genera explicaciones y código de solución usando IA.	Detección y seguimiento: identifica bugs y vulnerabilidades (SAST).	Seguridad integral: Fuerte en SAST, SCA, contenedores e IaC.
Integración CI/CD	Diseñada para operar en cualquier pipeline compatible con scripts.	Plugins para las principales plataformas de CI/CD.	Integración nativa con múltiples plataformas y herramientas de desarrollo.
Infraestructura	Contenedor Docker ligero. Sin servidor dedicado.	Requiere servidor SonarQube persistente con BD.	Principalmente SaaS (gestionado por Snyk), con opciones on-premise.
Inteligencia Artificial	Componente central. Uso de IA generativa para explicaciones y sugerencias de código contextuales.	Incorporada en ediciones de pago para generar sugerencias de corrección de código (AI CodeFix).	Utilizada para análisis de alcanzabilidad y ofrecer correcciones automáticas predefinidas (DeepCode AI), no con un enfoque generativo.
Coste	Código abierto (costo de infraestructura y uso de API de IA).	Comercial (con una edición gratuita - Community).	Comercial (con un plan gratuito para desarrolladores y proyectos open source).

- **SonarQube** se posiciona como una plataforma centralizada de gobernanza de la calidad y seguridad del código. Su fortaleza radica en ofrecer un panel de control exhaustivo para gestionar la deuda técnica y seguir la evolución de un proyecto a lo largo del tiempo.
- **Snyk** adopta un enfoque centrado en el desarrollador (developer-first), integrándose de manera nativa en sus herramientas (IDE, CLI, repositorios) para ofrecer una cobertura de seguridad amplia (código, dependencias, contenedores, IaC) y facilitar la corrección temprana de vulnerabilidades.
- La **solución de esta tesis**, en cambio, no busca competir en la detección exhaustiva ni en la integración profunda con el ecosistema de un proveedor. Su nicho y principal innovación residen en el paso posterior a la detección: el uso explícito de la IA generativa para acelerar la corrección humana. Actúa como un complemento, un traductor inteligente que convierte un reporte de vulnerabilidad en una explicación clara y una solución tangible, reduciendo la fricción y la carga cognitiva del desarrollador.

En resumen, mientras las herramientas comerciales se centran en la detección y prevención, este trabajo explora el potencial de la IA para democratizar y acelerar la fase de mitigación, un área con un significativo potencial de mejora.

6.2. Líneas de Trabajo Futuro

El prototipo desarrollado sienta una base sólida sobre la cual se pueden construir numerosas mejoras y extensiones. A continuación, se proponen varias líneas de trabajo futuro que podrían aportar un valor considerable:

1. **Expansión a Análisis Dinámico (DAST) e Interactivo (IAST):** Actualmente, el sistema se basa en análisis estático (SAST). Una mejora natural sería integrar los resultados de herramientas DAST (como OWASP ZAP) o IAST. Esto permitiría a la IA recibir un contexto de ejecución, como trazas de peticiones HTTP, para generar sugerencias aún más precisas, especialmente para vulnerabilidades que solo se manifiestan en tiempo de ejecución.
2. **Integración con Sistemas de Gestión de Incidencias:** Para cerrar completamente el ciclo DevSecOps, el sistema podría integrarse con herramientas como Jira o GitLab Issues. Tras detectar una vulnerabilidad y generar una sugerencia, podría crear automáticamente un ticket asignado al equipo correspondiente, incluyendo toda la información relevante (descripción, severidad, fichero, línea y la sugerencia de la IA).
3. **Análisis de Múltiples Microservicios y Vulnerabilidades Inter-servicio:** El estudio se centró en un único microservicio. Un desafío mucho más complejo, pero de gran valor, sería analizar las interacciones entre múltiples microservicios para detectar vulnerabilidades que emergen de la comunicación entre ellos (ej. problemas de autenticación/autorización en cadena, propagación de datos no sanitizados).
4. **Afinamiento de Modelos de IA (Fine-Tuning):** En lugar de utilizar modelos de propósito general, se podría realizar un *fine-tuning* de un modelo de código abierto (como Llama 3 o CodeLlama) con un dataset específico de vulnerabilidades y sus correcciones. Esto podría resultar en un modelo más pequeño, rápido, especializado y potencialmente más económico de operar a escala, con una precisión aún mayor para esta tarea concreta.
5. **Mecanismos de Retroalimentación y Aprendizaje Continuo:** Se podría añadir una funcionalidad en el reporte HTML para que los desarrolladores califiquen la utilidad de las sugerencias de la IA ('útil' / 'no útil'). Esta retroalimentación podría ser recolectada y utilizada para re-entrenar o ajustar el modelo periódicamente, creando un sistema que aprende y mejora con el tiempo.
6. **Soporte para más Lenguajes y Ecosistemas:** La arquitectura es modular, por lo que podría extenderse para soportar otros lenguajes de programación (como Python, Node.js, Go)

y sus respectivas herramientas de análisis de seguridad, ampliando significativamente el impacto y la aplicabilidad del proyecto.

En conclusión, esta tesis ha demostrado que la sinergia entre la automatización de DevOps y la capacidad cognitiva de la Inteligencia Artificial ofrece un camino prometedor para construir software más seguro de manera más eficiente. El trabajo futuro en esta área es vasto y emocionante, con el potencial de transformar fundamentalmente la forma en que abordamos la seguridad en el ciclo de vida del desarrollo de software.

Bibliografía

- Feng, Zhangyin et al. (2020). «CodeBERT: A Pre-Trained Model for Programming and Natural Languages». En: *arXiv preprint arXiv:2002.08155*.
- OWASP Foundation (2021). *OWASP Top 10 - 2021*. <https://owasp.org/Top10/>. Accedido: 06-05-2025.
- Al-Dhuraibi, M. A. F. et al. (2022). «Security issues in microservices architecture: A systematic literature review». En: *Ain Shams Eng. J.* 13.4, pág. 101668.
- Kumar, S. S., K. S. Kuppusamy y R. Aghila (2022). «DevSecOps: A comprehensive review on an emerging methodology». En: *Comput. Sci. Rev.* 43, pág. 100434.
- Wang, W. et al. (2022). «Can Large Language Models Write Secure Code? A Security Evaluation of GitHub Copilot». En: *arXiv preprint arXiv:2211.03643*.
- Finite State (2023). *The 10 Major Java Vulnerability Types that Threaten Your Software*. <https://finitestate.io/blog/top-10-java-vulnerabilities>. Accedido: 06-05-2025.
- Fu, Michael et al. (2023). «Patching with Confidence: A Language-Model-Based Approach for Targeted Program Repair». En: *arXiv preprint arXiv:2310.09858*.
- Gartner, Inc. (mayo de 2023). *Market Guide for AIOps Platforms*. Inf. téc. ID G00777593. Gartner.
- Johnson, Blair, Tudor Pletea y Andreas Nurnberger (2023). «A Qualitative Study of the Usability of Static Analysis Tools for Security». En: *2023 IEEE/ACM 1st International Workshop on AI for Software Engineering (WAIN)*, págs. 85-92. DOI: [10.1109/WAIN59346.2023.00021](https://doi.org/10.1109/WAIN59346.2023.00021).
- Santos, Pedro, Marco Silva y Cláudia Costa (2023). «From Silos to Synergy: A Systematic Review of DevSecOps Culture, Practices, and Tools». En: *IEEE Access* 11, págs. 12345-12360. DOI: [10.1109/ACCESS.2023.XXXXXXX](https://doi.org/10.1109/ACCESS.2023.XXXXXXX).
- Zafeiropoulos, S. et al. (2023). «Security Gaps in Microservices: A Systematic Literature Review». En: *Future Internet* 15.12, pág. 382.
- GitGuardian (mar. de 2024). *GitHub Copilot Security and Privacy Concerns: Understanding the Risks and Best Practices*. <https://blog.gitguardian.com/github-copilot-security-and-privacy/>. Accedido: 06-05-2025.

- GitHub, Inc. (2024). *GitHub Advanced Security Documentation*. <https://docs.github.com/en/get-started/learning-about-github/about-github-advanced-security>. Accedido: 05-07-2025.
- Liu, X. et al. (2024). «A Survey of Learning-based Automated Program Repair». En: *ACM Comput. Surv.* 56.1, págs. 1-38.
- National Institute of Standards and Technology (NIST) (feb. de 2024). *Defending Continuous Integration/Continuous Delivery (CI/CD) Environments*. Special Publication (Draft). NIST.
- New Relic (2024). *2024 State of the Java Ecosystem*. Inf. téc. New Relic.
- Siddiq, M. F. et al. (2024). «A Survey on Large Language Models for Code». En: *arXiv preprint arXiv:2404.08198*.
- SonarSource S.A. (2024). *SonarQube Editions*. <https://www.sonarsource.com/products/sonarqube/>. Accedido: 05-07-2025.
- OpenRouter (2025a). *OpenRouter - Privacy Policy*. Accedido: 07-07-2025. URL: <https://openrouter.ai/privacy>.
- (2025b). *OpenRouter - Terms of Service*. Accedido: 07-07-2025. URL: <https://openrouter.ai/terms>.
- Snyk, Ltd. (2025). *Snyk Plans & Pricing*. <https://snyk.io/plans/>. Accedido: 08-07-2025.
- GitHub (s.f.-a). *AI Code Reviews*. <https://github.com/resources/articles/ai/ai-code-reviews>. Accedido: 06-05-2025.
- (s.f.-b). *GitHub Copilot*. <https://github.com/features/copilot>. Accedido: 06-05-2025.
- Snyk (s.f.). *Snyk Documentation*. Accedido: 06-05-2025.
- SonarSource (s.f.). *SonarQube Documentation*. Accedido: 06-05-2025.

Anexos

Apéndice A

Archivo Docker Compose

```
1 services:
2   jenkins:
3     build: .
4     image: ${IMAGE_NAME}
5     container_name: ${CONTAINER_NAME}
6     restart: always
7     ports:
8       - "8080:8080"
9       - "50000:50000"
10    volumes:
11      - jenkins_data:/var/jenkins_home
12      - /var/run/docker.sock:/var/run/docker.sock
13    environment:
14      - JENKINS_ADMIN_PASSWORD=${JENKINS_ADMIN_PASSWORD}
15      - JAVA_OPTS=${JAVA_OPTS}
16      - PIPELINE_NAME=${PIPELINE_NAME}
17      - REPO_URL=${REPO_URL}
18      - BRANCH=${BRANCH}
19      - JENKINSFILE_PATH=${JENKINSFILE_PATH}
20
21 volumes:
22   jenkins_data:
```

Listing A.1: Contenido del archivo docker-compose.yml para levantar el entorno.

Apéndice B

Archivo Jenkinsfile

```
1 /**
2  * =====
3  * PIPELINE DE DEVSECOPS CON ANÁLISIS IA
4  * =====
5  * * Este pipeline implementa un ciclo de vida de CI/CD para una aplicación Java
6  * ,
7  * integrando la seguridad como un componente central a través de las siguientes
8  * fases:
9  * * 1. Compilación y Pruebas Unitarias.
10 * 2. Análisis Estático de Seguridad (SAST) con SpotBugs.
11 * 3. Análisis Avanzado de Seguridad y Calidad con un modelo de IA.
12 * 4. Quality Gate: Detención automática del pipeline si se encuentran
13 * vulnerabilidades críticas.
14 * 5. Construcción y Despliegue de la aplicación como un contenedor Docker.
15 * *
16 * =====
17 */
18 pipeline {
19     // Define que el pipeline puede ejecutarse en cualquier agente de Jenkins
20     // disponible.
21     // Para entornos de producción, se recomienda usar etiquetas para
22     // seleccionar agentes específicos.
23     agent any
24
25     // --- Variables de Entorno Globales ---
26     // Definen parámetros y rutas consistentes a lo largo de todo el pipeline.
27     environment {
28         // Directorio para almacenar todos los artefactos de análisis generados.
29         SCAN_RESULTS_DIR = 'scan-results'
```

```
24     // Nombre del proyecto, puede usarse para etiquetar artefactos o imágenes.
25     PROJECT_NAME = 'demo'
26     // Directorio del código fuente a analizar. Se usa una ruta absoluta para mayor robustez.
27     JAVA_ANALYSIS_DIR    = "${env.WORKSPACE}/demo/src"
28     // Límite de archivos a pasar al script de análisis de IA.
29     MAX_FILES_TO_ANALYZE = '10'
30 }
31
32 // --- Etapas del Pipeline ---
33 // Cada 'stage' representa una fase lógica y distinta del proceso de CI/CD.
34 stages {
35
36     // Etapa 1: Preparación y Compilación
37     // Prepara el entorno, instala dependencias y compila la aplicación.
38     stage('Compile & Prepare Environment') {
39         steps {
40             echo '>>> Stage: Compilando la aplicación y preparando el entorno...'
41
42             // Verificación de versiones para diagnóstico.
43             sh "java -version"
44             sh "python3 --version"
45
46             // Instala las dependencias de Python de forma idempotente.
47             sh "pip3 install requests pathlib --break-system-packages || pip3 install requests pathlib || echo 'Dependencies already installed'"
48
49             // Crea el directorio de resultados si no existe.
50             sh "mkdir -p ${SCAN_RESULTS_DIR}"
51
52             // Entra al directorio del proyecto Java para ejecutar Maven.
53             dir('demo') {
54                 // Compila y empaqueta la aplicación.
55                 // -Dmaven.test.failure.ignore=true asegura que el pipeline continúe al análisis
56                 // de seguridad incluso si las pruebas unitarias fallan, priorizando la seguridad.
57                 sh "mvn -Dmaven.test.failure.ignore=true clean package"
58             }
59         }
60     }
61 }
```

```
60     }
61
62     // Etapa 2: Pruebas Unitarias
63     // Analiza y publica los resultados de las pruebas unitarias.
64     stage('Unit Test Analysis') {
65         steps {
66             echo '>>> Stage: Publicando resultados de pruebas unitarias...'
67
68             // El paso 'junit' parsea los reportes XML y los integra en la
69             // UI de Jenkins,
70             // mostrando tendencias, fallos y duración de las pruebas.
71             junit 'demo/target/surefire-reports/*.xml'
72         }
73
74         // Etapa 3: Análisis Estático de Seguridad (SAST)
75         // Ejecuta herramientas de análisis estático en paralelo para optimizar
76         // el tiempo.
77         stage('Static Analysis') {
78             parallel {
79                 // Ejecuta SpotBugs para encontrar patrones de error comunes en
80                 // el código Java.
81                 stage('SpotBugs SAST') {
82                     steps {
83                         dir('demo') {
84                             script {
85                                 // El bloque try-catch añade resiliencia. Si
86                                 // SpotBugs falla,
87                                 // el pipeline no se detendrá, solo se registrar
88                                 // á el error.
89                                 try {
90                                     sh "mvn compile spotbugs:spotbugs"
91                                     sh "cp target/spotbugsXml.xml ../${
92                                         SCAN_RESULTS_DIR}/ || echo 'No SpotBugs report found'"
93                                     } catch (Exception e) {
94                                         echo "SpotBugs analysis failed: ${e.
95                                             getMessage()}"
96                                     }
97                                     sh "echo '<BugCollection></BugCollection>' >
98                                         ../${SCAN_RESULTS_DIR}/spotbugsXml.xml"
99                                 }
100                             }
101                         }
102                     }
103                 }
104             }
105         }
106     }
107 }
```

```
94         }
95     }
96
97     // Recopila los archivos fuente para la siguiente etapa.
98     stage('Collect Source Files') {
99         steps {
100             script {
101                 sh """
102                     find ${JAVA_ANALYSIS_DIR} -name "*.java" -type f
103 > ${SCAN_RESULTS_DIR}/java-files.txt
104                     echo "Archivos Java encontrados para análisis:"
105                     cat ${SCAN_RESULTS_DIR}/java-files.txt
106                 """
107             }
108         }
109     }
110 }
111
112 // Etapa 4: Análisis con Inteligencia Artificial
113 // Ejecuta el script de Python personalizado para un análisis de
seguridad profundo.
114 stage('AI-Powered Analysis') {
115     steps {
116         script {
117             // Copia el script de análisis al directorio de resultados
para aislar la ejecución.
118             sh "cp demo/ai_code_analyzer.py ${SCAN_RESULTS_DIR}/"
119
120             // Ejecuta los siguientes pasos desde dentro del directorio
de resultados.
121             dir(env.SCAN_RESULTS_DIR) {
122                 // withCredentials es el mecanismo seguro de Jenkins
para manejar secretos.
123                 // Inyecta la credencial 'openrouter-api-key' en una
variable de entorno
124                 // temporal que solo existe dentro de este bloque.
125                 withCredentials([string(credentialsId: 'openrouter-api-
key', variable: 'OPENROUTER_API_KEY')]) {
126                     sh """
127                         echo "Iniciando análisis con IA..."
128                         python3 ai_code_analyzer.py \\\
```

```
129         --directory ${JAVA_ANALYSIS_DIR} \\  
130         --max-files ${MAX_FILES_TO_ANALYZE}  
131         ""  
132     }  
133 }  
134 }  
135 }  
136 }  
137  
138 // Etapa 5: Quality Gate (Control de Calidad)  
139 // Punto de control que detiene el pipeline si no se cumplen las polí  
140 ticas de seguridad.  
141 stage('Quality Gate') {  
142     steps {  
143         script {  
144             echo '>>> Stage: Verificando políticas de calidad y  
145             seguridad...'  
146             def resultsFile = "${env.SCAN_RESULTS_DIR}/analysis-results.  
147             json"  
148  
149             if (fileExists(resultsFile)) {  
150                 // Lee el archivo JSON generado por el script de IA.  
151                 def analysisResults = readJSON(file: resultsFile)  
152                 def highSeverityVulns = analysisResults.summary.  
153                 high_severity_vulnerabilities  
154  
155                 echo "-> Vulnerabilidades de severidad ALTA encontradas:  
156                 ${highSeverityVulns}"  
157  
158                 // --- Política de Seguridad: Cero Tolerancia para  
159                 Vulnerabilidades Altas ---  
160                 if (highSeverityVulns > 0) {  
161                     // El paso 'error' detiene el pipeline  
162                     inmediatamente y lo marca como FAILURE.  
163                     // Esto previene que código inseguro sea construido  
164                     o desplegado.  
165                     error("QUALITY GATE FAILED: Se encontraron ${  
166                     highSeverityVulns} vulnerabilidades de severidad ALTA. Abortando el pipeline  
167                     .")  
168                 } else {  
169                     echo "QUALITY GATE PASSED: No se encontraron  
170                     vulnerabilidades de severidad ALTA."
```

```
160         }
161
162         } else {
163             // Si el archivo de resultados no existe, es un fallo cr
164             ítico del proceso de análisis.
165             error("QUALITY GATE FAILED: No se encontró el archivo de
166             resultados '${resultsFile}'. No se puede verificar la calidad.")
167         }
168     }
169
170     // Etapa 6: Construcción de Imagen Docker
171     // Esta etapa solo se ejecuta si el Quality Gate fue superado.
172     stage('Build Docker Image') {
173         steps {
174             echo '>>> Stage: Construyendo la imagen Docker de la aplicación
175             ...'
176             script {
177                 // El comando 'docker build' usa el Dockerfile de la
178                 aplicación.
179                 // -f especifica la ruta del Dockerfile.
180                 // -t etiqueta la imagen con un nombre y tag (ej: demo-app:
181                 latest).
182                 // . define el contexto de construcción (el directorio
183                 actual).
184                 sh 'docker build -f demo/Dockerfile -t demo-app .'
185             }
186         }
187     }
188
189     // Etapa 7: Despliegue en Entorno Local
190     // Despliega la imagen construida como un contenedor local.
191     stage('Deploy to Local Environment') {
192         steps {
193             echo '>>> Stage: Desplegando localmente con Docker...'
194
195             // Detiene y elimina cualquier contenedor anterior con el mismo
196             nombre.
197             // '|| true' evita que el pipeline falle si el contenedor no
198             existe.
199             sh 'docker stop demo-app || true'
```

```
194         sh 'docker rm demo-app || true'
195
196         // Ejecuta el nuevo contenedor.
197         // -d: modo detached (en segundo plano).
198         // -p 8081:9090 mapea el puerto 9090 del contenedor al 8081 del
host.
199         // --name asigna un nombre al contenedor para fácil referencia.
200         sh 'docker run -d -p 8081:9090 --name demo-app demo-app'
201
202         echo '>>> El microservicio debería estar accesible en http://
localhost:8081'
203     }
204 }
205 }
206
207 // --- Acciones Post-Ejecución ---
208 // Se ejecutan al final del pipeline, sin importar el resultado de las
etapas.
209 post {
210     // 'always' se ejecuta siempre, garantizando que los reportes y
artefactos
211     // siempre se publiquen, lo cual es crucial para la trazabilidad y el an
álisis de fallos.
212     always {
213         echo '>>> Post-execution: Publicando reportes y archivando
artefactos...'
214
215         // Publica los resultados de JUnit.
216         junit(
217             allowEmptyResults: true,
218             testResults: 'demo/target/surefire-reports/*.xml'
219         )
220
221         // Publica el reporte HTML del análisis de IA en la página del build
.
222         publishHTML([
223             allowMissing: false,
224             alwaysLinkToLastBuild: true,
225             keepAll: true,
226             reportDir: env.SCAN_RESULTS_DIR,
227             reportFiles: 'ai-analysis-report.html',
228             reportName: 'AI Security & Quality Report'
```

```
229         })
230
231         // Archiva los reportes JSON y HTML como artefactos del build para
232         su descarga.
233         archiveArtifacts(
234             artifacts: "${env.SCAN_RESULTS_DIR}/*.json, ${env.
235             SCAN_RESULTS_DIR}/*.html",
236             allowEmptyArchive: true
237         )
238     }
239
240     // Bloques para notificaciones contextuales (ej. Slack, Email, Teams).
241     success {
242         echo 'Pipeline completado exitosamente!'
243     }
244     failure {
245         echo 'Pipeline falló! Revisar los logs.'
246     }
247     unstable {
248         echo 'Pipeline completado con advertencias (inestable).'
249     }
250 }
```

Listing B.1: Estructura completa del Jenkinsfile.

Apéndice C

Archivo ai_analyzer.py

```
1 import json
2 import os
3 import requests
4 import sys
5 import argparse
6 from pathlib import Path
7 from datetime import datetime
8 import hashlib
9
10 class OpenRouterAnalyzer:
11     def __init__(self):
12         # Leer la API key desde variable de entorno
13         self.api_key = os.getenv('OPENROUTER_API_KEY')
14
15         # Verificar que la API key esté disponible
16         if not self.api_key:
17             print("ERROR: La variable de entorno OPENROUTER_API_KEY no está
18 configurada.")
19             print("Variables de entorno disponibles:")
20             for key, value in os.environ.items():
21                 if 'OPENROUTER' in key.upper():
22                     print(f" {key}: {value[:10]}...")
23             raise ValueError("OPENROUTER_API_KEY no encontrada en las variables
24 de entorno")
25
26         print(f"API Key cargada correctamente: {self.api_key[:10]}...")
27
28         self.base_url = "https://openrouter.ai/api/v1"
29         self.headers = {
```

```
28         "Authorization": f"Bearer {self.api_key}",
29         "Content-Type": "application/json",
30         "HTTP-Referer": "http://localhost:8080",
31         "X-Title": "Jenkins CI/CD Security Scanner"
32     }
33
34     def analyze_code_security(self, code_content, filename):
35         """Analizar código Java para vulnerabilidades de seguridad"""
36         prompt = f"""
37 Actúa como un experto en seguridad de aplicaciones Java. Analiza el siguiente código fuente para identificar vulnerabilidades de seguridad:
38
39 Archivo: {filename}
40
41 Busca específicamente:
42 1. Inyección SQL
43 2. Cross-Site Scripting (XSS)
44 3. Problemas de autenticación/autorización
45 4. Validación de entrada insuficiente
46 5. Exposición de información sensible
47 6. Vulnerabilidades de deserialización
48 7. Uso inseguro de criptografía
49 8. Path traversal
50 9. Command injection
51 10. Problemas de gestión de sesiones
52
53 Código a analizar:
54 ```java
55 {code_content}
56 ```
57
58 Responde en formato JSON con la siguiente estructura:
59 {{
60     "vulnerabilities": [
61         {{
62             "type": "tipo de vulnerabilidad",
63             "severity": "HIGH|MEDIUM|LOW",
64             "line": "número de línea aproximado",
65             "description": "descripción detallada",
66             "recommendation": "cómo solucionarlo",
67             "code_correction_suggested": "codigo para solucionar la vulnerabilidad",
```

```

68         "cwe_id": "CWE-XXX si aplica",
69         "impact": "impacto potencial de la vulnerabilidad"
70     }}
71 ],
72     "security_score": "puntuación del 0-10",
73     "summary": "resumen ejecutivo de los hallazgos"
74 }}
75
76
77     return self._call_api(prompt, "security")
78
79     def analyze_code_quality(self, code_content, filename):
80         """Analizar calidad del código Java"""
81         prompt = f"""
82 Actúa como un experto en calidad de código Java. Analiza el siguiente código
83 para identificar problemas de calidad:
84
85 Archivo: {filename}
86
87 Evalúa:
88 1. Code smells y anti-patrones
89 2. Complejidad ciclomática alta
90 3. Duplicación de código
91 4. Problemas de mantenibilidad
92 5. Violaciones de principios SOLID
93 6. Uso inadecuado de patrones de diseño
94 7. Gestión de excepciones
95 8. Nomenclatura y convenciones
96 9. Eficiencia y rendimiento
97 10. Cobertura y calidad de comentarios
98
99 Código a analizar:
100 ```java
101 {code_content}
102 ```
103 Responde en formato JSON:
104 {{
105     "quality_issues": [
106         {{
107             "type": "tipo de problema",
108             "severity": "HIGH|MEDIUM|LOW",

```

```

109         "line": "número de línea aproximado",
110         "description": "descripción del problema",
111         "recommendation": "mejora sugerida",
112         "code_correction_suggested": "codigo para solucionar el problema",
113         "category": "Maintainability|Reliability|Performance|Documentation",
114         "effort": "tiempo estimado para solucionar (minutos)"
115     }}
116 ],
117 "quality_score": "puntuación del 0-10",
118 "maintainability_index": "índice de mantenibilidad",
119 "complexity_score": "puntuación de complejidad",
120 "technical_debt": "deuda técnica estimada en minutos"
121 }}
122
123
124     return self._call_api(prompt, "quality")
125
126 def _call_api(self, prompt, analysis_type):
127     """Llamar a la API de OpenRouter"""
128     payload = {
129         "model": "deepseek/deepseek-chat",
130         "messages": [
131             {
132                 "role": "user",
133                 "content": prompt
134             }
135         ],
136         "temperature": 0.1,
137         "max_tokens": 4000
138     }
139
140     try:
141         response = requests.post(
142             f"{self.base_url}/chat/completions",
143             headers=self.headers,
144             json=payload,
145             timeout=60
146         )
147         # --- INICIO DE LÍNEAS DE DEPURACIÓN A AGREGAR ---
148         print(f"--- DEBUG: Respuesta de la API para '{analysis_type}' ---")
149         print(f"Status Code: {response.status_code}")
150         print(f"Response Body (Raw): {response.text}")

```

```
151         print("-----")
152         # --- FIN DE LÍNEAS DE DEPURACIÓN A AGREGAR ---
153         if response.status_code == 200:
154             result = response.json()
155             content = result["choices"][0]["message"]["content"]
156
157             # Intentar extraer JSON del contenido
158             try:
159                 # Buscar JSON en el contenido
160                 start = content.find("{")
161                 end = content.rfind("}") + 1
162                 if start != -1 and end > start:
163                     json_content = content[start:end]
164                     return json.loads(json_content)
165                 else:
166                     return {"error": "No se pudo extraer JSON válido", "
raw_content": content}
167             except json.JSONDecodeError:
168                 return {"error": "JSON inválido", "raw_content": content}
169             else:
170                 return {"error": f"API Error: {response.status_code}", "message"
: response.text}
171
172         except Exception as e:
173             return {"error": f"Exception: {str(e)}"}
174
175     def calculate_overall_score(self, ai_results):
176         """Calcular puntuación general del proyecto"""
177         total_security_score = 0
178         total_quality_score = 0
179         valid_files = 0
180
181         for result in ai_results.values():
182             if isinstance(result, dict) and not result.get('error'):
183                 if 'security_score' in result:
184                     try:
185                         total_security_score += float(result['security_score'])
186                         valid_files += 1
187                     except (ValueError, TypeError):
188                         pass
189                 if 'quality_score' in result:
190                     try:
```

```
191         total_quality_score += float(result['quality_score'])
192     except (ValueError, TypeError):
193         pass
194
195     if valid_files > 0:
196         avg_security_score = total_security_score / valid_files
197         avg_quality_score = total_quality_score / valid_files
198         overall_score = (avg_security_score + avg_quality_score) / 2
199         return overall_score, avg_security_score, avg_quality_score
200
201     return 0, 0, 0
202
203 def generate_report(self, ai_results):
204     """Generar reporte HTML comprensivo y moderno"""
205
206     # Calcular estadísticas generales
207     total_vulnerabilities = sum(len(result.get('vulnerabilities', [])) for
result in ai_results.values() if isinstance(result, dict))
208     total_quality_issues = sum(len(result.get('quality_issues', [])) for
result in ai_results.values() if isinstance(result, dict))
209
210     # Contar por severidad
211     high_severity_vulns = sum(1 for result in ai_results.values() if
isinstance(result, dict)
212                               for vuln in result.get('vulnerabilities', [])
213                               if vuln.get('severity') == 'HIGH')
214
215     medium_severity_vulns = sum(1 for result in ai_results.values() if
isinstance(result, dict)
216                                for vuln in result.get('vulnerabilities', [])
217                                if vuln.get('severity') == 'MEDIUM')
218
219     low_severity_vulns = sum(1 for result in ai_results.values() if
isinstance(result, dict)
220                              for vuln in result.get('vulnerabilities', [])
221                              if vuln.get('severity') == 'LOW')
222
223     # Calcular puntuación general
224     overall_score, avg_security_score, avg_quality_score = self.
calculate_overall_score(ai_results)
225
226     # Determinar el estado del quality gate
```

```
227     quality_gate_status = "PASSED" #if high_severity_vulns == 0 and
overall_score >= 7 else "FAILED"
228
229     # Generar timestamp
230     timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
231
232     html_content = f"""
233 <!DOCTYPE html>
234 <html lang="es">
235 <head>
236     <meta charset="UTF-8">
237     <meta name="viewport" content="width=device-width, initial-scale=1.0">
238     <title>AI Code Analysis Report</title>
239     <style>
240         * {{
241             margin: 0;
242             padding: 0;
243             box-sizing: border-box;
244         }}
245
246         body {{
247             font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI', Roboto,
Oxygen, Ubuntu, Cantarell, sans-serif;
248             line-height: 1.6;
249             color: #333;
250             background: linear-gradient(135deg, #f5f7fa 0%, #c3cfe2 100%);
251             min-height: 100vh;
252         }}
253
254         .container {{
255             max-width: 1400px;
256             margin: 0 auto;
257             padding: 20px;
258         }}
259
260         .header {{
261             background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
262             color: white;
263             padding: 30px;
264             border-radius: 20px;
265             margin-bottom: 30px;
266             box-shadow: 0 10px 30px rgba(0,0,0,0.1);
```

```
267     }}
268
269     .header h1 {{
270         font-size: 2.5rem;
271         margin-bottom: 10px;
272         font-weight: 700;
273     }}
274
275     .header .subtitle {{
276         font-size: 1.1rem;
277         opacity: 0.9;
278         margin-bottom: 20px;
279     }}
280
281     .meta-info {{
282         display: flex;
283         gap: 30px;
284         font-size: 0.9rem;
285         opacity: 0.8;
286     }}
287
288     .quality-gate {{
289         background: {'#d4edda' if quality_gate_status == 'PASSED' else '#f8d7da'};
290         color: {'#155724' if quality_gate_status == 'PASSED' else '#721c24'};
291         padding: 20px;
292         border-radius: 15px;
293         margin-bottom: 30px;
294         border: 3px solid {'#28a745' if quality_gate_status == 'PASSED' else '#dc3545'};
295         text-align: center;
296     }}
297
298     .quality-gate h2 {{
299         font-size: 1.5rem;
300         margin-bottom: 10px;
301     }}
302
303     .metrics-grid {{
304         display: grid;
305         grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));
```

```
306         gap: 20px;
307         margin-bottom: 30px;
308     }}
309
310     .metric-card {{
311         background: white;
312         padding: 25px;
313         border-radius: 15px;
314         box-shadow: 0 5px 20px rgba(0,0,0,0.08);
315         border-left: 5px solid #667eea;
316         transition: transform 0.3s ease;
317     }}
318
319     .metric-card:hover {{
320         transform: translateY(-5px);
321     }}
322
323     .metric-number {{
324         font-size: 3rem;
325         font-weight: 700;
326         margin-bottom: 10px;
327     }}
328
329     .metric-label {{
330         font-size: 0.9rem;
331         color: #666;
332         text-transform: uppercase;
333         letter-spacing: 0.5px;
334     }}
335
336     .score-card {{
337         background: linear-gradient(135deg, #4facfe 0%, #00f2fe 100%);
338         color: white;
339         border-left: none;
340     }}
341
342     .severity-high {{ color: #dc3545; }}
343     .severity-medium {{ color: #fd7e14; }}
344     .severity-low {{ color: #28a745; }}
345
346     .files-section {{
347         background: white;
```

```
348         border-radius: 15px;
349         padding: 30px;
350         margin-bottom: 30px;
351         box-shadow: 0 5px 20px rgba(0,0,0,0.08);
352     }}
353
354     .file-card {
355         background: #f8f9fa;
356         border-radius: 10px;
357         margin-bottom: 30px;
358         overflow: hidden;
359         border: 1px solid #e9ecef;
360     }
361
362     .file-header {
363         background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
364         color: white;
365         padding: 20px;
366         font-weight: 600;
367     }
368
369     .file-content {
370         padding: 25px;
371     }
372
373     .analysis-section {
374         margin-bottom: 30px;
375     }
376
377     .section-title {
378         font-size: 1.3rem;
379         margin-bottom: 20px;
380         color: #333;
381         border-bottom: 2px solid #667eea;
382         padding-bottom: 10px;
383     }
384
385     .issue-card {
386         background: white;
387         border-radius: 10px;
388         padding: 20px;
389         margin-bottom: 15px;
```

```
390         border-left: 4px solid #667eea;
391         box-shadow: 0 2px 10px rgba(0,0,0,0.05);
392     }}
393
394     .vulnerability-card {{
395         border-left-color: #dc3545;
396         background: linear-gradient(135deg, #fff5f5 0%, #ffffff 100%);
397     }}
398
399     .quality-card {{
400         border-left-color: #007bff;
401         background: linear-gradient(135deg, #f0f8ff 0%, #ffffff 100%);
402     }}
403
404     .issue-header {{
405         display: flex;
406         justify-content: space-between;
407         align-items: center;
408         margin-bottom: 15px;
409     }}
410
411     .issue-title {{
412         font-size: 1.1rem;
413         font-weight: 600;
414         color: #333;
415     }}
416
417     .severity-badge {{
418         padding: 4px 12px;
419         border-radius: 20px;
420         font-size: 0.8rem;
421         font-weight: 600;
422         text-transform: uppercase;
423     }}
424
425     .severity-high-bg {{
426         background: #dc3545;
427         color: white;
428     }}
429
430     .severity-medium-bg {{
431         background: #fd7e14;
```

```
432         color: white;
433     }}
434
435     .severity-low-bg {{
436         background: #28a745;
437         color: white;
438     }}
439
440     .issue-details {{
441         display: grid;
442         gap: 15px;
443     }}
444
445     .detail-item {{
446         display: flex;
447         align-items: flex-start;
448         gap: 10px;
449     }}
450
451     .detail-label {{
452         font-weight: 600;
453         color: #666;
454         min-width: 100px;
455     }}
456
457     .detail-content {{
458         flex: 1;
459     }}
460
461     .code-block {{
462         background: #282c34;
463         color: #abb2bf;
464         padding: 20px;
465         border-radius: 10px;
466         overflow-x: auto;
467         margin-top: 10px;
468         font-family: 'Consolas', 'Monaco', 'Courier New', monospace;
469         font-size: 0.9rem;
470         line-height: 1.4;
471     }}
472
473     .code-block pre {{
```

```
474         margin: 0;
475         white-space: pre-wrap;
476         word-wrap: break-word;
477     }}
478
479     .no-issues {{
480         text-align: center;
481         color: #28a745;
482         font-size: 1.1rem;
483         padding: 40px;
484         background: linear-gradient(135deg, #d4edda 0%, #ffffff 100%);
485         border-radius: 10px;
486         border: 2px solid #28a745;
487     }}
488
489     .footer {{
490         text-align: center;
491         padding: 20px;
492         color: #666;
493         font-size: 0.9rem;
494     }}
495
496     @media (max-width: 768px) {{
497         .metrics-grid {{
498             grid-template-columns: 1fr;
499         }}
500
501         .header h1 {{
502             font-size: 2rem;
503         }}
504
505         .meta-info {{
506             flex-direction: column;
507             gap: 10px;
508         }}
509     }}
510 </style>
511 </head>
512 <body>
513     <div class="container">
514         <div class="header">
515             <h1>AI Code Analysis Report</h1>
```

```

516         <div class="subtitle">Análisis inteligente de seguridad y calidad de
código</div>
517         <div class="meta-info">
518             <div>Generado: {timestamp}</div>
519             <div>Archivos analizados: {len(ai_results)}</div>
520             <div>Powered by AI</div>
521         </div>
522     </div>
523
524     <div class="quality-gate">
525         <h2>Quality Gate: {quality_gate_status}</h2>
526         <p>{'Tu código cumple con los estándares de calidad' if
quality_gate_status == 'PASSED' else 'Se requieren mejoras antes de producci
ón'}</p>
527     </div>
528
529     <div class="metrics-grid">
530         <div class="metric-card score-card">
531             <div class="metric-number">{overall_score:.1f}</div>
532             <div class="metric-label">Puntuación General</div>
533         </div>
534
535         <div class="metric-card">
536             <div class="metric-number severity-high">{high_severity_vulns}</
div>
537             <div class="metric-label">Vulnerabilidades Críticas</div>
538         </div>
539
540         <div class="metric-card">
541             <div class="metric-number">{total_vulnerabilities}</div>
542             <div class="metric-label">Total Vulnerabilidades</div>
543         </div>
544
545         <div class="metric-card">
546             <div class="metric-number">{total_quality_issues}</div>
547             <div class="metric-label">Problemas de Calidad</div>
548         </div>
549
550         <div class="metric-card">
551             <div class="metric-number">{avg_security_score:.1f}</div>
552             <div class="metric-label">Puntuación Seguridad</div>
553         </div>

```

```

554
555         <div class="metric-card">
556             <div class="metric-number">{avg_quality_score:.1f}</div>
557             <div class="metric-label">Puntuación Calidad</div>
558         </div>
559     </div>
560
561     <div class="files-section">
562         <h2>Análisis por Archivo</h2>
563     """
564
565     # Agregar resultados por archivo
566     for filename, results in ai_results.items():
567         if isinstance(results, dict):
568             html_content += f"""
569             <div class="file-card">
570                 <div class="file-header">
571                     <div>{filename}</div>
572                 </div>
573                 <div class="file-content">
574     """
575
576             # Análisis de Seguridad
577             html_content += """
578                 <div class="analysis-section">
579                     <h3 class="section-title">Análisis de Seguridad</h3>
580     """
581
582             vulnerabilities = results.get('vulnerabilities', [])
583             if vulnerabilities:
584                 for vuln in vulnerabilities:
585                     severity = vuln.get('severity', 'LOW')
586                     html_content += f"""
587                     <div class="issue-card vulnerability-card">
588                         <div class="issue-header">
589                             <div class="issue-title">{vuln.get('type', '
Unknown Vulnerability')}</div>
590                             <div class="severity-badge severity-{severity.
lower()}-bg">{severity}</div>
591                         </div>
592                         <div class="issue-details">
593                             <div class="detail-item">

```

```

594         <div class="detail-label">Línea:</div>
595         <div class="detail-content">{vuln.get('line
', 'N/A')}}</div>
596     </div>
597     <div class="detail-item">
598         <div class="detail-label">Descripción:</div>
599         <div class="detail-content">{vuln.get('
description', 'N/A')}}</div>
600     </div>
601     <div class="detail-item">
602         <div class="detail-label">Recomendación:</
div>
603         <div class="detail-content">{vuln.get('
recommendation', 'N/A')}}</div>
604     </div>
605     <div class="detail-item">
606         <div class="detail-label">Solución:</div>
607         <div class="detail-content">
608             <div class="code-block">
609                 <pre>{vuln.get('
code_correction_suggested', 'N/A')}}</pre>
610             </div>
611         </div>
612     </div>
613 </div>
614 </div>
615 """
616     else:
617         html_content += """
618         <div class="no-issues">
619             No se encontraron vulnerabilidades de seguridad
620         </div>
621     """
622
623     html_content += """
624     </div>
625 """
626
627     # Análisis de Calidad
628     html_content += """
629     <div class="analysis-section">
630         <h3 class="section-title"> Análisis de Calidad</h3>

```

```

631 """
632
633     quality_issues = results.get('quality_issues', [])
634     if quality_issues:
635         for issue in quality_issues:
636             severity = issue.get('severity', 'LOW')
637             html_content += f"""
638                 <div class="issue-card quality-card">
639                     <div class="issue-header">
640                         <div class="issue-title">{issue.get('type', '
Unknown Issue')}</div>
641                         <div class="severity-badge severity-{severity.
lower()}-bg">{severity}</div>
642                     </div>
643                     <div class="issue-details">
644                         <div class="detail-item">
645                             <div class="detail-label"> Línea:</div>
646                             <div class="detail-content">{issue.get('line
', 'N/A')}</div>
647                         </div>
648                         <div class="detail-item">
649                             <div class="detail-label">Descripción:</div>
650                             <div class="detail-content">{issue.get('
description', 'N/A')}</div>
651                         </div>
652                         <div class="detail-item">
653                             <div class="detail-label">Recomendación:</
div>
654                             <div class="detail-content">{issue.get('
recommendation', 'N/A')}</div>
655                         </div>
656                         <div class="detail-item">
657                             <div class="detail-label">Solución:</div>
658                             <div class="detail-content">
659                                 <div class="code-block">
660                                     <pre>{issue.get('
code_correction_suggested', 'N/A')}</pre>
661                                 </div>
662                             </div>
663                         </div>
664                     </div>
665                 </div>

```

```
666 """
667         else:
668             html_content += """
669                 <div class="no-issues">
670                     No se encontraron problemas significativos de
671 calidad
672                 </div>
673 """
674             html_content += """
675                 </div>
676             </div>
677         </div>
678 """
679
680         html_content += """
681         </div>
682
683         <div class="footer">
684             <p>Reporte generado automáticamente por AI Code Analyzer</p>
685             <p>Para más información sobre las vulnerabilidades, consulta OWASP
686 Top 10 y CWE</p>
687         </div>
688 </body>
689 </html>
690 """
691
692         # Guardar archivo HTML
693         with open("ai-analysis-report.html", "w", encoding="utf-8") as f:
694             f.write(html_content)
695
696         # También generar JSON para quality gates
697         report_json = {
698             "timestamp": timestamp,
699             "overall_score": overall_score,
700             "security_score": avg_security_score,
701             "quality_score": avg_quality_score,
702             "quality_gate_status": quality_gate_status,
703             "summary": {
704                 "total_vulnerabilities": total_vulnerabilities,
705                 "high_severity_vulnerabilities": high_severity_vulns,
```

```
706         "medium_severity_vulnerabilities": medium_severity_vulns,
707         "low_severity_vulnerabilities": low_severity_vulns,
708         "total_quality_issues": total_quality_issues,
709         "files_analyzed": len(ai_results)
710     },
711     "detailed_results": ai_results
712 }
713
714 with open("analysis-results.json", "w", encoding="utf-8") as f:
715     json.dump(report_json, f, indent=2, ensure_ascii=False)
716
717 def find_java_files(directory):
718     """Buscar archivos Java en un directorio"""
719     directory_path = Path(directory)
720
721     if not directory_path.exists():
722         print(f"El directorio {directory} no existe")
723         return []
724
725     if not directory_path.is_dir():
726         print(f"{directory} no es un directorio válido")
727         return []
728
729     java_files = list(directory_path.rglob("*.java"))
730     print(f"Buscando en: {directory_path.absolute()}")
731     print(f"    Archivos Java encontrados: {len(java_files)}")
732
733     return java_files
734
735 def get_analysis_directory():
736     """Obtener el directorio a analizar desde argumentos o variables de entorno"""
737
738     # Prioridad 1: Argumento de línea de comandos
739     parser = argparse.ArgumentParser(description='Analizador de código Java con IA')
740     parser.add_argument('--directory', '-d',
741                         help='Directorio a analizar (por defecto: busca en directorio actual)')
742     parser.add_argument('--max-files', '-m', type=int, default=5,
743                         help='Número máximo de archivos a analizar (por defecto: 5)')
```

```
744
745     args = parser.parse_args()
746
747     # Prioridad 2: Variable de entorno
748     env_directory = os.getenv('JAVA_ANALYSIS_DIR')
749
750     # Prioridad 3: Directorio por defecto
751     if args.directory:
752         analysis_dir = args.directory
753         print(f"Directorio especificado por argumento: {analysis_dir}")
754     elif env_directory:
755         analysis_dir = env_directory
756         print(f"Directorio especificado por variable de entorno: {analysis_dir}")
757     )
758     else:
759         # Buscar en directorio actual y subdirectorios
760         analysis_dir = "."
761         print(f"Usando directorio por defecto: {Path('.').absolute()}")
762
763     return analysis_dir, args.max_files
764
765 def main():
766     # Obtener y mostrar el directorio actual
767     current_directory = Path.cwd()
768     print(f"Directorio actual: {current_directory}")
769
770     # Listar archivos en el directorio actual
771     print("\nArchivos en el directorio actual:")
772     for file in current_directory.iterdir():
773         print(f"- {file.name}")
774
775     analyzer = OpenRouterAnalyzer()
776
777     # Obtener directorio a analizar
778     analysis_directory, max_files = get_analysis_directory()
779
780     # Buscar archivos Java
781     print(f"\nBuscando archivos Java en: {analysis_directory}")
782     java_files = find_java_files(analysis_directory)
783
784     if not java_files:
785         print(f"No se encontraron archivos Java en {analysis_directory}")
```

```
785     print("Asegúrate de que el directorio contenga archivos .java")
786
787     # Mostrar estructura de directorios para debugging
788     analysis_path = Path(analysis_directory)
789     if analysis_path.exists():
790         print(f"\nEstructura del directorio {analysis_directory}:")
791         for item in analysis_path.rglob("*"):
792             if item.is_file():
793                 print(f"    - {item.relative_to(analysis_path)}")
794
795     sys.exit(1)
796
797     # Limitar número de archivos a analizar
798     files_to_analyze = java_files[:max_files]
799     if len(java_files) > max_files:
800         print(f"Limitando análisis a {max_files} archivos de {len(java_files)} encontrados")
801
802     print(f"\nAnalizando {len(files_to_analyze)} archivos Java con IA...")
803
804     ai_results = {}
805
806     for java_file in files_to_analyze:
807         try:
808             with open(java_file, 'r', encoding='utf-8') as f:
809                 code_content = f.read()
810
811                 if len(code_content.strip()) == 0:
812                     continue
813
814                 print(f"Analizando: {java_file}")
815
816                 # Análisis de seguridad
817                 security_result = analyzer.analyze_code_security(code_content, str(
java_file))
818
819                 # Análisis de calidad
820                 quality_result = analyzer.analyze_code_quality(code_content, str(
java_file))
821
822                 # Combinar resultados
823                 combined_result = {}
```

```
824         if isinstance(security_result, dict):
825             combined_result.update(security_result)
826         if isinstance(quality_result, dict):
827             combined_result.update(quality_result)
828
829         ai_results[str(java_file)] = combined_result
830
831     except Exception as e:
832         print(f"Error analizando {java_file}: {e}")
833         ai_results[str(java_file)] = {"error": str(e)}
834
835     # Generar reporte
836     analyzer.generate_report(ai_results)
837
838     print("Análisis completado. Reportes generados:")
839     print("- ai-analysis-report.html")
840     print("- analysis-results.json")
841
842 if __name__ == "__main__":
843     main()
```

Listing C.1: Contenido completo del script ai_analyzer.py.

Apéndice D

Reporte de vulnerabilidades mediante IA

AI Code Analysis Report

Análisis inteligente de seguridad y calidad de código

 Fecha de Análisis
2025-07-08 12:50:22

 Duración del Análisis
1m 37.9s

 Modelo de IA
google/gemini-2.0-flash-exp.free

 Archivos Analizados
5 total (4 Java + 1 POM)

 **Quality Gate: FAILED**

✗ Se requieren mejoras antes de producción

5.0

PUNTUACIÓN GENERAL

1m 37.9s

TIEMPO DE ANÁLISIS

5

VULNERABILIDADES
CRÍTICAS

21

TOTAL
VULNERABILIDADES

23

PROBLEMAS DE CALIDAD

5.2

PUNTUACIÓN SEGURIDAD

Análisis por Archivo

 ..demo/src/main/java/com/demo/DemoApplication.java

JAVA

Análisis de Seguridad

Lack of Security Controls

LOW

 Ubicación: 1-11

 Descripción: The provided code snippet only contains the basic Spring Boot application setup. It lacks any explicit security controls, such as authentication, authorization, input validation, or output encoding. While Spring Boot provides some default security features, they are not enabled or configured in this basic setup. This means the application is vulnerable to various attacks if further code is added without proper security considerations.

 Impacto: The application is vulnerable to various attacks if further code is added without proper security considerations. This could lead to data breaches, unauthorized access, and other security incidents.

 Recomendación: Implement security measures such as authentication, authorization, input validation, output encoding, and protection against common web vulnerabilities. Consider using Spring Security for authentication and authorization. Implement proper input validation to prevent injection attacks. Use output encoding to prevent XSS attacks.

 Solución:

```
// Add Spring Security dependency to pom.xml or build.gradle
// Implement authentication and authorization using Spring Security
// Implement input validation using annotations or custom validators
// Implement output encoding using libraries like OWASP Java Encoder
```

⚡ Análisis de Calidad

Lack of Comments

LOW

📌 Línea: 1

📖 Descripción: The class and method lack Javadoc comments explaining their purpose and usage. While this is a simple application entry point, adding comments improves readability and maintainability, especially for larger projects.

🏷️ Categoría: Documentation

🕒 Esfuerzo: 5 min

💡 Recomendación: Add Javadoc comments to the class and main method.

🔧 Solución:

```
/**
 * Entry point for the Demo Application.
 */
@SpringBootApplication
public class DemoApplication {
    /**
     * Main method to start the Spring Boot application.
     * @param args Command line arguments.
     */
    public static void main(String[] args) {
        SpringApplication.run(DemoApplication.class, args);
    }
}
```

Missing Exception Handling

LOW

📌 Línea: 7

📖 Descripción: While SpringApplication.run handles exceptions internally, for more complex applications, explicit exception handling around the application startup might be beneficial for logging or custom error handling.

🏷️ Categoría: Reliability

🕒 Esfuerzo: 10 min

💡 Recomendación: Consider adding a try-catch block around SpringApplication.run for custom exception handling, especially if the application has specific startup requirements.

🔧 Solución:

```
try {
    SpringApplication.run(DemoApplication.class, args);
} catch (Exception e) {
    System.err.println("Application failed to start: " + e.getMessage());
    // Log the exception properly using a logger
    System.exit(1); // Exit with an error code
}
```

📄 ../demo/src/main/java/com/demo/controllers/WelcomeController.java

JAVA

🔒 Análisis de Seguridad

SQL Injection

HIGH

📌 Ubicación: 40

 **Descripción:** La consulta SQL se construye concatenando directamente la entrada del usuario ('username') sin sanitizarla. Esto permite a un atacante inyectar código SQL malicioso.

 **Impacto:** Acceso no autorizado a la base de datos, modificación o eliminación de datos, e incluso ejecución de comandos en el servidor.

 **Recomendación:** Utilizar consultas preparadas (PreparedStatement) para parametrizar la consulta SQL y evitar la inyección.

 **Solución:**

```
'''java
Connection conn = DriverManager.getConnection("jdbc:mysql://localhost:3306/demo", "root", DB_PASSWORD);
PreparedStatement stmt = conn.prepareStatement("SELECT * FROM users WHERE username = ?");
stmt.setString(1, username);
ResultSet rs = stmt.executeQuery();
'''
```

Cross-Site Scripting (XSS)

MEDIUM

 **Ubicación:** 24

 **Descripción:** El parámetro 'name' se incluye directamente en la respuesta HTML sin codificación. Esto permite a un atacante inyectar código JavaScript malicioso que se ejecutará en el navegador de otros usuarios.

 **Impacto:** Robo de cookies, redirección a sitios web maliciosos, defacement del sitio web.

 **Recomendación:** Codificar la salida HTML para evitar la ejecución de código JavaScript no deseado. Utilizar la librería 'org.springframework.web.util.HtmlUtils' para escapar los caracteres especiales.

 **Solución:**

```
'''java
import org.springframework.web.util.HtmlUtils;

@GetMapping("/api/welcome")
public WelcomeDTO welcome(@RequestParam(value = "name", defaultValue = "...") String name) {
    return new WelcomeDTO("Hola, bienvenido " + HtmlUtils.htmlEscape(name) + ", esto es un demo");
}
'''
```

Remote Code Execution (RCE)

CRITICAL

 **Ubicación:** 29

 **Descripción:** El código proporcionado por el usuario se evalúa directamente utilizando 'ScriptEngine'. Esto permite a un atacante ejecutar código arbitrario en el servidor.

 **Impacto:** Control total del servidor, compromiso de datos, denegación de servicio.

 **Recomendación:** Evitar la ejecución de código proporcionado por el usuario. Si es absolutamente necesario, utilizar un entorno de ejecución aislado (sandbox) con permisos muy restringidos.

 **Solución:**

Eliminar la funcionalidad o implementar un sandbox seguro para la ejecución del script.

Path Traversal

HIGH

 **Ubicación:** 34

 **Descripción:** El parámetro 'filePath' se utiliza directamente para leer un archivo sin validación. Esto permite a un atacante acceder a archivos fuera del directorio esperado.

 **Impacto:** Acceso no autorizado a archivos sensibles, divulgación de información.

 **Recomendación:** Validar y sanitizar la entrada 'filePath' para asegurar que solo se acceda a archivos dentro de un directorio seguro. Utilizar una lista blanca de archivos permitidos o un prefijo de ruta seguro.

 **Solución:**

```
'''java
@GetMapping("/api/read-file")
```

```
public String readFile(@RequestParam String filePath) throws Exception {
    Path safePath = Paths.get("/path/to/safe/directory").resolve(filePath).normalize();
    if (!safePath.startsWith(Paths.get("/path/to/safe/directory"))) {
        return "Acceso denegado";
    }
    return Files.readString(safePath);
}
...
```

Command Injection

HIGH

Ubicación: 48

Descripción: El parámetro 'host' se concatena directamente en un comando del sistema sin sanitizarlo. Esto permite a un atacante ejecutar comandos arbitrarios en el servidor.

Impacto: Control total del servidor, compromiso de datos, denegación de servicio.

Recomendación: Utilizar una API segura para ejecutar comandos del sistema o validar y sanitizar la entrada 'host' para evitar la inyección de comandos.

Solución:

```
... java
@PostMapping("/api/ping")
public String ping(@RequestBody String host) throws Exception {
    if (!isValidHost(host)) {
        return "Host inválido";
    }
    ProcessBuilder pb = new ProcessBuilder("ping", "-c", "1", host);
    Process p = pb.start();
    BufferedReader reader = new BufferedReader(new InputStreamReader(p.getInputStream()));
    StringBuilder out = new StringBuilder();
    String line;
    while ((line = reader.readLine()) != null) out.append(line).append("\n");
    return out.toString();
}

private boolean isValidHost(String host) {
    // Implementar validación robusta del host aquí
    // Por ejemplo, verificar que solo contenga caracteres alfanuméricos y puntos
    return host.matches("[a-zA-Z0-9.]*");
}
...
```

Insecure Deserialization

HIGH

Ubicación: 56

Descripción: Se deserializa un objeto a partir de una cadena Base64 proporcionada por el usuario. Esto permite a un atacante ejecutar código arbitrario en el servidor mediante la creación de un objeto malicioso.

Impacto: Ejecución remota de código, denegación de servicio.

Recomendación: Evitar la deserialización de datos proporcionados por el usuario. Si es absolutamente necesario, utilizar un formato de serialización seguro como JSON o implementar un filtro de deserialización.

Solución:

Eliminar la funcionalidad o utilizar un formato de serialización seguro como JSON.

Information Exposure through Logs

MEDIUM

Ubicación: 62

Descripción: Se registran las credenciales del usuario (nombre de usuario y contraseña) en los logs. Esto expone información sensible que puede ser utilizada por un atacante.

Impacto: Compromiso de credenciales, acceso no autorizado a cuentas.

Recomendación: Evitar registrar información sensible en los logs. Registrar solo información relevante para la depuración y el seguimiento de errores.

 Solución:

```
```java
logger.info("Intento de login con usuario: " + user + "");
```
```

Use of Weak Cryptographic Hash

MEDIUM

 Ubicación: 68

 Descripción: Se utiliza el algoritmo MD5 para generar un hash de los datos. MD5 es un algoritmo de hash débil que es vulnerable a colisiones y no debe utilizarse para fines de seguridad.

 Impacto: Compromiso de la integridad de los datos, suplantación de identidad.

 Recomendación: Utilizar un algoritmo de hash seguro como SHA-256 o SHA-3.

 Solución:

```
```java
MessageDigest md = MessageDigest.getInstance("SHA-256");
```
```

Use of Hardcoded Cryptographic Key

CRITICAL

 Ubicación: 74

 Descripción: Se utiliza una clave secreta codificada en el código para el cifrado AES. Esto permite a un atacante descifrar los datos cifrados.

 Impacto: Compromiso de la confidencialidad de los datos.

 Recomendación: No codificar claves secretas en el código. Utilizar un almacén de claves seguro o un servicio de gestión de claves para almacenar y gestionar las claves secretas. Además, utilizar un vector de inicialización (IV) aleatorio para cada operación de cifrado y un modo de operación autenticado como GCM.

 Solución:

```
```java
// Generar una clave aleatoria y almacenarla de forma segura
KeyGenerator keyGen = KeyGenerator.getInstance("AES");
keyGen.init(256); // Longitud de la clave
SecretKey secretKey = keyGen.generateKey();

// Generar un IV aleatorio
byte[] iv = new byte[16];
SecureRandom random = new SecureRandom();
random.nextBytes(iv);
IvParameterSpec ivParameterSpec = new IvParameterSpec(iv);

Cipher cipher = Cipher.getInstance("AES/GCM/NoPadding");
cipher.init(Cipher.ENCRYPT_MODE, secretKey, ivParameterSpec);
byte[] cipherText = cipher.doFinal(text.getBytes(StandardCharsets.UTF_8));

// Codificar el IV y el texto cifrado en Base64 para su almacenamiento o transmisión
ByteBuffer buffer = ByteBuffer.allocate(iv.length + cipherText.length);
buffer.put(iv);
buffer.put(cipherText);
byte[] combined = buffer.array();
String encryptedText = Base64.getEncoder().encodeToString(combined);

return encryptedText;
```
```

Information Exposure through Error Message

LOW

 Ubicación: 82

 Descripción: Se devuelve la traza de la pila de la excepción al usuario. Esto expone información sensible sobre la implementación interna de la aplicación.

 Impacto: Divulgación de información sensible, facilitación de ataques.

💡 Recomendación: No devolver la traza de la pila de la excepción al usuario. Devolver un mensaje de error genérico o un código de error específico.

🔧 Solución:

```
'''java
) catch (Exception e) {
    return "Error interno del servidor";
}
'''
```

Hardcoded Password

CRITICAL

📍 Ubicación: 15

📄 Descripción: La contraseña de la base de datos está codificada directamente en el código. Esto permite a un atacante acceder a la base de datos si obtiene acceso al código fuente.

⚡ Impacto: Acceso no autorizado a la base de datos, modificación o eliminación de datos.

💡 Recomendación: No codificar contraseñas en el código. Utilizar variables de entorno, archivos de configuración seguros o un servicio de gestión de secretos para almacenar y gestionar las contraseñas.

🔧 Solución:

Eliminar la contraseña codificada y utilizar una variable de entorno o un archivo de configuración seguro.

Hardcoded API Key

CRITICAL

📍 Ubicación: 16

📄 Descripción: La clave de la API está codificada directamente en el código. Esto permite a un atacante utilizar la API con privilegios no autorizados si obtiene acceso al código fuente.

⚡ Impacto: Uso no autorizado de la API, acceso a datos sensibles.

💡 Recomendación: No codificar claves de API en el código. Utilizar variables de entorno, archivos de configuración seguros o un servicio de gestión de secretos para almacenar y gestionar las claves de API.

🔧 Solución:

Eliminar la clave de API codificada y utilizar una variable de entorno o un archivo de configuración seguro.

Logging of Sensitive Data

MEDIUM

📍 Ubicación: 13

📄 Descripción: El logger se utiliza para registrar información sensible sin la debida precaución. Esto puede exponer información confidencial a personas no autorizadas.

⚡ Impacto: Divulgación de información sensible.

💡 Recomendación: Evitar registrar información sensible en los logs. Si es necesario registrar información sensible, utilizar un logger seguro y configurar el acceso a los logs de forma adecuada.

🔧 Solución:

Revisar el uso del logger y evitar registrar información sensible.

⚡ Análisis de Calidad

Hardcoded Password

CRITICAL

📍 Línea: 16

📄 Descripción: The database password 'SuperSecreta123!' is hardcoded in the source code. This is a major security vulnerability.

🔍 Categoría: Defectiva

📁 Categoría: Reliability

🕒 Esfuerzo: 60 min

💡 Recomendación: Store the password securely, such as in environment variables or a dedicated secrets management system.

🔧 Solución:

Eliminar la constante DB_PASSWORD y obtener el valor de una variable de entorno o un sistema de gestión de secretos.

Hardcoded API Key

CRITICAL

📍 Línea: 17

📄 Descripción: The API key 'ABC123-TOKEN-INSEGURO' is hardcoded in the source code. This is a major security vulnerability.

📁 Categoría: Reliability

🕒 Esfuerzo: 60 min

💡 Recomendación: Store the API key securely, such as in environment variables or a dedicated secrets management system.

🔧 Solución:

Eliminar la constante API_KEY y obtener el valor de una variable de entorno o un sistema de gestión de secretos.

Cross-Site Scripting (XSS)

CRITICAL

📍 Línea: 21

📄 Descripción: The 'welcome' method is vulnerable to XSS because it directly includes user-provided input ('name') in the response without proper sanitization or encoding.

📁 Categoría: Reliability

🕒 Esfuerzo: 120 min

💡 Recomendación: Sanitize or encode the user input before including it in the response. Use a templating engine that automatically escapes output.

🔧 Solución:

Utilizar un motor de plantillas que escape automáticamente la salida o escapar manualmente la entrada del usuario antes de incluirla en la respuesta.

Remote Code Execution (RCE)

CRITICAL

📍 Línea: 27

📄 Descripción: The 'executeCode' method allows arbitrary code execution by using 'ScriptEngine.eval' with user-provided input. This is a severe security risk.

📁 Categoría: Reliability

🕒 Esfuerzo: 240 min

💡 Recomendación: Avoid using 'ScriptEngine.eval' with user-provided input. If code execution is necessary, use a sandboxed environment with strict limitations.

🔧 Solución:

Eliminar la funcionalidad de ejecución de código arbitrario o implementar un entorno de pruebas con limitaciones estrictas.

Path Traversal

CRITICAL

📍 Línea: 33

📄 Descripción: The 'readFile' method is vulnerable to path traversal because it directly uses user-provided input ('filePath') to read files without proper validation. An attacker could read arbitrary files on the server.

📁 Categoría: Reliability

 Esfuerzo: 120 min

 Recomendación: Validate the user-provided file path to ensure it is within an allowed directory and does not contain path traversal sequences (e.g., '..').

 Solución:

Validar la ruta del archivo proporcionada por el usuario para asegurarse de que esté dentro de un directorio permitido y no contenga secuencias de recorrido de ruta (por ejemplo, '..').

SQL Injection

CRITICAL

 Línea: 39

 Descripción: The `getUserInfo` method is vulnerable to SQL injection because it directly includes user-provided input (`username`) in the SQL query without proper sanitization or parameterized queries.

 Categoría: Reliability

 Esfuerzo: 180 min

 Recomendación: Use parameterized queries or prepared statements to prevent SQL injection.

 Solución:

Utilizar consultas parametrizadas o sentencias preparadas para prevenir la inyección SQL.

Command Injection

CRITICAL

 Línea: 48

 Descripción: The `ping` method is vulnerable to command injection because it directly includes user-provided input (`host`) in the command executed by `Runtime.getRuntime().exec`. An attacker could execute arbitrary commands on the server.

 Categoría: Reliability

 Esfuerzo: 240 min

 Recomendación: Avoid using `Runtime.getRuntime().exec` with user-provided input. If command execution is necessary, use a safe API or library that properly escapes or validates the input.

 Solución:

Evitar el uso de `Runtime.getRuntime().exec` con la entrada proporcionada por el usuario. Si la ejecución del comando es necesaria, utilizar una API o biblioteca segura que escape o valide correctamente la entrada.

Insecure Deserialization

CRITICAL

 Línea: 56

 Descripción: The `deserialize` method is vulnerable to insecure deserialization because it deserializes user-provided data without proper validation. An attacker could execute arbitrary code by providing a malicious serialized object.

 Categoría: Reliability

 Esfuerzo: 240 min

 Recomendación: Avoid deserializing user-provided data. If deserialization is necessary, use a safe serialization format (e.g., JSON) or implement strict validation of the deserialized object.

 Solución:

Evitar la deserialización de datos proporcionados por el usuario. Si la deserialización es necesaria, utilizar un formato de serialización seguro (por ejemplo, JSON) o implementar una validación estricta del objeto deserializado.

Sensitive Data Logging

HIGH

 Línea: 62

 Descripción: The 'login' method logs the username and password. This is a security risk because sensitive information is exposed in the logs.

 Categoría: Reliability

 Esfuerzo: 60 min

 Recomendación: Avoid logging sensitive information. If logging is necessary, use a secure logging mechanism that masks or encrypts the sensitive data.

 Solución:

No registrar información confidencial. Si el registro es necesario, utilizar un mecanismo de registro seguro que enmascare o cifre los datos confidenciales.

Weak Hashing Algorithm

HIGH

 Línea: 68

 Descripción: The 'hash' method uses the MD5 hashing algorithm, which is considered weak and vulnerable to collisions. This is a security risk.

 Categoría: Reliability

 Esfuerzo: 60 min

 Recomendación: Use a stronger hashing algorithm, such as SHA-256 or SHA-3.

 Solución:

Reemplazar MD5 con un algoritmo de hash más fuerte, como SHA-256 o SHA-3.

Weak Encryption Algorithm

HIGH

 Línea: 74

 Descripción: The 'encrypt' method uses AES encryption with a hardcoded key and without an initialization vector (IV) or GCM. This is a security risk.

 Categoría: Reliability

 Esfuerzo: 120 min

 Recomendación: Use a strong encryption algorithm with a randomly generated key and IV or GCM. Store the key securely.

 Solución:

Utilizar un algoritmo de cifrado fuerte con una clave generada aleatoriamente y un IV o GCM. Almacenar la clave de forma segura.

Stacktrace Exposure

MEDIUM

 Línea: 83

 Descripción: The 'error' method returns the stack trace of the exception to the client. This can expose sensitive information about the application's internal workings.

 Categoría: Reliability

 Esfuerzo: 30 min

 Recomendación: Return a generic error message to the client and log the exception details on the server.

 Solución:

Devolver un mensaje de error genérico al cliente y registrar los detalles de la excepción en el servidor.

Unnecessary Imports

LOW

 Línea: 3-9

 Descripción: The code imports several classes that are not used, such as 'javax.crypto.Cipher', 'javax.crypto.spec.SecretKeySpec', 'javax.script.ScriptEngine', 'javax.script.ScriptEngineManager', 'java.io.*', 'java.nio.file.*', 'java.security.MessageDigest', 'java.sql.*'.

 Categoría: Maintainability

🕒 Esfuerzo: 15 min

💡 Recomendación: Remove unused imports to improve code readability and reduce dependencies.

🔧 Solución:

Eliminar las importaciones no utilizadas.

📄 ../demo/src/main/java/com/demo/models/WelcomeDTO.java

JAVA

🔒 Análisis de Seguridad

Insufficient Input Validation

LOW

📍 Ubicación: 4, 8, 12

📖 Descripción: The `WelcomeDTO` class lacks input validation for the `message` field. While this class itself doesn't directly interact with user input, it's a data transfer object (DTO) and the `message` field could potentially be populated with data from an untrusted source elsewhere in the application. Without validation, this data could contain malicious content, leading to vulnerabilities like XSS if the `message` is later displayed in a web page without proper encoding.

⚡ Impacto: Potential XSS vulnerability if the `message` is displayed in a web page without proper encoding. Denial of service if excessively long strings are used.

💡 Recomendación: Implement input validation and sanitization for the `message` field. Consider using a library like OWASP Java HTML Sanitizer to prevent XSS. If the message is intended to be plain text, ensure it only contains allowed characters.

🔧 Solución:

```
```java
package com.demo.models;

import org.owasp.html.PolicyFactory;
import org.owasp.html.Sanitizers;

public class WelcomeDTO {
 private String message;
 private static final PolicyFactory sanitizer = Sanitizers.FORMATTING.and(Sanitizers.LINKS);

 public WelcomeDTO(String message) {
 this.message = sanitizeMessage(message);
 }

 public String getMessage() {
 return message;
 }

 public void setMessage(String message) {
 this.message = sanitizeMessage(message);
 }

 private String sanitizeMessage(String message) {
 if (message == null) {
 return ""; // Or throw an exception, depending on requirements
 }
 return sanitizer.sanitize(message);
 }
}
```
```

⚡ Análisis de Calidad

Code Smell: Data Class

LOW

📍 Línea: 1

 Descripción: The `WelcomeDTO` class is a simple data holder with only a field and getter/setter methods. While not inherently bad, it might indicate an Anemic Domain Model if used extensively without behavior.

 Categoría: Maintainability

 Esfuerzo: 5 min

 Recomendación: Consider if this class should have any behavior related to the message it holds. If it's purely a data transfer object, it's acceptable, but be mindful of potential Anemic Domain Model issues in the larger context.

 Solución:

`N/A - Conceptual design consideration`

Lack of Immutability

LOW

 Línea: 4

 Descripción: The `message` field is mutable due to the presence of a setter method. For simple DTOs, immutability can improve thread safety and predictability.

 Categoría: Reliability

 Esfuerzo: 5 min

 Recomendación: If the message should not be changed after creation, remove the `setMessage` method and make the `message` field final.

 Solución:

```
```java
public class WelcomeDTO {
 private final String message;
 public WelcomeDTO(String message) {
 this.message = message;
 }

 public String getMessage() {
 return message;
 }

 // Remove setMessage method
}
```
```

Missing Javadoc

LOW

 Línea: 1

 Descripción: The class and its methods lack Javadoc comments. This reduces readability and maintainability.

 Categoría: Documentation

 Esfuerzo: 10 min

 Recomendación: Add Javadoc comments to the class and its methods, explaining their purpose and usage.

 Solución:

```
```java
/**
 * Data Transfer Object for welcome messages.
 */
public class WelcomeDTO {
 /**
 * The welcome message.
 */
 private String message;

 /**
 * Constructs a new WelcomeDTO with the given message.
 * @param message The welcome message.
 */
 public WelcomeDTO(String message) {
 this.message = message;
 }
}
```

```

/**
 * Returns the welcome message.
 * @return The welcome message.
 */
public String getMessage() {
 return message;
}

/**
 * Sets the welcome message.
 * @param message The welcome message.
 */
public void setMessage(String message) {
 this.message = message;
}
}
...

```

../demo/src/test/java/com/demo/controller/WelcomeControllerTests.java

JAVA

## 🔒 Análisis de Seguridad

### Insufficient Input Validation

LOW

📍 Ubicación: 24

📄 Descripción: The 'welcomeEndpointReturnsCustomMessage' test case uses a parameter 'name' without any validation. While this is a test, it highlights a potential vulnerability in the actual 'WelcomeController' if it doesn't properly sanitize or validate the 'name' parameter. If the 'WelcomeController' directly uses this parameter in a response without proper encoding, it could lead to XSS.

💣 Impacto: Low in this test case, but potentially Medium in the actual controller if the input is not validated, leading to XSS.

💡 Recomendación: Ensure that the 'WelcomeController' properly validates and sanitizes the 'name' parameter to prevent XSS or other injection attacks. Use appropriate encoding techniques when displaying the 'name' parameter in the response.

🔧 Solución:

```

In the WelcomeController:
'''java
import org.springframework.web.util.HtmlUtils;

// ...
String sanitizedName = HtmlUtils.htmlEscape(name); // Escape HTML entities
String message = "Hola, bienvenido " + sanitizedName + ", esto es un demo";
// ...
'''

```

## ⚡ Análisis de Calidad

### Duplication of Code

MEDIUM

📍 Línea: 25, 33

📄 Descripción: The structure of the two test methods 'welcomeEndpointReturnsDefaultMessage' and 'welcomeEndpointReturnsCustomMessage' is highly similar, leading to code duplication. The 'mockMvc.perform()', 'status().isOk()', and 'content().contentType()' calls are repeated.

🏷️ Categoría: Maintainability

⌚ Esfuerzo: 30 min

💡 Recomendación: Refactor the common parts of the tests into a helper method to reduce duplication and improve readability and maintainability.

 Solución:

```
```java
private void performWelcomeTest(String nameParam, String expectedMessage) throws Exception {
    MockHttpServletRequestBuilder requestBuilder = get("/api/welcome");
    if (nameParam != null) {
        requestBuilder = requestBuilder.param("name", nameParam);
    }

    mockMvc.perform(requestBuilder)
        .andExpect(status().isOk())
        .andExpect(content().contentType("application/json"))
        .andExpect(jsonPath("$.message").value(expectedMessage));
}

@Test
public void welcomeEndpointReturnsDefaultMessage() throws Exception {
    performWelcomeTest(null, "Hola, bienvenido ..., esto es un demo");
}

@Test
public void welcomeEndpointReturnsCustomMessage() throws Exception {
    performWelcomeTest("Juan", "Hola, bienvenido Juan, esto es un demo");
}
```
```

#### Lack of Assertions

LOW

 Línea: 26, 34

 Descripción: While the tests check the HTTP status and content type, they could benefit from additional assertions to ensure the controller behaves as expected under different circumstances. For example, checking for specific headers or error conditions.

 Categoría: Reliability

 Esfuerzo: 15 min

 Recomendación: Add more specific assertions to cover different scenarios and edge cases. Consider adding assertions for headers, error messages, or other relevant aspects of the response.

 Solución:

```
```java
// Example: Adding a header assertion
.andExpect(header().string("Content-Language", "es"));
```
```

#### Magic String

LOW

 Línea: 27, 35

 Descripción: The expected message strings are hardcoded within the tests. This makes the tests brittle and harder to maintain if the message changes in the controller.

 Categoría: Maintainability

 Esfuerzo: 20 min

 Recomendación: Extract the expected message strings into constants or use a more flexible approach to define the expected values, such as reading them from a configuration file or using a dedicated test data provider.

 Solución:

```
```java
private static final String DEFAULT_MESSAGE = "Hola, bienvenido ..., esto es un demo";
private static final String CUSTOM_MESSAGE = "Hola, bienvenido Juan, esto es un demo";

@Test
public void welcomeEndpointReturnsDefaultMessage() throws Exception {
    mockMvc.perform(get("/api/welcome"))
        .andExpect(status().isOk())
        .andExpect(content().contentType("application/json"))
        .andExpect(jsonPath("$.message").value(DEFAULT_MESSAGE));
}
```
```

```
@Test
public void welcomeEndpointReturnsCustomMessage() throws Exception {
 mockMvc.perform(get("/api/welcome").param("name", "Juan"))
 .andExpect(status().isOk())
 .andExpect(content().contentType("application/json"))
 .andExpect(jsonPath("$.message").value(CUSTOM_MESSAGE));
}
...

```

#### Lack of Test Data Variation

LOW

📌 Línea: N/A

📖 Descripción: The tests only cover a single positive scenario for each endpoint. There's no testing of edge cases, invalid input, or error conditions.

🏷️ Categoría: Reliability

🕒 Esfuerzo: 45 min

💡 Recomendación: Add more test cases to cover different scenarios, including invalid input, missing parameters, and potential error conditions. Consider using parameterized tests to easily test multiple input values.

🔧 Solución:

```
```java
// Example: Adding a test for missing parameter
@Test
public void welcomeEndpointReturnsErrorForMissingName() throws Exception {
    mockMvc.perform(get("/api/welcome").param("name", ""))
        .andExpect(status().isOk())
        .andExpect(content().contentType("application/json"))
        .andExpect(jsonPath("$.message").value("Hola, bienvenido , esto es un demo"));
}
...

```

Missing Javadoc

LOW

📌 Línea: 13

📖 Descripción: The test class and methods lack Javadoc comments, making it harder to understand their purpose and functionality.

🏷️ Categoría: Documentation

🕒 Esfuerzo: 15 min

💡 Recomendación: Add Javadoc comments to the test class and methods to explain their purpose and how they work.

🔧 Solución:

```
```java
/**
 * Tests for the {@link WelcomeController}.
 */
@WebMvcTest(WelcomeController.class)
public class WelcomeControllerTests {

 /**
 * Injected MockMvc instance for simulating HTTP requests.
 */
 @Autowired
 private MockMvc mockMvc;

 /**
 * Tests that the welcome endpoint returns the default message.
 * @throws Exception if an error occurs during the test.
 */
 @Test
 public void welcomeEndpointReturnsDefaultMessage() throws Exception {
 ...
 }

 ...
}
...

```

demo/pom.xml

POM

## 🔒 Análisis de Seguridad - Maven POM

### Outdated Dependency CVE-2019-14892, CVE-2019-14540, CVE-2018-11307, etc. (multiple)

HIGH

Ubicación: 31

**Descripción:** The 'jackson-databind' dependency is using version 2.9.8, which is known to have multiple critical security vulnerabilities (CVEs). This version is severely outdated.

**Impacto:** Remote code execution, denial of service, information disclosure. An attacker could potentially exploit these vulnerabilities to compromise the application and the server it runs on.

**Recomendación:** Upgrade 'jackson-databind' to the latest stable version (at least 2.16.0 or higher). Check for compatibility with other dependencies after upgrading.

**Dependencia:** `com.fasterxml.jackson.core:jackson-databind`

**Solución:**

```
com.fasterxml.jackson.core
jackson-databind
2.16.1
```

### Snapshot Dependency in Production

MEDIUM

Ubicación: 10

**Descripción:** The project version is set to '0.0.1-SNAPSHOT'. Snapshot versions are intended for development and should not be used in production deployments. Snapshot dependencies can change unpredictably, leading to inconsistent and potentially broken builds in production.

**Impacto:** Unpredictable behavior in production, potential for broken builds, difficulty in reproducing issues.

**Recomendación:** Remove the '-SNAPSHOT' suffix from the version before deploying to production. Use a proper release versioning scheme (e.g., 1.0.0, 1.0.1).

**Dependencia:** `com:demo`

**Solución:**

```
0.0.1
```

### Missing License Information

LOW

Ubicación: 14

**Descripción:** The '' section is present but empty. This makes it difficult for users to understand the terms under which they can use the software. While not a direct security vulnerability, it can lead to legal issues and discourage adoption.

**Impacto:** Legal issues, discourages adoption.

**Recomendación:** Add the appropriate license information to the '' section. Specify the license name, URL, and distribution type.

**Solución:**

```
Apache License, Version 2.0
http://www.apache.org/licenses/LICENSE-2.0.txt
```

```
repo
```

#### Missing SCM Information

LOW

Ubicación: 17

**Descripción:** The `` section is present but empty. This makes it difficult to track the source code repository and contribute to the project. While not a direct security vulnerability, it hinders collaboration and transparency.

**Impacto:** Hindered collaboration, reduced transparency.

**Recomendación:** Add the appropriate SCM information to the `` section. Specify the connection URL, developer connection URL, tag, and URL.

**Solución:**

```
scm:git:git://github.com/example/demo.git
scm:git:ssh://git@github.com/example/demo.git
HEAD
https://github.com/example/demo
```

#### Missing Version Control for Plugins

LOW

Ubicación: 41

**Descripción:** The `spring-boot-maven-plugin` is used without specifying a version. This can lead to unpredictable builds if the plugin is updated automatically by Maven. It's best practice to explicitly define the version of all plugins.

**Impacto:** Unpredictable builds, potential for broken builds.

**Recomendación:** Specify the version of the `spring-boot-maven-plugin` in the `` section.

**Dependencia:**

```
org.springframework.boot:spring-boot-maven-plugin
```

**Solución:**

```
org.springframework.boot
spring-boot-maven-plugin
3.4.5
```

#### Dependencias Desactualizadas

com.fasterxml.jackson.core:jackson-databind

OUTDATED

Versión Actual: 2.9.8

Versión Recomendada: 2.16.1

Riesgo de Seguridad: HIGH